



UNIVERSIDAD TÉCNICA DEL NORTE

FACULTAD DE INGENIERÍA EN CIENCIAS APLICADAS

CARRERA DE INGENIERÍA DE TELECOMUNICACIONES

TEMA:

“SISTEMA DE MONITOREO CON ARQUITECTURA IOT Y
PROCESAMIENTO DE VIDEO PARA LA SEGURIDAD DE
PLANTACIONES DE AGUACATE DEL SECTOR SAN JOSÉ, MIRA”

**Trabajo de titulación previo a la obtención del título de Ingeniero en
Telecomunicaciones**

Línea de investigación: Desarrollo, aplicación de software y cyber security

AUTOR:

Alder Stalin Navarrete tipas

DIRECTOR:

Ing. Jaime Roberto Michilena Calderón, Msc

Ibarra-Ecuador 2026



UNIVERSIDAD TÉCNICA DEL NORTE

DIRECCIÓN DE BIBLIOTECA

AUTORIZACIÓN DE USO Y PUBLICACIÓN A FAVOR DE LA UNIVERSIDAD TÉCNICA DEL NORTE

1. IDENTIFICACIÓN DE LA OBRA

En cumplimiento del Art. 144 de la Ley de Educación Superior, hago la entrega del presente trabajo a la Universidad Técnica del Norte para que sea publicado en el Repositorio Digital Institucional, para lo cual pongo a disposición la siguiente información:

DATOS DE CONTACTO	
APELLIDOS Y NOMBRES:	NAVARRETE TIPAS ALDER STALIN

DATOS DE LA OBRA	
TÍTULO:	"SISTEMA DE MONITOREO CON ARQUITECTURA IOT Y PROCESAMIENTO DE VIDEO PARA LA SEGURIDAD DE PLANTACIONES DE AGUACATE DEL SECTOR SAN JOSÉ, MIRA"
AUTOR (ES):	ALDER STALIN NAVARRETE TIPAS
FECHA: DD/MM/AAAA	31/08/2026
SOLO PARA TRABAJOS DE GRADO	
PROGRAMA:	☒ PREGRADO ☐ POSGRADO
TÍTULO POR EL QUE OPTA:	INGENIERO EN TELECOMUNICACIONES
DIRECTOR /ASESOR:	ING. JAIME ROBERTO MICHILENA CALDERÓN, MSC ING. SUÁREZ ZAMBRANO LUIS EDILBERTO, MSC

2. CONSTANCIAS

El autor manifiesta que la obra objeto de la presente autorización es original y se la desarrolló, sin violar derechos de autor de terceros, por lo tanto, la obra es original y que es el titular de los derechos patrimoniales, por lo que asume la responsabilidad sobre el contenido de la misma y saldrá en defensa de la Universidad en caso de reclamación por parte de terceros.

Ibarra, a los 31 días del mes de agosto de 2026

EL AUTOR:

(Firma).....

Nombre: Navarrete Tipas Alder Stalin

CERTIFICACIÓN DEL DIRECTOR DEL TRABAJO DE INTEGRACIÓN CURRICULAR

Ibarra, 31 de agosto de 2026

ING. JAIME ROBERTO MICHILENA CALDERÓN, MSC

DIRECTOR DEL TRABAJO DE INTEGRACIÓN CURRICULAR

CERTIFICA:

Haber revisado el presente informe final del trabajo de Integración Curricular, el mismo que se ajusta a las normas vigentes de la Universidad Técnica del Norte; en consecuencia, autorizo su presentación para los fines legales pertinentes.

(f)
Ing. Jaime Roberto Michilena Calderón, Msc
C.C.: 1002198438

DEDICATORIA

A mi familia, porque son la base fundamental y el principal componente que me ha acompañado con el tiempo, con cada noche de estudio, con cada esfuerzo, con cada obstáculo sorteado a lo largo de mi formación académica.

A mis padres, de una manera muy especial, por su amor incondicional y porque me han enseñado que con disciplina, con paciencia y con trabajo se logran los más altos objetivos de la vida. Gracias por no dejar que me falte nada y por creer siempre en mí.

La realización de esta tesis, así como la finalización de mi carrera profesional, están dedicadas también para ustedes, pues este logro es también suyo, y es tan mío como de ustedes.

Alder Stalin Navarrete Tipas

AGRADECIMIENTO

Expreso mi más sincero agradecimiento a la Universidad Técnica del Norte y a la Carrera de Ingeniería de Telecomunicaciones por haberme brindado una formación académica y profesional de excelencia.

De manera especial, agradezco a mi director de tesis, el MSc. Jaime Roberto Michilena Calderón, y a mi asesor, el Ing. Luis Edilberto Suárez Zambrano, por su paciencia, su valiosa guía y por compartir sus conocimientos técnicos, los cuales fueron fundamentales para la consecución de este proyecto.

Finalmente, extiendo mi gratitud a los moradores del sector San José, por su apertura, confianza y colaboración desinteresada durante las fases de investigación y validación en campo de este trabajo.

Alder Stalin Navarrete Tipas

RESUMEN EJECUTIVO

El presente trabajo de integración curricular da cuenta del diseño e implementación de un sistema de monitoreo autónomo basado en la arquitectura de Internet de las Cosas (IoT, por su sigla en inglés) y procesamiento en el borde (Edge Computing), teniendo presente las crecientes cifras de intrusiones en las plantaciones de aguacate del sector San José, cantón Mira. En efecto, el interés principal se encuentra en promover una solución tecnológica proactiva y autodidacta de procesamiento de algoritmos de visión artificial, localmente, que permita la detección de personas y de vehículos, evitando las dificultades propias de conectividad e infraestructura eléctrica de las áreas rurales. Con este fin, la metodología adoptada tiene un enfoque de tipo cascada, dado que se llevó a cabo tanto el diseño de un nodo de seguridad alimentado por un sistema fotovoltaico como la ejecución de redes neuronales convolucionales (YOLOv8n y EasyOCR) en un microcomputador Raspberry Pi 5. Los resultados de las pruebas de campo con respecto al prototipo indican que el consumo de datos de transmisión de evidencias fotográficas es sostenido al transmitir únicamente imágenes fotográficas estáticas a través de la API de Telegram, evitando la saturación del enlace LTE. A su vez, los resultados mostraron que este modelo es capaz de filtrar las falsas alarmas por efectos ambientales y de notificar al usuario inmediatamente. El sistema demuestra que la integración de Edge AI e IoT puede realizarse de una manera adecuada y necesaria para la seguridad agrícola, manteniendo un tiempo de respuesta menor de 10 segundos y entregando al agricultor una alternativa de vigilancia que protege la producción de ingresos no autorizados.

Palabras clave: Agricultura de Precisión, Edge Computing, Internet de las Cosas, Seguridad Perimetral, Visión Artificial, YOLOv8n.

ABSTRACT

The work of curricular integration proposes an autonomous monitoring system built on IoT (Internet of Things) technology, which aims to address the increasing intrusion issues in avocado plantations in San José, located in the Mira canton. The basic aim is to deliver the technological solution of the local usage of computer vision algorithms for detection of vehicles and people while getting rid of problematic connections and electricity sources existing in rural areas. A waterfall approach was applied, which allowed developing a security node optimized to work with independently operating photovoltaic supplies. After finishing the prototype testing in practical conditions, it was approved by the first field studies that the data sending through Telegram in the format of static photos turned out to save dramatically the amount of data sent through LTE. It was found out that this technology efficiently excluded false calls from environmental reasons. To sum up, this project shows the highest viability of Edge AI and IoT combination in agricultural monitoring with fast response times.

Keywords: Computer Vision, Edge Computing, Internet of Things, Perimeter Security, Precision Agriculture, YOLOv8n.

LISTA DE SIGLAS

API. Application Programming Interface o Interfaz de Programación de Aplicaciones

CGNAT. Carrier-Grade Network Address Translation

CNN. Convolutional Neural Network o Red Neuronal Convolucional

CSI. Camera Serial Interface o Interfaz Serial de Cámara

FPS. Frames Per Second o Fotogramas por Segundo

GPIO. General Purpose Input/Output o Entrada/Salida de Propósito General

HTTP. Hypertext Transfer Protocol o Protocolo de Transferencia de Hipertexto

HTTPS. Hypertext Transfer Protocol Secure o P. Seguro de Transferencia de Hipertexto

IoT. Internet of Things o Internet de las Cosas

IP. Internet Protocol o Protocolo de Internet

LED. Light-Emitting Diode o Diodo Emisor de Luz

LTE. Long Term Evolution o Evolución a Largo Plazo

OCR. Optical Character Recognition o Reconocimiento Óptico de Caracteres

ODS. Objetivos de Desarrollo Sostenible

PIR. Passive Infrared o Infrarrojo Pasivo

PWM. Pulse Width Modulation o Modulación por Ancho de Pulsos

RAM. Random Access Memory o Memoria de Acceso Aleatorio

RSRP. Reference Signal Received Power o Potencia de Señal de Referencia Recibida

RSRQ. Reference Signal Received Quality o Calidad de Señal de Referencia Recibida

SINR. Signal to Interference plus Noise Ratio o Relación Señal a Interferencia más Ruido

SoC. System on a Chip o Sistema en un Chip

SSID. Service Set Identifier o Identificador de Conjunto de Servicios

WLAN. Wireless Local Area Network o Red de Área Local Inalámbrica

YOLO. You Only Look Once o Solo Miras Una Vez

ÍNDICE DE CONTENIDOS

1. CAPITULO I: ANTECEDENTES.....	20
1.1. Tema.....	20
1.2. Problema.....	20
1.3. Objetivos	22
1.4. Alcance.....	23
1.5. Justificación.....	25
2. CAPÍTULO II: FUNDAMENTOS TEORICOS	28
2.1 Arquitectura de sistemas IoT aplicados a zonas rurales y agrícolas.....	28
2.1.1 Conceptos básicos de IoT y su aplicación.....	29
2.1.2 Arquitectura de un sistema de seguridad para entornos rurales basada en las capas fundamentales del modelo IoT.....	31
2.2 Capa de Percepción: Captura de datos en el entorno agrícola.....	34
2.2.1 Sensores de movimiento PIR, infrarrojos y magnéticos para el monitoreo perimetral en entornos agrícolas.....	35
2.2.2 Cámaras IP para vigilancia en caminos, accesos y cultivos.....	36
2.2.3 Comparación con métodos tradicionales de vigilancia en el medio rural.....	38
2.2.4 Condiciones técnicas en zonas agrícolas: iluminación, clima, polvo y obstáculos físicos.	39
2.3 Capa de Red: Conectividad en áreas rurales	41
2.3.1 Tecnologías de comunicación adaptadas a zonas rurales.....	41
2.3.2 Protocolos de red aplicados a sistemas de seguridad agrícola	44
2.3.3 Retos de cobertura y fiabilidad: caminos dispersos, distancias largas y señal limitada	47

2.4 Capa de Procesamiento: Análisis de información para seguridad en el campo	48
2.4.1 Visión por computadora aplicada a vigilancia agrícola	52
2.4.2 Procesamiento de imágenes y video capturados en ambientes rurales	53
2.4.3 Detección de movimiento y análisis en tiempo real para prevención de robos	55
2.4.4 Reconocimiento de vehículos y matrículas en caminos agrícolas	56
2.4.5 Herramientas y modelos preentrenados utilizados.....	58
2.4.6 Desafíos técnicos: resolución, ángulos de captura, conectividad y entorno natural ...	59
2.5 Capa de Aplicación: Monitoreo y respuesta en zonas agrícolas.....	60
2.5.1 Sistemas de videovigilancia integrados a IoT en fincas y zonas rurales.....	61
2.5.2 Almacenamiento de datos: soluciones locales y en la nube adaptadas al entorno rural	63
2.5.3 Tipos de alertas útiles para propietarios agrícolas	64
2.5.4 Comunicación en tiempo real desde zonas alejadas o con baja conectividad.....	65
2.5.5 Protocolos de respuesta ante intrusos o emergencias en terrenos agrícolas.....	66
2.6 Análisis comparativo de tecnologías de hardware para el sistema de seguridad	67
2.6.1 Comparativa de unidades de procesamiento (Edge Computing){	68
2.6.2 Comparativa de módulos de adquisición de imagen.....	69
3.CAPÍTULO III: DISEÑO DEL SISTEMA DE MONITOREO.....	71
3.1. Metodología	71
3.2. Etapa de Análisis.....	73
3.2.1. Situación Actual y problemática en el sector San José	74
3.3. Etapa de Planificar	76
3.3.1. Identificación de Actores y Stakeholders.....	76
3.3.2. Nomenclatura de Requerimientos	77
3.3.3. Requerimiento de Stakeholders.....	81

3.3.4. Requerimientos del sistema.....	82
3.3.5. Requerimientos de Arquitectura.....	85
3.3.6. Selección de Hardware.....	87
3.3.7. Selección de Software	96
3.4. Etapa de Diseño.....	99
3.4.1. Diagrama de bloques del Sistema	99
3.4.2. Diseño y Descripción del Sistema de Monitoreo	101
3.4.3. Diagrama eléctrico y esquemático de conexiones.....	103
3.5. Desarrollo de Software y Algoritmos de Procesamiento	107
3.5.1. Inicialización y Control.....	107
3.5.2. Integración en la Nube y Desarrollo de la Interfaz Web.....	115
3.5.3. Gestión Criptográfica y Variables de Entorno (Secrets).....	123
3.6. Almacenamiento de Datos.....	127
3.6.1 Almacenamiento de Datos Local	127
3.6.2. Almacenamiento de Datos Web (Nube).....	128
3.7. Integración y pruebas locales.....	129
3.7.1. Configuración del enrutador LTE	129
3.7.2. Depuración de arranque de la Raspberry Pi 5	130
3.7.3. Validación del sistema de alertas y visualización de datos.....	132
3.7.3.1. Inferencia y notificaciones en tiempo real	132
3.7.4 Despliegue del panel de control web.....	134
4.CAPÍTULO IV: IMPLEMENTACION Y PRUEBAS FUNCIONALES.....	136
4.1. Implementación Estructural del Sistema.....	136
4.2. Implementación Electrónica del Sistema.....	139
4.2.1. Subsistema de Procesamiento	139

4.2.2. Subsistema de Detección y Actuación	140
4.3. Implementación del Sistema Tecnológico y de Comunicaciones.....	141
4.4. Pruebas de Hardware y Software.....	142
4.4.1. Cronograma de Pruebas	142
4.4.2. Pruebas de Acoplamiento de los Componentes	143
4.4.3. Prueba de Funcionamiento del Nodo central	147
4.4.5. Prueba del Flujo de Almacenamiento Local y en la nube.....	152
4.4.6. Seguridad de la Conexión y Privacidad	153
4.5. Etapa de Evaluación de Resultados.....	156
4.5.1. Análisis de la Seguridad Tradicional	156
4.5.2. Análisis del Sistema de Seguridad Automático	156
4.6. Costos del Sistema.....	159
4.6.1. Costo de Hardware y Procesamiento	159
4.6.2. Costo de Infraestructura y Energía.....	160
4.6.3. Costos de Ingeniería y Software.....	161
4.6.4. Costo Global del Sistema	162
4.6.5. Costos Operativos y de Mantenimiento	163
4.7. Beneficios del Sistema.....	163
CONCLUSIONES.....	165
RECOMENDACIONES.....	166
REFERENCIAS BIBLIOGRÁFICAS.....	167
ANEXOS.....	171

Código Fuente del Nodo de Edge Computing (monitoreo_yolo5.py).....	171
Configuración del Servicio de Arranque Automático (Systemd)	181
Código Fuente de la Interfaz Web (app.py)	182
Estructura de Credenciales y Variables de Entorno (Secrets)	189

ÍNDICE DE TABLAS

Tabla 1. Tecnologías Inalámbricas.....	43
Tabla 2. Comparativa de placas de procesamiento para visión artificial en el borde.....	68
Tabla 3. Comparativa de módulos de cámara para sistemas de seguridad basados en Raspberry Pi.....	69
Tabla 4. Identificación de Stakeholders	77
Tabla 5. Nomenclatura de Requerimientos	78
Tabla 6. Priorización de los Requerimientos del Sistema	78
Tabla 7. Requerimientos Operacionales.....	80
Tabla 8. Requerimientos del Usuario	80
Tabla 9. Requerimiento de Stakeholders.....	81
Tabla 10. Requerimientos del sistema (SySR).....	83
Tabla 11. Requerimientos Técnicos Consolidados	84
Tabla 12. Requerimientos de Arquitectura	86
Tabla 13. Selección de Sensor de Movimiento	87
Tabla 14. Selección de Cámara de Video	88
Tabla 15. Selección de Microcomputadora.....	89
Tabla 16. Selección de Fuente de Alimentación	90
Tabla 17. Consumo Eléctrico Nodo de Seguridad y Componentes	92
Tabla 18. Selección del Lenguaje de Programación	96
Tabla 19. Selección de Algoritmo de Detección.....	97
Tabla 20. Selección de Software OCR.....	97
Tabla 21. Selección de Plataforma de Notificaciones.....	98
Tabla 22. Cronograma de Pruebas	142

Tabla 23.Pruebas de Acoplamiento de los Componentes	143
Tabla 24. Análisis Comparativo de Parámetros RF LTE en el Sector San José	151
Tabla 25. Evaluación de eventos de detección.....	157
Tabla 26 .Costos de Hardware y Procesamiento.....	160
Tabla 27.Costo de Infraestructura y Energía.....	161
Tabla 28.Costos de Ingeniería y Software	162
Tabla 29. Costo Global del Sistema	162
Tabla 30. Costos Operativos y de Mantenimiento	163

ÍNDICE DE FIGURAS

<i>Figura 1. Árbol de Problemas.....</i>	<i>21</i>
<i>Figura 2. Arquitectura.....</i>	<i>25</i>
<i>Figura 3. Fases de la Arquitectura.....</i>	<i>34</i>
<i>Figura 4. Análisis del movimiento y detección.....</i>	<i>51</i>
<i>Figura 5. Metodología en cascada aplicada al sistema de monitoreo IoT.....</i>	<i>72</i>
<i>Figura 6. Ubicación de la zona de implementación del prototipo en el Sector San José.....</i>	<i>75</i>
<i>Figura 7. Sector San José.</i>	<i>75</i>
<i>Figura 8. Arducam IMX477 HQ (12MP)</i>	<i>89</i>
<i>Figura 9. Raspberry pi 5 placa.....</i>	<i>90</i>
<i>Figura 10. Reflector LED de 12V DC para iluminación nocturna.....</i>	<i>91</i>
<i>Figura 11. Módulo de relé para control del reflector desde la Raspberry Pi.</i>	<i>92</i>
<i>Figura 12. Batería de Gel Ciclo Profundo 12V 50Ah</i>	<i>94</i>
<i>Figura 13. Controlador de carga PWM registrando los parámetros de tensión.</i>	<i>95</i>
<i>Figura 14. Panel Solar 100 w</i>	<i>95</i>
<i>Figura 15. Diagrama de bloques.....</i>	<i>99</i>
<i>Figura 16. Topología del Sistema de Monitoreo Edge IoT.....</i>	<i>102</i>
<i>Figura 17. Diagrama de Conexiones Eléctricas y Esquemático del Nodo Central.....</i>	<i>104</i>
<i>Figura 18. Diagrama de Flujo Funcionamiento.....</i>	<i>105</i>
<i>Figura 19. Importación de Librerías y Dependencias del Sistema.....</i>	<i>108</i>
<i>Figura 20. Configuración de Variables Globales y Credenciales.....</i>	<i>108</i>
<i>Figura 21. Inicialización de Hardware y Modelos de Inteligencia Artificial</i>	<i>109</i>
<i>Figura 22. Funciones Auxiliares de Almacenamiento, Notificación y Sincronización.....</i>	<i>110</i>
<i>Figura 23. Bucle Principal de Monitoreo, Activación de Hardware e Inferencia</i>	<i>111</i>
<i>Figura 24. Procesamiento Digital de Vehículos y Extracción de Caracteres (OCR)</i>	<i>112</i>
<i>Figura 25. Filtrado de Personas, Manejo de Falsas Alarmas y Excepciones</i>	<i>113</i>
<i>Figura 26. Bloque de Seguridad y Cierre Controlado.....</i>	<i>114</i>
<i>Figura 27. Registro de Arranque y Operación del Sistema en Consola</i>	<i>115</i>

<i>Figura 28.</i>	116
<i>Figura 29. Configuración de la Cuenta de Servicio en Google Cloud</i>	117
<i>Figura 30. Repositorio del Proyecto Web en GitHub</i>	118
<i>Figura 31. Archivo de Dependencias del Servidor (requirements.txt)</i>	119
<i>Figura 32. Lógica de Autenticación y Peticiones REST a Google Drive</i>	120
<i>Figura 33. Lógica de Navegación en Cascada y Renderizado de Interfaz</i>	121
<i>Figura 34. Despliegue de la Aplicación en Streamlit Community Cloud</i>	122
<i>Figura 35. Interfaz Final de Usuario y Barrera de Acceso Restringido</i>	123
<i>Figura 36. Generación de Claves de Autenticación en Google Cloud</i>	124
<i>Figura 37. Estructura Interna del Archivo de Credenciales JSON</i>	125
<i>Figura 38. Inyección de Variables de Entorno en Streamlit Community Cloud</i>	126
<i>Figura 39. Estructura de Directorios para el Almacenamiento Local en la Raspberry Pi</i>	128
<i>Figura 40. Sincronización Cronológica de Evidencias en Google Drive</i>	129
<i>Figura 41. Configuración de credenciales de administrador en enrutador D-Link</i>	129
<i>Figura 42. Configuración de red WLAN SSID y seguridad</i>	130
<i>Figura 43. Edición de parámetros de arranque en cmdline.txt</i>	131
<i>Figura 44. Enmascaramiento de servicios de emergencia mediante systemctl</i>	132
<i>Figura 45. Ejecución del script de inferencia y registro de eventos en consola</i>	133
<i>Figura 46. Notificación, Reconocimiento vehicular y lectura de placa a través de Telegram</i>	134
<i>Figura 47. Panel de control web para la visualización de registros almacenados en la nube</i>	135
<i>Figura 48. Estructura metálica elevada para la protección del nodo principal</i>	137
<i>Figura 49. Instalación del panel solar monocristalino en la estación de energía</i>	138
<i>Figura 50. Componentes electrónicos y ópticos confinados en el gabinete IP65</i>	139
<i>Figura 51. Conexión del módulo reductor de voltaje a la Raspberry Pi 5 para alimentación de 25W.</i>	140
<i>Figura 52. Sensor PIR HC-SR501</i>	141
<i>Figura 53. Acceso remoto al entorno de escritorio de la Raspberry Pi 5 para la configuración del sistema y gestión del modelo YOLOv8n.</i>	142
<i>Figura 54. Medición de voltaje de salida del módulo reductor hacia la Raspberry Pi 5</i>	144
<i>Figura 55. Prueba de Activación del Módulo Relé Mediante Script de Control por Consola</i>	145

<i>Figura 56. Capturas de Detección Diurna y Nocturna.....</i>	<i>146</i>
<i>Figura 57. Modificación de Parámetros del Kernel para el Reinicio Automático y Enmascaramiento de Servicios de Recuperación en Systemd para Operación Desatendida.....</i>	<i>146</i>
<i>Figura 58. Temperatura Operativa del Nodo en Estado de Carga Moderada</i>	<i>147</i>
<i>Figura 59. Consumo exclusivo de notificaciones de texto y fotografías estáticas.....</i>	<i>149</i>
<i>Figura 60. Interfaz de Diagnóstico del Enrutador D-Link G403C con Red AkiMovil.....</i>	<i>150</i>
<i>Figura 61. Conexión Estable y Parámetros Operativos en la Red Claro.....</i>	<i>151</i>
<i>Figura 62. Almacenamiento Local.....</i>	<i>153</i>
<i>Figura 63. Respaldo en la nube.....</i>	<i>153</i>
<i>Figura 64. Portal de Autenticación Segura para la Administración Remota del Nodo</i>	<i>154</i>
<i>Figura 65. Despliegue del Agente de Sincronización Segura en el Nodo de Borde.....</i>	<i>155</i>
<i>Figura 66. Panel de Acceso Restringido y Autenticación Segura para el Monitoreo Web</i>	<i>156</i>
<i>Figura 67. Detección Diurna de Maquinaria Agrícola mediante Visión Artificial.....</i>	<i>158</i>
<i>Figura 68. Detección Nocturna de Vehículo de Carga bajo Iluminación Artificial.....</i>	<i>159</i>

ÍNDICE DE ECUACIONES

Ecuación 1. Formula consumo promedio de la batería.	93
Ecuación 2. Cálculo con datos teóricos.....	93
Ecuación 3. Cálculo de dimensión Promedio Batería	94
Ecuación 4. Calculo consumo promedio de transmisión continua.....	148

1. CAPITULO I: ANTECEDENTES

1.1. Tema

Sistema de Monitoreo con Arquitectura IoT y Procesamiento de Video para la Seguridad de Plantaciones de Aguacate del sector San José, Mira

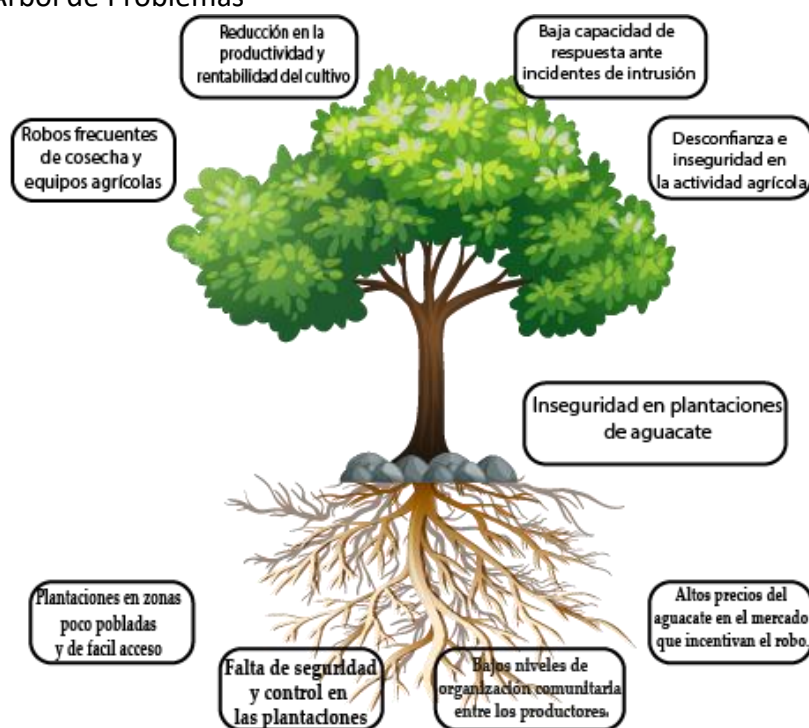
1.2. Problema

En el cantón Mira de la provincia del Carchi, durante los últimos años, se ha incrementado significativamente el cultivo de aguacate, en particular el de la variedad Hass, en gran medida debido al interés mostrado por parte del Estado ecuatoriano para diversificar la matriz productiva y hacer más fuerte la oferta exportable del país (Ministerio de Agricultura y Ganadería, 2020). Durante este tiempo, los productores locales han logrado poner en el mercado internacional el aguacate de Mira, caracterizándose por su calidad, palatabilidad y elevados estándares fitosanitarios (Herrera, 2022).

No obstante, el impulso de dicha actividad agrícola ha conllevado un problema emergente: el incremento de robos en las plantaciones. Las áreas cultivadas, muchas localizadas en los sectores rurales como el de San José, se encuentran en condiciones favorables para los accesos ilícitos, sobre todo por los caminos principales que conectan diferentes unidades productivas. La existencia de escasa vigilancia y de limitada infraestructura de seguridad ha incrementado la vulnerabilidad de las fincas que afectan directamente a los productores, que informan de pérdidas considerables en la época de cosecha (El Universo, 2024). Esta situación se encuentra representada en la Figura 1, donde se presenta un "Árbol de Problemas", en el cual se explican los factores que propiciaron la inseguridad en las plantaciones de aguacate en el sector San José-Mira. En las raíces del árbol se encuentran causas como la ausencia de vigilancia tecnológica, escasa previa

vigilancia carretera, escasa vinculación entre productores y problemas vinculados al entorno rural; después de todas estas consideraciones, toma los trabajos más relevantes como el de (Szeliski, 2011). Estas causas en definitiva alimentan el problema central que es la inseguridad en las plantaciones, lo que genera a su vez efectos visibles en la agricultura; efectos que son representados en las ramas del árbol y que ejemplifican robos a las cosechas y al equipo agrícola, escasa capacidad de reacción ante los incidentes, desconfianza en la actividad agrícola y finalmente una baja en la capacidad de producción y por último en el sostenimiento del cultivo (Mundoagro, 2017).

Figura 1. Árbol de Problemas



Nota: Diagrama de árbol explicando los problemas del proyecto

A pesar del notable potencial que tiene el sector agrícola de la parroquia de Mira, la no utilización de tecnologías avanzadas de monitoreo y control ha limitado también su capacidad de respuesta ante eventos de intrusión. La inexistencia de sistemas que sean capaces de alertar en tiempo real de cualquier actividad sospechosa, ha logrado una reducción drástica de las capacidades de acción de los agricultores y por lo tanto también limita la sostenibilidad de los cultivos. En este contexto, es necesario desarrollar soluciones tecnológicas de monitoreo y control de cultivos basadas en el uso de Internet de las Cosas (IoT) que permitan mitigar los

riesgos de inseguridad en las zonas agrícolas y adicionalmente proteger los esfuerzos y la inversión de los pequeños y medianos productores de aguacate de Mira. (Palacios, 2025)

1.3.Objetivos

Objetivo General

- Implementar un sistema de monitoreo basado en una arquitectura IoT y procesamiento de video para la detección de intrusiones y generación de alertas en plantaciones de aguacate del sector San José, Mira

Objetivos Específicos

- Llevar a cabo investigaciones respecto a los requerimientos de solución al problema y obtener datos necesarios de la zona.
- Diseñar un prototipo funcional de seguridad que pueda ser desplegado y adaptado a las condiciones del sector San José, Mira.
- Implementar el sistema de procesamiento y análisis de datos que genere alertas en tiempo real.
- Realizar pruebas de campo para evaluar la efectividad del sistema de seguridad y su capacidad de reacción ante eventos de intrusión.

1.4. Alcance

El alcance del proyecto consiste en el diseño y la implementación del prototipo de un sistema de monitoreo inteligente basado en Arquitectura IoT para la detección de intrusiones en plantaciones de aguacate en el sector Pueblo Viejo, Mira, el cuál busca mejorar la seguridad del entorno agrícola, reducir el robo de aguacate y mejorar la capacidad de respuesta a partir de la analítica de datos y la obtención de datos.

El que el proyecto se vaya ejecutando en distintas fases, va de la mano a los objetivos específicos predefinidos. (PP RAY, 2016)

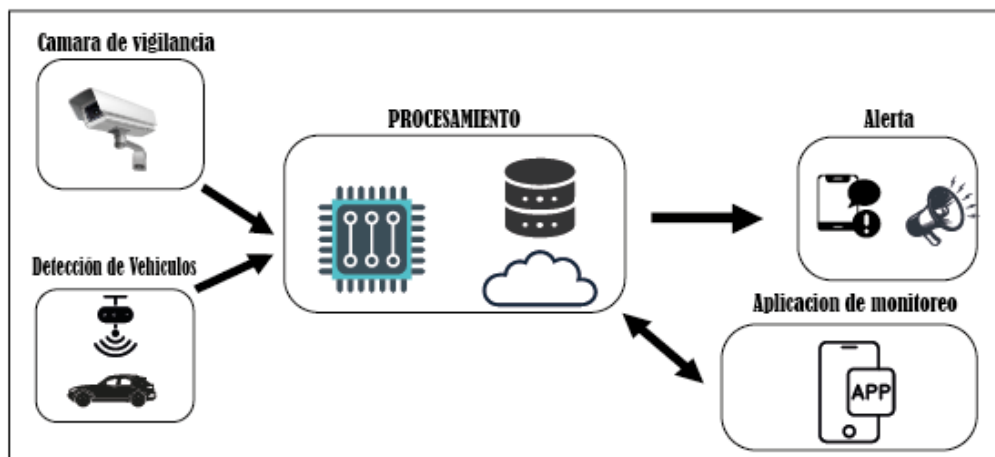
En la Fase inicial, realizaremos una investigación a fondo sobre la circunstancia actual del ámbito San José, Mira, que comprenderá el recabado de datos topográficos, la determinación de requerimientos técnicos y un estudio de campo (Sommerville, 2014). Esta etapa permitirá escoger los sensores más apropiados para detectar intrusiones, vehículos y cualquier actividad sospechosa en la vía de acceso que conecta varias parcelas agrícolas. La selección de los recursos técnicos a usar tomará en cuenta los criterios de precisión, robustez ante las condiciones ambientales del medio rural y capacidad de uso ininterrumpido. Igualmente, se valorarán aquellas posibles fuentes de alimentación eléctrica que garanticen el funcionamiento autónomo y estable del sistema de monitorización. (Gupta, 2020)

La segunda fase terminará en el diseño e implementación de un prototipo de sistema de seguridad adaptable al entorno agrícola, que integre los dispositivos necesarios para la detección y las comunicaciones IoT necesarias para la transmisión de datos y de alertas en tiempo real (Gubbi, 2013). Teniendo en cuenta las características necesarias y futuras de estos sistemas de seguridad inteligentes, el componente que destaca es capaz de registrar vehículos que transiten por una vía principal que une territorios agrícolas, registrando y

comparando los datos de identificación de los moradores locales, con la finalidad de identificar ingresos no deseados y respuestas inmediatas (Microsegur, 2022).

En la tercera fase, el prototipo será instalado y se elaborará el software necesario de control y análisis que, debiendo la información principalmente del vídeo, permita un sistema automatizado de gestión de alertas (Reina Jurado, 2023), enviándose alertas a los propietarios o responsables mediante un tipo de alertas computacional basado en los requerimientos y necesidades de los moradores, ya sea a través de mensajes de texto, llamadas, alertas estacionarias o con un volumen de sonidos. Se diseñará o utilizará, además, una aplicación que sirva o permita el seguimiento de los distintos elementos que se gestionan en el sistema, aportando en este sentido un seguimiento y monitoreo de los elementos en tiempo real, donde existen diferentes registros de movimientos o de ingresos de vehículos (Rodríguez, 2018).

Por último, se ejecutarán las pruebas de campo en plantaciones reales para probar el funcionamiento y eficacia del sistema. Con ello se van a poder ajustar el prototipo, verificar con el mismo su fiabilidad y comprobar su impacto sobre la reducción de los casos de intrusión. A partir de unos resultados validados, se propondrá un plan de mantenimiento preventivo que permita garantizar la operatividad y la durabilidad del sistema en el largo plazo. Como se puede ver en la figura 2, que presenta la arquitectura por bloques que se considera la adecuada para la implementación del sistema de seguridad basado en arquitectura IoT en el sector San José, Mira (Borgini, 2023).

Figura 2. Arquitectura

Nota: Arquitectura del funcionamiento del proyecto

Esta solución comprende la intervención de la detección de vehículos, así como la de las cámaras de vigilancia y de la vigilancia en general como unidades dependientes de captura de datos, que son recabados, como decía anteriormente, a un módulo central de procesamiento (Placa). El procesamiento de datos de vídeos se realiza en la placa (Placa) constituyendo la funcionalidad para la identificación de los vehículos que circulan por la vía agrícola y para la detección de los vehículos que no son registrados. De este modo, el sistema puede generar sus alertas y enviarlas, mediante la opción más adecuada, a los moradores, permitiendo, por lo tanto, actuar con rapidez ante cualquier amenaza. Por su parte, el módulo de visualización de las grabaciones, en complemento, permite visualizar las grabaciones, dando una opción a los responsables de contar en todo momento con la información que se refiera a las grabaciones de los vehículos generando así control y vigilancia en el ámbito de la agricultura.

1.5. Justificación

Se trata de la aplicación de un sistema inteligente de monitoreo y seguridad para las plantaciones agrícolas de aguacate en el sector San José, Mira, que surge de la imperante necesidad de proteger los recursos productivos por parte de las comunidades rurales que, enfrentan a diario las intrusiones y robos. En este ámbito se establece la correspondencia con el ODS número 11 (2015) que relacionado con el desarrollo de comunidades inclusivas, seguras, resilientes y sostenibles. Ahora bien, este proyecto busca contribuir a profundizar la seguridad de las zonas agrícolas mediante el uso de tecnología IoT y el análisis de datos de video que permite tener información en tiempo real del ingreso de vehículos no registrados, generando alertas preventivas a los moradores en base a un sistema eficiente de datos de los moradores.

Cuando se instale este sistema en la vía de acceso que articula muchos terrenos agrícolas, se pretende que emerjan entornos rurales más protegidos, donde pequeños y grandes productores puedan adquirir mayor control de sus cultivos y reaccionar rápidamente ante acontecimientos sospechosos. De tal forma, se incrementa la permanencia de los agricultores, se reducen las pérdidas económicas y se beneficia la calidad de vida en su medio.

El Plan de Creación de Oportunidades 2021–2025 impulsado por la Secretaría Nacional de Planificación relaciona el presente proyecto con el Eje 2, Social, el cual persigue la creación de bienestar y elementos que generen nuevas oportunidades para el campo, siendo este eje muy significativo para pueblos y nacionalidades. Resumiendo, entre los elementos que promueven la inclusión y la contextualización al aplicar tecnologías emergentes en contextos agrícolas está la innovación, la cual permite generar nuevas oportunidades en las condiciones específicas del sector a través de la tecnología emergente (Secretaría Nacional de Planificación, 2021).

De la misma forma, el sistema de seguridad basado en IoT y procesamiento de video también proporciona una solución tecnológica concreta para un problema real. A su vez, potenciar la cohesión comunitaria, mejorar la seguridad rural y contribuir al desarrollo sostenible de las zonas rurales agrícolas cumple con los objetivos definidos por el ODS 11.

2. CAPÍTULO II: FUNDAMENTOS TEORICOS

El presente capítulo pasará a centrarse en consultar la teoría existente correspondiente con la información recabada de los distintos bloques tecnológicos tratados durante este trabajo de titulación. Se exponen las bases que permitirán la comprensión del funcionamiento de los sistemas de videovigilancia, la detección y reconocimiento de vehículos mediante visión por computadora, y por último la arquitectura del Internet de las Cosas (IoT) puesta en práctica para el caso de los entornos rurales. En ese sentido, además se analizarán el comportamiento de las redes de comunicación, los principios básicos de procesamiento de imágenes, así como el uso de tecnologías preentrenadas para el reconocimiento de matrículas de vehículos. En suma, el estudio se centrará en caracterizar las principales peculiaridades del hardware indicado en el apartado anterior, así como de los sensores, microcontroladores y cámaras, teniendo en cuenta en este sentido aspectos tales como la conectividad, el consumo energético y la adaptación capaz del hardware hacia el entorno.

2.1 Arquitectura de sistemas IoT aplicados a zonas rurales y agrícolas

La introducción de sistemas IoT en zonas rurales y agrícolas tiene que contar con arquitecturas que sempiternamente se encuentren definidas por las limitantes de conectividad, los recursos que resultan escasos y las exigencias que corresponden al ámbito agropecuario, habitual en este sector. Las arquitecturas que se suelen proponer para su expresión suelen dividirse en capas, que oscilan desde la parte de la percepción (cada uno de los sensores y dispositivos) hasta la parte donde se produce la gestión de datos y procesos a través de plataformas en la nube. Se han impulsado diversos trabajos en el campo de la tecnología que demuestran que esta manera de organizar los recursos en capas permite una buena integración de las tecnologías para realizar las funciones de monitorear y controlar la agricultura. Como por ejemplo el sistema FoodCraft que elige una solución de bajo coste para las comunidades

indígenas que consiste en utilizar sensores medioambientales y en comunicación satelital para recoger datos en tiempo real de las variables del cultivo que son importantes para la toma de decisiones (González, 2023).

En la misma línea, Köksal, (2019) propone un modelo que se encuentra estructurado a partir de la relación entre cada una de las capas y un medio con unos requerimientos funcionales específicos, como, por ejemplo, automatizaciones de tareas que deben ser incluidas en la de un riego o en la de control ambiental. Asimismo, se han podido concretar experiencias prácticas, como se expone en el trabajo de Doe, (2017), que reconducen a la utilización de redes de bajo consumo energético y plataformas en la nube como vías para habilitar funciones como los de alertas o el de control remoto de los recursos, que son necesarios para la agricultura de precisión, etc.

2.1.1 Conceptos básicos de IoT y su aplicación.

El Internet de las Cosas (IoT, por sus siglas en inglés) es una tecnología nueva y emergente que ha cambiado drásticamente la forma en que se encuentra, transmite y reutiliza la información actual. En el entorno rural y agrícola, el IoT permite nuevas posibilidades para incrementar la productividad, sostenibilidad y eficiencia de los procesos con soluciones automatizadas y fundamentadas en datos. En este sentido, la técnica se ha utilizado para el monitoreo de las condiciones ambientales, la automatización del riego, el uso eficiente de insumos, así como la toma de decisiones, lo que pone de manifiesto cómo puede lidiarse con los problemas de zonas con poca infraestructura o geografía difusa.

Uno de los elementos más destacados y a la vez importantes de los sistemas IoT en cultivos agrícolas es el uso de los sensores ambientales, que pueden ser usados para medir variables como la temperatura, humedad del terreno, intensidad luminosa, presión atmosférica o movimiento. Estos sensores han sido ideados para tener bajo consumo y gran exactitud, adaptándose bien a entornos rurales con poco acceso a fuentes de energía o a la conectividad

(Sharma, 2019). Además, se han desarrollado sensores multiparamétricos capaces de llevar a cabo la medición de varias variables de forma simultánea, llevando así a un mejor del resultado y una medición más detallada, justo para la agricultura de precisión (Li, 2015). Consecuentemente uno de los principales retos técnicos es la interoperabilidad entre sensores de diferentes fabricantes, así como su sensibilidad a las condiciones cambiantes; resolver estos problemas podría ser un factor relevante para asegurar que la recolección de datos sea confiable en las zonas rurales.

Para que dichas tecnologías funcionen con coordinación es necesario contar con tecnologías que permitan la comunicación dentro de las condiciones de la ruralidad. La conectividad en esta área es una de las dificultades a superar, por lo tanto el tipo de tecnología escoger debe considerar factores como el alcance, el consumo energético, el ancho de banda y los costes. Raza (2017), hace un comparativo detallado de opciones como Wi-Fi, ZigBee, LoRa, Bluetooth y redes celulares, concluyendo que las tecnologías LPWAN como LoRa y Sigfox son especialmente adecuadas para entornos rurales por su amplio alcance y bajo consumo. Gubbi (2013), destaca la necesidad de infraestructuras escalables ante la creciente capacidad de dispositivos conectados. Por otro lado (Sahoo, 2018) señala que estos sistemas son importantes para asegurar la entrega confiable de datos entre sensores y plataformas remotas en fincas extensivas.

Cuando ya se encuentren disponibles, los datos deben pasar a una fase de elaboración y análisis de los mismos para convertirlos en información útil. Las plataformas de continuación han sido la parte fundamental del IoT moderno, pues cuentan con almacenamiento de grandes volúmenes de datos, procesamiento en tiempo real y herramientas de visualización de datos. Las plataformas no solamente permiten la monitorización de las variables ambientales en ubicaciones remotas sino que además permiten la automatización y el control, de los dispositivos conectados. Botta (2016) indica que estas soluciones en la nube permiten

escalabilidad y reducción de costes operativos, mientras que Chen M. M. (2014) pone de relieve su contribución hacia la integración de sistemas heterogéneos. En ambientes rurales, donde son escasos los recursos humanos y técnicos, las plataformas de continuación son una herramienta esencial del IoT que contribuye a mejorar la eficiencia de la producción. Sin embargo, es necesario tener en cuenta temas de seguridad y privacidad, especialmente si se gestionan datos sensibles referentes a la producción o a la geografía de los cultivos. Khan R. M. (2015) advierte que la seguridad de la información y la disponibilidad climática del sistema son temas críticos para consolidar la confianza y la aceptación sobre el IoT en el contexto rural.

Finalmente, la interacción con el usuario se produce a través de distintos tipos de interfaces como son las aplicaciones móviles, dashboards o plataformas web, que permiten visualizar información, configurar el sistema o activar la respuesta manualmente. Las aplicaciones son necesarias para que los agricultores puedan acceder a la información de manera oportuna, lo que les impide no sólo manejar su cultivo sino hacerlo incluso desde lugares lejanos. Una interfaz altamente intuitiva, que sea adaptable a distintos niveles de alfabetización tecnológica, se hace clave para facilitar la adopción del IoT en poblaciones rurales.

2.1.2 Arquitectura de un sistema de seguridad para entornos rurales basada en las capas fundamentales del modelo IoT.

Con el objetivo de representar de forma estructurada un sistema determinado del tipo de interés In-House, más exactamente el desarrollo de un Sistema de monitoreo bajo un Modelo arquitectónico del tipo IoT para la seguridad en plantaciones de aguacate en el sector San José, Mira, el modelo se cobra especial sentido en virtud de las características que presentan las zonas rurales: poca densidad de población, infraestructuras pobres, alto riesgo de no detección del intruso, necesidad de un cierto trabajo de campo.

El documento de Al-Fuqaha (2015), presenta una disposición de IoT estructurada en cuatro capas principales: percepción, red, procesamiento y aplicación. Esta disposición permite llevar a cabo de una manera escalonada la recolección, transmisión, análisis y presentación de datos relevantes en condiciones de trabajo escasas. En la capa de percepción, el uso de tipos de sensores como los sensores de movimiento, las cámaras de vigilancia, o los actuadores como por ejemplo alarmas o luces de seguridad, permite detectar presencias no autorizadas o comportamientos extraños en tiempo real lo que es fundamental en zonas agrícolas extensas como San José en donde la presencia humana no siempre es constante.

Según comentan Sicari (2015) y Khan (2014), la capa de red se enfrenta a graves inconvenientes cuando la utilizamos en zonas rurales. La elección de los protocolos de comunicación deberá estar basada en tecnologías de larga distancia y bajo consumo, tales como LoRa o las redes móviles de bajo ancho de banda. Las mencionadas tecnologías garantizan la transmisión de información, tales como datos, incluso desde lugares alejados dentro de una plantación. Para asegurar la integridad de los datos, serán necesarios mecanismos de cifrado y de autenticación muy robustos con el objetivo de prevenir las intrusiones o modificaciones malintencionadas sobre los datos de seguridad.

En lo que respecta a la capa del proceso, esta se convierte en un factor determinante en el marco del sistema que se presentará, ya que en el mismo se deben incorporar tanto el almacenamiento y analítica de datos, como el procesamiento de video. Por este motivo, las plataformas de procesamiento locales o en la nube han de poder procesar las imágenes y los videos generados por las cámaras, mediante algoritmos de visión artificial, con el fin de detectar patrones sospechosos y generar alertas automatizadas. Borgia (2013) menciona que, en el contexto rural, este procesamiento debe ser eficiente, tolerante a los fallos y capaz de llevarse a cabo con conectividad intermitente.

La capa de aplicación expone las interfaces a partir de las cuales los agricultores o responsables de la seguridad pueden comunicarse con el sistema, como paneles de control accesibles desde dispositivos móviles, navegadores web, que permiten visualizar dispositivos, recibir alertas y tomar decisiones en tiempo real. El alcance local son elementos importantes para garantizar la adopción del sistema por parte de las comunidades rurales, que muchas veces son débiles en el acceso y el dominio de las herramientas digitales.

La arquitectura que aparece expuesta en este artículo deberá contemplar ciertas características clave, esto es, la escalabilidad y la resiliencia, tal como se aprecia en la Figura 3. La escalabilidad permitirá que el sistema pueda realizar un enganche de sensores, de cámaras o de sectores a vigilar si así lo hubiesen requerido, a medida que el ingreso de usuarios o el grado de riesgos fuesen a mayor. Este aspecto es clave en zonas rurales del tipo del sector San José, dado que las condiciones de operación pueden ser modificadas sin aviso; mientras que la resiliencia permite que el sistema pueda seguir funcionando en situaciones del tipo de un corte de energía o conectividad interrumpida. El uso del almacenamiento local temporal garantiza que los datos críticos no se vean comprometidos lo que permitirá que el nivel de respuesta se mantenga aún en situaciones de tipo remoto.

Figura 3. Fases de la Arquitectura

MODELO DE ARQUITECTURA DE SISTEMA DE SEGURIDAD



Nota: Fases de cómo se realizará el proyecto

El modelo de IoT por capas presentado en la Figura 3 permite implementar un sistema de monitoreo de seguridad cómodo y eficaz para las explotaciones agrícolas, siendo la combinación de tecnologías para la captura de eventos, la transmisión eficiente, el procesamiento inteligente (con video) y la presentación de información útil para los usuarios finales, que mejoran la seguridad de los cultivos de aguacate y a la vez, propicia el desarrollo tecnológico sostenible de las comunidades rurales para una agricultura más segura, automatizada y basada o dependiente de los datos.

2.2 Capa de Percepción: Captura de datos en el entorno agrícola

La capa de percepción, que corresponde, por tanto, a la primera etapa dentro de la arquitectura IoT, permite la captura directa de los datos físicos del entorno a través de los sensores y de los dispositivos inteligentes. La capa de percepción hace posible medir la variabilidad del entorno en la agricultura, permitiendo, por ejemplo, supervisar valores de humedad del suelo, y temperatura, presencia de movimiento o iluminación. Así, es la que

permite que, mediante la recopilación de información, se pueda llevar a cabo la toma de decisiones automatizada e inteligente, teniendo como base la información que proporciona.

Además, el correcto funcionamiento de la capa de percepción juega un papel fundamental para permitir la correcta ambientación de la información.

“La capa de percepción contiene todos los dispositivos y todos los sensores que permiten la captura de los datos del entorno físico y da entrada a la información para el sistema IoT” (Al-Fuqaha, 2015).

2.2.1 Sensores de movimiento PIR, infrarrojos y magnéticos para el monitoreo perimetral en entornos agrícolas

En el contexto de la seguridad rural, especialmente en plantaciones agrícolas con extensiones amplias y acceso limitado a vigilancia continua, el uso de sensores de movimiento en la capa de percepción del modelo IoT resulta crucial para la detección temprana de intrusiones. Los sensores PIR (Passive Infrared) se destacan por su capacidad para detectar cambios en la radiación infrarroja emitida por cuerpos en movimiento, como personas o animales. Este tipo de sensor es ampliamente utilizado en zonas agrícolas debido a su bajo consumo energético, facilidad de instalación y fiabilidad en exteriores, lo que permite su uso continuo incluso en condiciones ambientales variables (Chen, 2019)

Los sensores infrarrojos activos operan con un sistema emisor-receptor que cuenta con una barrera de visibilidad, sólo se desconecta cuando se emite una señal. Resulta una gran tecnología evitando la patrulla visual de entradas como caminos de servicio, entradas de bodegas, plantaciones, ya que responde instantáneamente a la intrusión (Khan R. K., 2014) Por su parte, los sensores magnéticos ubicados en portones y puertas o muros de cercas, miden modificaciones en el campo magnético (de la distancia o por la separación de dos partes correspondientes del sensor). Resultan útiles para controlar físicamente lugares estratégicos a las puertas de fincas.

La incorporación de estos sensores a un sistema de monitoreo IoT elimina la necesidad de constante vigilancia humana y permite la generación de alarmas automáticas y la comunicación en tiempo real a sistemas de control remoto. En el caso de las plantaciones de aguacate como las del sector San José, Mira, este tipo de dispositivo es una solución tangible de bajo coste para prevenir hurtos o accesos no autorizados, aumentando la seguridad en combinación con sistemas que pueden escalarse o complementarse con videovigilancia.

2.2.2 Cámaras IP para vigilancia en caminos, accesos y cultivos

En la capa de percepción de un sistema IoT, los dispositivos que tienen la función de obtener información del contexto son parte crucial para garantizar el correcto monitoreo. Es en este sentido que las cámaras IP, por hacer referencia a una de las tecnologías más utilizadas para la supervisión visual de entornos agrícolas, han adquirido una importancia que ha trascendido hasta el punto de ser utilizadas especialmente en explotaciones agrícolas para realizar la supervisión de accesos, caminos y perimetraciones que son vulnerables. Las cámaras IP cuentan con la función de enviar video en tiempo real a través de redes IP, lo que les permite tanto grabar imágenes como analizar imágenes de forma remota y automática.

En espacios rurales como el sector San José, Mira, donde las plantaciones de aguacate se distribuyen por áreas extensas y fuera de forma compacta, el uso de cámaras IP brinda ventajas considerables y su implementación permite cubrir diferentes puntos estratégicos sin la necesidad de que exista el sistema de vigilancia humano, lo cual es especialmente útil ante la carencia de personal o ante la imposibilidad de acceder a determinados puntos en todo momento. Además, dadas sus condiciones para poder funcionar en redes como LoRa, en redes wifi o incluso mediante enlaces satelitales y mediante paneles solares, se estima que deben de tener una viabilidad en utilizarse en zonas sin cobertura tradicional y sin disponibilidad de red eléctrica, es decir, en zonas remotas o de acceso dificultoso (Mehta, 2021).

Otra ventaja importante es que se integran con tecnologías de visión artificial. Las cámaras ip modernas pueden utilizar algoritmos de inteligencia artificial que tengan la capacidad de detectar movimientos sospechosos, distinguir entre personas, animales y vehículos e incluso generar avisos automáticos en tiempo real. Esta característica resulta altamente interesante en sistemas de seguridad que pretenden una detección de robos o intrusiones en cultivos, pues toda respuesta temprana puede prevenir pérdidas económicas importantes. Las cámaras pueden integrarse con sensores de movimiento PIR o magnéticos, haciendo que grabe y/o avise solo cuando hay un evento relevante evitando un uso innecesario del ancho de banda y de la energía (Baccarini, 2023).

De la misma forma, la visión nocturna y la alta resolución son características técnicas muy importantes en el ámbito agrario. Las IP que incluyen infrarrojos o con tecnología térmica permiten realizar una supervisión del medio durante la noche y durante el mal tiempo, lo que permite la supervisión del medio noche y durante mal tiempo, lo que permite garantizar la supervisión del medio a todas horas del día, lo que es muy importante en cualquier tipo de situación, donde los accesos muy propensos a ser considerados menos seguros y con horario nocturno. Por otra parte, dejar almacenar las grabaciones de filmaciones en la nube para poder visualizar desde otro lugar, también contribuye a la garantía del control de la información generada (Botta, 2016).

Las cámaras IP, al integrarse dentro de la arquitectura IoT en la capa de percepción, permiten construir soluciones de monitoreo perimetral altamente eficaces, autónomas y desarrolladas específicamente para abordar los retos de las zonas rurales. Su implementación a lo largo de caminos, accesos y áreas de cultivo representa una de sus principales características al permitir reforzar la seguridad de las plantaciones de alto valor como el aguacate y aprovechar al máximo los recursos tecnológicos disponibles en entornos con una infraestructura limitada.

2.2.3 Comparación con métodos tradicionales de vigilancia en el medio rural

De acuerdo con la forma única que se ha creado la seguridad en las zonas rurales y agrícolas que ha consistido sólo en recorrer los predios a vigilar por los porteros, la perforación de férrea, o la utilización de alarmas manuales, también hay que mencionar que estas prácticas o técnicas tan sólo tienen muchas limitaciones, especialmente en los lugares con escasa densidad de población o en zonas de intransitabilidad por los diferentes sistemas de seguridad de la anterior forma como el sector San José, Mira. La falta de un sistema de vigilancia permanente y autónoma; los precios del coste del trabajo; la probabilidad (los recursos humanos son equivocados, siempre erran); o el limitado tiempo de respuesta y la escasa posibilidad ante sucesos inesperados son algunas limitaciones con las que tienen que luchar los sistemas que permiten obtener la seguridad de forma convencional.

Ante esta problemática, los sistemas de vigilancia tradicionales han encontrado en las tecnologías IoT una alternativa a la incapacidad de algunas del sector para dar soluciones eficientes y sostenibles. Para ello, los sistemas propuestos utilizan dispositivos tales como: sensores de movimiento, cámaras IP, módulos de comunicación inalámbricos y plataformas de procesamiento para llevar a cabo la vigilancia continua, detectar eventos automáticamente y reaccionar de forma instantánea, sin la intervención constante de un ser humano. En esta línea, Jawad (2017), destaca que la capacidad de los sistemas IoT en zonas remotas sin supervisión directa es una ventaja importante, especialmente en áreas agrícolas escasamente pobladas donde la entrada continua de los humanos es difícil.

La comparación de los dos enfoques pone de manifiesto que las soluciones de IoT no solo posibilitan la vigilancia 24/7 con una menor dependencia del personal, sino que además registran y almacenan pruebas gráficas junto con datos históricos. Lo que aporta una base importante para la toma de decisiones, la investigación de incidentes y la mejora continua de la solución de seguridad. (Lan, 2020) afirma que los sensores conectados a través de

tecnologías de comunicación inalámbrica como LoRa o Wi-Fi brindan la posibilidad de reaccionar a situaciones de sospecha en tiempo real, lo que incrementa el nivel de protección ante robos o intrusiones.

En segundo lugar, se observan métodos tradicionales, como los vigilantes de comunidad o las llamadas de emergencia, que carecen de trazabilidad, precisión y efectividad pues dependen en mayor medida de factores humanos como la atención, la disponibilidad o el tiempo de respuesta. Bouabdellah (2019), indica que los sistemas IoT pueden, gracias a la introducción de fuentes de energía alternativas, tales como paneles solares y baterías de respaldo, continuar su funcionamiento aun en condiciones adversas de infraestructura eléctrica, lo que se convierte en una ventaja de gran magnitud para garantizar la continuidad en la vigilancia de las plantaciones rurales.

Con referencia a la situación concreta de la seguridad de las plantaciones de aguacate, en la que la protección perimetral y la de caminos de acceso es fundamental, la seguridad, los sistemas IoT se presentan como una solución actual, eficaz y flexible. Estas tecnologías no solo abordan las limitaciones de las técnicas tradicionales, sino que también incrementan el perímetro de la vigilancia, refuerzan la seguridad proactiva y mejoran la gestión de los recursos a través de un uso inteligente de la tecnología.

2.2.4 Condiciones técnicas en zonas agrícolas: iluminación, clima, polvo y obstáculos físicos.

La puesta en marcha de aplicaciones de IoT en el campo está influida por problemas técnicos que afectan a la calidad de los dispositivos que forman la capa de percepción. La capa de percepción se encarga de la detección y la captura de datos del entorno físico, pero depende en gran medida de la calidad en sensores y actuadores, así como de las condiciones del entorno en el que podrían circular las soluciones de IoT. En el caso de plantaciones agrícolas, como las del sector de aguacates en San José, Mira, el polvo, la humedad, la vegetación densa, el sol

intenso y la presencia, más o menos marcada, de obstáculos físicos naturales o artificiales son los aspectos que más afectan al diseño de la captura de datos en la plantación.

El polvo y la suciedad en suspensión pueden obstruir sensores ópticos y afectar la precisión de cámaras IP, reduciendo su eficacia en la detección perimetral. Asimismo, la exposición prolongada a la radiación solar y a temperaturas elevadas puede sobrecalentar los equipos electrónicos, mientras que lluvias intensas o niebla pueden distorsionar o interrumpir las señales de sensores infrarrojos o cámaras convencionales. En este sentido, se recomienda el uso de carcasas con clasificación IP65 o superior, que garanticen protección contra polvo y humedad, así como materiales anticorrosivos y recubrimientos térmicos para preservar la integridad de los componentes electrónicos (Ayaz, 2018).

La iluminación es un tema a tener en cuenta. Al llevar a cabo videovigilancia en la actividad agrícola, o bien, en situaciones donde la actividad tiene lugar normalmente por la noche (o bien en condiciones de visibilidad muy reducida), los sistemas de videovigilancia tradicionales dejan de cumplir su función y, por tanto, es necesario y conveniente complementarlos con sensores infrarrojos o cámaras térmicas que mantienen la videovigilancia funcionando, aunque la actividad se realice durante la noche o haya niebla espesa (Ray, 2018). Por otra parte, el diseño tiene que tener en cuenta el establecimiento de ubicaciones específicas donde colocar los dispositivos para evitar interferencias provocadas por taludes, máquinas agrícolas, zonas aledañas con árboles altos o bien, cercas, así como posibles interferencias entre la línea de visión y la transmisión inalámbrica de datos.

En relación con la energía, muchas zonas rurales carecen de acceso confiable a la red eléctrica. Por tanto, resulta crucial emplear fuentes autónomas como paneles solares con baterías integradas, acompañadas de reguladores térmicos y sistemas de ventilación pasiva, que permitan a los sensores funcionar de forma continua sin sobrecalentarse (Jawad H. M., 2017). Igualmente, la presencia de insectos, barro o residuos agrícolas puede dañar conectores y

puertos expuestos, por lo que se recomienda encapsular estos elementos y aplicar sistemas de autolimpieza o calibración automática.

La elaboración y la creación de la capa de percepción para un sistema de supervisión IoT orientado al seguimiento de la seguridad de los campos de producción debe adaptarse a la complejidad de la actividad en el mismo campo. Todo lo cual significa, no solo escoger qué sensores y cámaras son los adecuados, sino que también se deben proteger contra los factores externos que degraden el funcionamiento esperado, de ahí que el planteamiento de estos aspectos técnicos desde el inicio del diseño, hará que la robustez, eficacia y sostenibilidad del sistema sean considerablemente mayores en situaciones como las del campo del San José, en Mira.

2.3 Capa de Red: Conectividad en áreas rurales

La capa de red es crucial para el establecimiento de la conectividad entre dispositivos en general, pero en entornos rurales cobra especial auge dada la situación geográfica y la escasa infraestructura que imposibilita el acceso a Internet. Éstos hacen uso de tecnologías tales como Wi-Fi de largo alcance, redes celulares 3G, 4G, LoRaWAN o enlaces satelitales para poder establecer rutas de comunicación estables y que sean eficientes. La solución propuesta hace posible el envío de datos entre los sensores, nodos e incluso servidores asegurando la operatividad de sistemas de monitorización, automatización o servicios de conectividad comunitaria, a pesar de las limitaciones que presenta el entorno.

2.3.1 Tecnologías de comunicación adaptadas a zonas rurales

Las zonas rurales presentan desafíos únicos en cuanto a conectividad debido a factores como la baja densidad poblacional, la dispersión geográfica, la limitada infraestructura eléctrica y las dificultades orográficas. Ante estas condiciones, han emergido diversas tecnologías de comunicación que permiten establecer redes de datos eficientes, sostenibles y

de bajo costo, adaptadas a estos contextos. A continuación, se describen algunas de las más relevantes:

Wi-Fi de largo alcance (Wi-Fi Rural)

El Wi-Fi convencional se puede ajustar al entorno de áreas rurales para mejorar la cobertura de forma práctica y económica, aumentando la potencia de unos equipos en lugar de otros, utilizando antenas direccionales, así como utilizando bandas libres como la de 2,4 GHz o la de 5 GHz, lo que permite utilizar enlaces punto a punto o multipunto que pueden cubrir distancias de varios kilómetros. Este tipo de estándares de la tecnología Wi-Fi es una forma económica y fácil de aplicar el acceso a Internet en entornos de áreas rurales, aunque pueden verse perjudicados por obstáculos físicos y condiciones climatológicas (Zennaro et al. 2008).

LoRa y LoRaWAN

LoRa (Long Range) es una tecnología de modulación de espectro ensanchado diseñada para comunicaciones de largo alcance con bajo consumo energético. LoRaWAN (LoRa Wide Area Network) es la arquitectura de red que permite conectar múltiples dispositivos IoT a través de gateways. Estas tecnologías son ideales para entornos rurales donde se requieren sensores para monitoreo ambiental, agrícola o de infraestructura, ya que pueden cubrir varios kilómetros con baterías que duran años (Augustin et al., 2016).

Redes celulares (3G/4G/LTE)

Las redes móviles siguen siendo una de las opciones más utilizadas en áreas rurales, siempre que exista cobertura. El despliegue de estaciones base 4G permite acceso a internet de banda ancha y comunicaciones de voz confiables. En algunos casos, se recurre a antenas

repetidoras o femtoceldas para mejorar la señal en lugares remotos. El principal desafío es el costo de operación y la disponibilidad de energía eléctrica (ITU, 2021)

Tecnologías satelitales

En zonas extremadamente aisladas, donde no es viable el despliegue de infraestructuras terrestres, las conexiones satelitales son una solución efectiva. Los servicios de banda ancha satelital, como los ofrecidos por Starlink, permiten velocidades comparables a las redes urbanas, aunque a un costo elevado. Son útiles para aplicaciones críticas o educativas donde no hay otra alternativa (Borgia, 2013)

Redes de malla (Mesh Networks)

Las redes de malla inalámbricas permiten que varios nodos (routers o dispositivos) se conecten entre sí para ampliar la cobertura sin necesidad de una infraestructura centralizada. Estas redes se autoconfiguran y son tolerantes a fallos, lo que las convierte en una opción atractiva para comunidades rurales que desean compartir una única conexión a internet entre varios hogares o instalaciones (Akyildiz, 2005)

Tabla 1.

Tecnologías Inalámbricas

Tecnología	Frecuencia	Tasa de Datos	Alcance	Consumo de Energía	Costo
2G / 3G	Bandas Celulares	10 Mb/s	Varios km	Alta	Alto
802.15.4	2.4 GHz	250 kb/s	100 m	Baja	Bajo
Bluetooth	2.4 GHz	1, 2.1, 3 Mb/s	100 m	Baja	Bajo
LoRa	< 1 GHz	< 50 kb/s	2 – 5 km	Baja	Medio

LTE Cat 0/1	Bandas	1 – 10 Mb/s	Varios	Media	Alto
	Celulares		km		
NB-IoT	Bandas	0.1 – 1 Mb/s	Varios	Media	Medio
	Celulares		km		
SIGFOX	< 1 GHz	Muy baja	Varios	Baja	Medio
			km		
Weightless	< 1 GHz	0.1 – 24 Mb/s	Varios	Baja	Bajo
			km		
Wi-Fi (11n)	2.4, 5, <1 GHz	0.1 – 1 Mb/s	Varios	Media	Bajo
			km		
WirelessHART	2.4 GHz	250 kb/s	100 m	Media	Medio
ZigBee	2.4 GHz	250 kb/s	100 m	Baja	Bajo
Z-Wave	908.42 MHz	40 kb/s	30 m	Baja	Medio

Nota: Está tabla muestra algunas de las tecnologías inalámbricas que se podrían usar en el proyecto

2.3.2 Protocolos de red aplicados a sistemas de seguridad agrícola

El avance de la tecnología en el ámbito agropecuario ha dado lugar al desarrollo de sistemas de seguridad inteligentes que ofrecen protección para las áreas rurales haciendo uso de sensores, cámaras, drones y plataformas de monitorización a distancia. Dicha tecnología, asociada a la infraestructura de red adecuada, pone en juego protocolos que permitan la transmisión de datos por múltiples nodos, garantizando de esta manera la interoperabilidad, bajísima latencia y fiabilidad aún en entornos de conectividad mínima.

Uno de los fundamentos básicos es el modelo TCP/IP empleado en redes actuales y modernas. El protocolo IP (Internet Protocol) se encarga de situar y dirigir los datos, dándose la circunstancia de que dispositivos que se encuentran alejados sean capaces de comunicarse en una red común; TCP (Transmission Control Protocol) garantiza la llegada de los paquetes

de forma ordenada y fiable, siendo una opción muy útil para el envío de registros de sensores o de imágenes de cámaras donde la integridad de los datos es particularmente importante (Forouzan, 2017).

En las aplicaciones donde la velocidad es fundamental, como la transmisión de vídeo en tiempo real desde cámaras de seguridad instaladas en el campo, se utilizará el User Datagram Protocolo (UDP), dado que no hace una verificación de la entrega de cada paquete. La una verificación de la entrega de paquetes al destinatario implica que hay que esperar más tiempo para saber si el paquete fue recibido o no y esto va en detrimento de la rapidez en la entrega. En la vigilancia en tiempo real, la rapidez es más importante que la confiabilidad, y es por eso que el protocolo UDP es el más adecuado para este tipo de aplicaciones en soportes de vigilancia en tiempo real, aun a costa de la fiabilidad de la entrega de datos (Kurose y Ross, 2021).

El protocolo Message Queuing Telemetry Transport (MQTT) se ha convertido en una solución efectiva para ambientes de baja potencia y ancho de banda como es el caso de las zonas rurales. Este protocolo utiliza un modelo de publicación/suscripción que funciona a través del Bus (broker), de tal forma que sensores de movimiento, humedad, cámaras PIR o dispositivos de apertura de puertas puedan informar de su estado a un servidor central lo más rápidamente posible y sin necesidad de grandes recursos (Banks y Gupta, 2014), lo que nos permite implementar sistemas que sean escalables y de vigilancia y monitorización que puedan alertar al usuario mediante notificaciones móviles o gráficos informativos en tiempo real.

Con todo, esto no es más que una extensión de la posibilidad de cada uno de los protocolos HTTP y HTTPS que implementa poder ver y manejar el sistema desde una o varias páginas web que se pueden visualizar desde cualquier navegador. El uso de éstas nos hace que se puedan dar de alta o baja usuarios, consultar los logs, controlar cámaras PTZ, leer eventos históricos, activar alguna alarma o mecanismo físico desde cualquier parte donde haya acceso a una conexión de internet, etc. Por otro parte, el uso de HTTPS es obligatorio si tenemos información sensible, ya que es la forma de encriptar la información para no exponerla a la interceptación o a la manipulación por algún “hacker” (Rescorla, 2000).

De la misma forma, existen protocolos como el denominado SNMP (Simple Network Management Protocol), el cual sirve para la supervisión del estado de los dispositivos de la red tales como gateways, repetidores Wi-Fi, cámaras IP o nodos del control, y que permite llevar a cabo la administración del sistema de forma remota, generar alertas ante fallos en el hardware o pérdidas de conexión, algo esencial en aquellos entornos donde no hay personal técnico disponible de forma permanente (Stallings, 2020).

En contextos más complejos y/o desarrollados, los sistemas de seguridad agrícola ya contarían con sistemas de seguridad agrícola que dispusiesen de protocolos como el CoAP o el RTSP. Mientras que el primero de este listado implementa el Protocolo de Aplicación Específica de bajo consumo y está diseñado para funcionar en dispositivos limitados, el segundo se implementa para establecer y gestionar las sesiones de streaming multimedia en tiempo real; de este modo, se podrá satisfacer la transmisión de video a partir de cámaras IP el cual se encuentra situado en lugares accesibles, por ejemplo.

La elección de estos protocolos debe hacerse dependiendo de las necesidades del sistema: confiabilidad, velocidad, consumo energético, tipo de terminal, infraestructura de red, etc. Una adecuada combinación de ellos puede permitir la implementación de soluciones de seguridad avanzada, sostenibles y perfectamente adaptadas al entorno agrícola rural.

2.3.3 Retos de cobertura y fiabilidad: caminos dispersos, distancias largas y señal limitada

La puesta en práctica de sistemas de comunicación en entornos rurales plantea varios desafíos técnicos, sobresaliendo entre ellos, la buena cobertura, la fiabilidad de las conexiones, y las malas condiciones geográficas. En un entorno agrícola donde los cultivos, fincas y sistemas de vigilancia se encuentran situadas en extensiones muy grandes de terreno (a menudo de acceso disperso), con largas distancias y escasa infraestructura, mantener una red buena y eficaz se convierte en un gran reto.

Un problema importante está asociado a la escasa cobertura de las redes celulares y Wi-Fi, que se explica, entre otros, por la insuficiente inversión en infraestructura de telecomunicaciones para muchas áreas rurales. Las ondas de radiofrecuencia también pueden ser perturbadas por la topografía del terreno, la espesura de la vegetación o por instalaciones agrícolas cuando las líneas de visión entre el transmisor y el receptor son restringidas (ITU, 2021). Este fenómeno influye en el desarrollo de la calidad de los servicios de monitoreo, vigilancia y recolección de datos en tiempo real, que son determinantes para la seguridad agrícola.

Una cuestión sin duda trascendental es la distancia existente entre dispositivos. En muchas aplicaciones relacionadas con la agricultura, los sensores, las cámaras o los nodos de comunicación pueden estar separados cientos de metros o incluso kilómetros del punto de recogida o del servidor. Para poder cubrir estas distancias, se precisan tipos de soluciones concretos, como pueden ser antenas de alta ganancia, enlaces punto a punto mediante Wi-Fi de largo alcance, o también tecnologías como LPWAN (Low Power Wide Area Network), por ejemplo, LoRa, que van permitiendo mantener la comunicación a pesar de la separación realizada, aunque con capacidades de ancho de banda limitadas (Augustin et al. 2016).

Además de lo anterior, existe el riesgo de interrupción de la señal además de la posibilidad de que las condiciones ambientales adversas (precipitación, viento, tormentas)

generen una interrupción o falla en la conectividad de los dispositivos, lo cual afecta la confiabilidad del sistema. En este contexto es necesario contar con mecanismos de redundancia, almacenamiento de datos local ante fallas y con protocolos de comunicación que sean capaces de soportar reconexión automática y tolerancia a fallas como son MQTT o CoAP (Banks y Gupta, 2014).

Además de lo anterior, existe el riesgo de interrupción de la señal además de la posibilidad de que las condiciones ambientales adversas (precipitación, viento, tormentas) generen una interrupción o falla en la conectividad de los dispositivos, lo cual afecta la confiabilidad del sistema. En este contexto es necesario contar con mecanismos de redundancia, almacenamiento de datos local ante fallas y con protocolos de comunicación que sean capaces de soportar reconexión automática y tolerancia a fallas como son MQTT o CoAP (Banks y Gupta, 2014).

De la misma forma, también se presentan complicaciones relacionadas con la disponibilidad de energía. Muchos sistemas funcionan en escenarios donde existe la posibilidad de que no se disponga de la red eléctrica adecuadamente, y en estos casos, deben depender de sistemas de baterías, paneles solares o generadores. Esto conlleva restricciones al tipo de tecnología que se puede poner en marcha, favoreciendo así dispositivos de bajo consumo y la optimización de las comunicaciones.

La confiabilidad de los enlaces de comunicaciones en estos contextos depende, no solamente de la tecnología, sino también de la planificación de la red; de los repetidores, buenas localizaciones de gateways dentro de la red y de protocolos que controlen la pérdida de los datos. La planificación de la red ha de preverse en los puntos críticos de cobertura; la ampliación futura y el mantenimiento, entre otras variables, ya que el soporte en zonas rurales es, por norma general, escaso.

2.4 Capa de Procesamiento: Análisis de información para seguridad en el campo

En un escenario de IoT para sistemas de seguridad en entornos agrícolas, la capa de procesamiento es un elemento clave, ya que transforma, analiza, interpreta y genera decisiones con respecto a los datos recolectados por los sensores o dispositivos de captura, como una cámara o módulos de detección de movimiento. A través de esta capa, se lleva a cabo el paso de transformación de los datos crudos a información útil, lo que permite, por tanto, poder automatizar las respuestas ante eventos sospechosos o situaciones de riesgo en el interior de una plantación agrícola.

El tratamiento de la información puede llevarse a cabo bien de un modo local (edge computing) o bien de forma centralizada (cloud computing). En el primer caso, se utilizan microcontroladores, microprocesadores o dispositivos de borde (placas de procesamiento) que permiten llevar a cabo tareas de análisis en el mismo lugar donde se capturan los datos. Este tipo de procesamiento es muy práctico en todo caso en zonas rurales como San José, Mira, dado que redunda también en una menor dependencia del uso constante de internet y la posibilidad de que se tomen decisiones casi o en tiempo real (Sahoo, 2018). En el segundo caso, el tratamiento de la información implica enviar la información a servidores en la nube (cloud) que permiten mayor capacidad de cómputo para realizar análisis complejos, aplicar algoritmos de visión por computador y/o algoritmos de machine learning.

Un aspecto importante de esta capa es el procesamiento de video que además de identificar situaciones de intrusión, movimiento no autorizado o eventos anómalos mediante cámaras de vigilancia emplea técnicas tales como la detección de movimiento basada en diferencias de imágenes o video, la detección de objetos mediante Redes Neuronales Convolucionales (CNN) o la clasificación de eventos, por mencionar algunas. Para lograr todos estos tipos de problemas del procesamiento de video se implementan algoritmos de detección como el YOLO (You Only Look Once), OpenCV o TensorFlow para llevar a cabo la detección de personas, vehículos o animales en el caso de cultivos en tiempo real (Rodríguez, 2018).

Por su parte, también se lleva a cabo una integración de los datos obtenidos a partir de la tecnología que recogen los distintos sensores IoT (por ejemplo, PIR, sensores de proximidad, sensores de apertura y/o vibración) para extraer finalmente un análisis cruzado y mejorar la certeza del sistema. Por ejemplo, en el caso de que el sensor PIR mande una señal confirmando que ha habido detección de un movimiento y, simultáneamente una cámara pueda hacer la captura de una imagen en la que aparezca alguna figura humana, se podría disparar al sistema una alerta automática del tipo "más seguro". Estos datos podrían guardarse temporalmente en una base de datos local o, bien, ser enviados al servidor para una posterior consulta histórica, además de permitir adaptar el aprendizaje para mejorar la estrategia de seguridad.

Además, el uso de técnicas de procesamiento distribuido, unido a protocolos ligeros como el de MQTT hace que la información circule entre los módulos del sistema sin generar cuellos de botella. También se pueden usar técnicas de compresión de video (como H.264 o H.265) para optimizar el ancho de banda, lo cual es indispensable en entornos con señal escasa.

Figura 4. Análisis del movimiento y detección

En el procesamiento de video para el análisis de tráfico se establece un proceso en forma de jerarquías que empieza con la adquisición de la secuencia de video que se encuentra representada por la fuente de información que se grava o capturaba en tiempo real desde las cámaras, en las vías, o en otras zonas de interés; a continuación se aplica a los cuadros la detección de los objetos (diferencia de trama, eliminación de fondo o flujo óptico) con la finalidad de identificar los elementos móviles (vehículos o peatones).

Una vez que se detectan, los objetos se clasifican en categorías (coches, motos, camiones, personas...) utilizando métodos basados en el movimiento, la textura, la forma o el color, lo que permite la discriminación de las diferentes clases de actores viales. Y después se realiza el seguimiento de los objetos a lo largo de los fotogramas utilizando los métodos basados en puntos clave, en núcleos o en siluetas, lo que permite analizar trayectorias, determinar velocidades, contabilizar vehículos y generar métricas espacio-temporales que son relevantes para la gestión del tráfico. Este enfoque global puede ser aplicado en sistemas de

transporte inteligente y videovigilancia urbana, y supone un conjunto de herramientas clave para la regulación de tráfico vehicular, la detección de eventos anómalos y la toma de decisiones basada en datos.

2.4.1 Visión por computadora aplicada a vigilancia agrícola

La visión artificial ha llegado a ser una de las herramientas más poderosas que hay en los sistemas de seguridad actuales, pues da la posibilidad a las máquinas de traducir y obtener información útil a partir de imágenes y vídeos. En un universo agrícola, y más concretamente en plantaciones de aguacate en el ámbito rural de San José, Mira, la visión artificial es fundamental para la vigilancia automatizada, la detección de intrusos y el reconocimiento de actividades anómalas que puedan significar lesiones para la producción o para la infraestructura de los cultivos.

Uno de los métodos más comunes es la detección mediante comparación de frames de video consecutivos, que permite detectar movimientos o cambios en la escena que podrían ser causados por la presencia de personas, animales o vehículos. Aunque se considera un método básico, su implementación es sencilla y se puede realizar en tiempo real, por medio de la librería OpenCV, por ejemplo, especialmente en dispositivos de borde (Borgia, 2013).

En aquellos momentos en que se necesita más precisión, recurrimos a algoritmos de detección y reconocimiento de objetos basados en el aprendizaje profundo. Modelos que permiten identificar varios objetos en una imagen a gran velocidad y precisión como YOLO (you only look once), SSD (single shot multibox detector) o Faster R-CNN. Estos modelos pueden ser entrenados para reconocer figuras humanas, vehículos no autorizados o herramientas agrícolas, generando alertas cuando se detecta la presencia de objetos no deseados en una zona prohibida (Ray, 2018)

Además, se puede llevar a cabo el seguimiento de objetos (object tracking) para mantener a un objetivo en el campo de visión de la cámara a lo largo del tiempo. Este tipo de

seguimiento es útil en situaciones en las que hay que calcular rutas de desplazamiento o generar registros sobre conductas o movimientos sospechosos. Combinado con análisis de comportamiento, permite clasificar tipos de eventos tales como entradas no autorizadas, recorridos fuera de horario o patrullajes incompletos.

Algunas de las técnicas del procesamiento de imágenes térmicas o infrarrojas también se pueden sumar a la visión por computador, especialmente en entornos oscuros. Proporciona la capacidad de trabajo a partir de las condiciones adversas típicas de la actualidad nocturna.

La aplicación de esta tecnología requiere considerar aspectos relacionados en el entrenamiento del modelo, el almacenamiento eficiente de datos, el procesamiento en tiempo real y la seguridad de la información transmitida. La implementación en zonas rurales requiere soluciones que optimicen el procesamiento local y el consumo eléctrico para asegurar el funcionamiento continuo y en autonomía del sistema.

En conjunto, los sistemas de visión por computadora aplicados a la vigilancia agrícola permiten diseñar sistemas inteligentes, automatizados y autónomos que aumentan la seguridad en los campos, evitan el robo, mejoran la gestión de los recursos humanos o tecnológicos, y para ello resulta especialmente útil en áreas de difícil acceso o con dificultades logísticas.

2.4.2 Procesamiento de imágenes y video capturados en ambientes rurales

El procesamiento de imágenes y video en ambientes rurales, como las plantaciones de aguacate en el sector San José, Mira, enfrenta retos particulares que deben considerarse para garantizar un análisis efectivo y confiable. Las condiciones ambientales, la variabilidad lumínica, la presencia de elementos naturales (como árboles, sombras y viento) y la limitada infraestructura de comunicación impactan directamente en la calidad de las imágenes y la capacidad de procesamiento.

Cabe mencionar que uno de los principales inconvenientes es la variación de iluminación, que puede abarcar desde una gran luminosidad al sol en el día, a una baja

iluminación en la noche, o a condiciones de mala visibilidad debido al mal tiempo. Este inconveniente se salda, entre otras técnicas, utilizando la ecualización de histograma, los filtros adaptativos, y / o métodos de mejora de contraste que mejoran la visibilidad de los objetos en las imágenes González (2023), justo antes de la etapa del procesamiento.

A medida que los autores gozan de una mayor experiencia en el uso de una serie de técnicas para reducir el ruido en las imágenes, el ruido que procede de las condiciones naturales, que corresponde al movimiento del follaje agitado por el viento (la presencia del polvo y de la humedad en el cristal de las cámaras), también puede causar dificultades en la detección precisa de los objetos de una escena determinada y, en consecuencia, en la toma de decisiones basadas en las imágenes captadas. No obstante, con la finalidad de limpiar las imágenes y eliminar el ruido sin llegar a perder o a destruir una cantidad excesiva de información, se implementan filtros de reducción de ruido, como el filtro gaussiano o el filtro mediano, entre otros (Szeliski, 2011).

El proceso de procesamiento de video de tiempo real necesita algoritmos eficientes que permitan identificar eventos relevantes sin que la limitación del hardware o la conectividad hagan que se creen retrasos importantes. En este sentido, existen técnicas de compresión para el video que son utilizadas extensamente como el H.264 y H.265 para reducir el tamaño de los archivos al ser transmitidos o almacenados, con el fin de optimizar el ancho de banda y el espacio en el disco (Sharma, 2019).

Igualmente, la ejecución del procesamiento puede abarcar el empleo de técnicas de segmentación, para poder segmentar los objetos en movimiento, así como también la idea de análisis temporal, para realizar la distinción de eventos relevantes de falsos positivos que sean provocados por acontecimientos naturales como la existencia de pequeños animales o

variaciones climatológicas. La utilización de aprendizaje automático y de redes neuronales va a permitir conseguir mayor precisión, en los entornos complejos, mediante la adaptación de los modelos a las propiedades específicas del entorno rural (González, 2023).

Por último, dada la escasa disponibilidad de conectividad en áreas rurales el procesamiento en el borde (edge computing) adquiere relevancia puesto que permite a los dispositivos locales realizar el preprocesado y análisis preliminar de imágenes y vídeos y remitir únicamente los datos de relevancia o las alertas al centro de control, de manera que se reduce este sentido de dependencia de la red y con ello la rápida identificación de una respuesta del sistema ante los eventos críticos.

2.4.3 Detección de movimiento y análisis en tiempo real para prevención de robos

La detección de movimiento resulta ser un elemento fundamental en los sistemas de seguridad de los cultivos, dado que permite detectar la presencia de intrusos o de actos sospechosos en el momento, permitiendo actuar a tiempo ante posibles robos o daños en la plantación. Esta función es especialmente crítica en los entornos rurales como San José, Mira, donde el control y la vigilancia física son limitados y las pérdidas por robos pueden llegar a ser muy altas.

Los algoritmos de detección de movimiento con mayor aplicación, están basados en una comparación de imágenes sucesivas (frames) para reconocer cambios en el entorno. Métodos como la sustracción de fondo (background subtraction) nos permiten identificar los objetos en movimiento del fondo estático, optimizando así el procesamiento y también disminuyendo las falsedades (falsos positivos) que pueden ser provocados por elementos que no nos interesan (Piccardi, 2004). Algunos de estos métodos se pueden combinar con otros como los filtros morfológicos, a fin de optimizar la precisión y robustez ante el ruido y el ambiente cambiante.

Para poder ejecutar la aplicación en tiempo real estos algoritmos deben ser eficientes y deben ser ejecutados en dispositivos de recursos limitados como pueden ser por ejemplo microprocesadores integrados o cámaras inteligentes. Utilizar bibliotecas optimizadas como OpenCV facilita el uso de estas técnicas para la detección de movimientos sospechosos sin necesidad de enviar vídeo continuamente al servidor maximizando así el ancho de banda (Bradski, 2000).

No solo se realiza la detección, sino que en el análisis en tiempo real se añade la clasificación de los eventos que se clasifica en el caso de que se detecte a poca distancia a un animal, un ser humano o un vehículo y que se pueda realizar mediante algoritmos de aprendizaje automático o redes neuronales convolucionales (CNN), de manera que se pueden filtrar alarmas falsas y priorizar respuestas ante amenazas declaradas (Redmon et al. 2016).

Adicionalmente, la integración con los sistemas IoT permite que, cuando se produzca un movimiento, se genere automáticamente una alerta a través de un mensaje SMS, un mensaje de correo electrónico, una notificación en un móvil o la activación de la grabación de un vídeo para evidencias. Esta activación puede programarse para ser ejecutada en horas determinadas, optimizando así el funcionamiento del sistema.

2.4.4 Reconocimiento de vehículos y matrículas en caminos agrícolas

El reconocimiento automático de vehículos y matrículas en caminos de cultivos agrícolas se convierte en una tecnología fundamental para ofrecer seguridad en las zonas de cultivos rurales, como los cultivos de aguacate en el sector San José, Mira: gracias a dicho dispositivo se pueden identificar los vehículos, controlar los que están autorizados, combatir el robo y también en la medida que se va realizando la entrada y salida de maquinaria y/o personas, gestionar la vigilancia y control de acceso a distancia.

La tecnología toma como base procedimientos de visión por computadora y de reconocimiento óptico de caracteres (OCR) para la recuperación y el tratamiento de las imágenes que capturan las cámaras instaladas en puntos estratégicos, como entradas, salidas de caminos o accesos. Primero se realiza la detección del vehículo por medio de los algoritmos de detección de objetos, como por ejemplo YOLO (You Only Look Once) o bien SSD (Single Shot MultiBox Detector), que son sugeridos para poder localizar los automóviles, camionetas o tractors dentro de la escena con alta precisión y rapidez (Raza, 2017).

A continuación, se busca en la imagen la detección de la matrícula. Para llegar a esta fase se trata de obtener, mediante técnicas de segmentación y localización, la región en la que se localiza la matrícula. Para ello se aplican procedimientos basados en características de las formas, del color o de las texturas, así como redes neuronales dedicadas caracterizando también la robustez ante cambios de luz o de ángulo (Al-Fuqaha, 2015).

Finalmente, bajo el marco de un análisis de patrones, se utilizan algoritmos de OCR para transformar imágenes de matrículas en texto digital, permitiendo su posterior almacenamiento en base de datos y su posterior comparación con bases de datos de vehículos autorizados (y la posibilidad de generación de alertas en caso de ocurrencias no permitidas). Esta información podría ser integrada dentro de la plataforma IoT como evento y propiciar un seguimiento remoto (Silveira et al. 2020).

Además, la parte rural también requiere adaptaciones profundas para operar con recursos escasos y condiciones cambiantes, que incluyen polvo, sombra o climas cambiante. Para ello, el uso de cámaras con filtros infrarrojos y algoritmos de preprocesamiento para mejorar la calidad de la imagen generada antes de su reconocimiento es algo bastante habitual (García y Sánchez, 2017).

La asociación de estos sistemas ya con la arquitectura IoT permite poder automatizar el control del acceso, permitir el escaneado y control de entradas y salidas en tiempo real y permitir, incluso, un aumento de la seguridad en general de la plantación, logrando disminuir el riesgo de robos y permitiendo un correcto manejo de la finca.

2.4.5 Herramientas y modelos preentrenados utilizados

Al desarrollar sistemas de seguridad fundamentados en visión por computadora aplicados a zonas rurales la aplicación de herramientas y modelos preentrenados permite implementar de manera rápida y con un buen rendimiento técnicas de detección y reconocimiento. Dichos modelos están preparados para funcionar en situaciones de recursos limitados y condiciones ambientales diversas, atributos típicos de las condiciones rurales como la de las plantaciones de aguacate del sector San José, Mira.

YOLO (You Only Look Once) es uno de los modelos más populares para detección de objetos en tiempo real debido a su alta velocidad y precisión. Su arquitectura permite detectar múltiples objetos en una imagen con un solo pase por la red neuronal, lo que reduce significativamente el tiempo de procesamiento (Redmon et al., 2016). YOLO puede ser entrenado o adaptado para identificar personas, vehículos, animales y otros objetos relevantes para la seguridad agrícola.

Otra de las herramientas que hay que destacar es OpenALPR (Automatic License Plate Recognition), una solución específica para el reconocimiento automático de matrículas vehiculares. OpenALPR depende de la visión por computador y el aprendizaje automático para detectar y extraer información de las matrículas en imágenes o vídeos, y para el control de los accesos en caminos agrícolas, o en los perímetros (OpenALPR, 2023), justo para esos proyectos que requieren sistemas IoT donde se permitan realizar análisis rápido y fiable, sin tener que desarrollar modelos de cero.

Existen modelos preentrenados en Tensorflow Hub o PyTorch Hub que permiten utilizar redes para clasificación de imágenes, detección de objetos y segmentación semántica, las cuales se pueden adaptar a diferentes aplicaciones rurales. Al utilizar estos modelos se disminuye el tiempo de desarrollo y se pueden implementar sistemas robustos sin tener que contar con grandes bases de datos propias para el entrenamiento.

Con objeto de optimizar el rendimiento en áreas deficitarias en el uso de la energía y la conectividad, las herramientas de visualización suelen utilizarse junto a métodos de edge computing, que trabajan procesando localmente en dispositivos con capacidades reducidas, eliminando prácticamente la dependencia de la nube y mejorando la latencia en cuanto a la detección y respuesta (Sharma, 2019).

Gracias a estas tecnologías se pueden implementar los sistemas de monitoreo inteligentes, que aumentan la seguridad y la eficiencia en la administración de las plantaciones agrícolas, adaptando la administración a las características particulares de los contextos agrícolas.

2.4.6 Desafíos técnicos: resolución, ángulos de captura, conectividad y entorno natural

La implementación de sistemas de vigilancia y seguridad mediante procesamiento de video en zonas rurales, como las plantaciones de aguacate en San José, Mira, presenta una serie de retos técnicos que afecta la calidad y eficacia de la vigilancia. Uno de los principales retos es la calidad de la resolución de las cámaras, los ángulos de captura, las limitaciones en conectividad y las condiciones del entorno natural.

La resolución de las cámaras se considera fundamental para adquirir imágenes con un detalle suficiente que permiten el reconocimiento fiable de objetos, personas o matrículas. Sin embargo, cámaras de alta resolución producen un volumen de datos excesivo que podría colapsar las redes rurales de ancho de banda limitado, y también incrementan los requerimientos de almacenamiento y de procesamiento (Patel et al. 2019). Por ende, requiere

un equilibrio entre calidad de la imagen y recursos que se tienen disponibles, y además el uso de técnicas de compresión eficientes como H.265 para optimizar la utilización de la red.

Los diferentes ángulos de la captura de la imagen tienen una influencia directa en la capacidad que posee el sistema del robot agrícola para detectar y clasificar objetos. En los caminos de los cultivos agrícolas o bien en los perímetros de formas irregulares de la imagen, las cámaras se deben situar en ocasiones estratégicamente para cubrir el máximo espacio posible y evitar puntos ciegos. Una mala colocación de las cámaras se puede traducir en imágenes transformadas, que pueden llevar a oclusiones parciales y/o en una mala localización de los objetos, las cuales pueden afectar al sistema de detección y clasificación de los objetos que tiene el sistema (Liu, 2021).

La limitada conectividad en áreas rurales representa un reto conveniente. Las redes inalámbricas suelen tener como consecuencia una señal escasa, interferencias o desconexiones frecuentes, lo que imposibilita la transmisión ininterrumpida de los vídeos en tiempo real, obligando a que una parte de su procesamiento se desarrolle in situ (edge computing) y que los datos relevantes o las alarmas se transmitan solamente, maximizando el uso de la infraestructura existente (Shi et al. 2016).

Por último, el ambiente de los sistemas introduce complejidades como los cambios climáticos (lluvia, niebla y polvo), las variaciones de luz (sol directo y sombras) y los elementos que se mueven y que no tienen que ver con posibles amenazas (animales, hojas en movimiento, etc.). Todos estos elementos pueden provocar ruidos en las imágenes, falsos positivos o fallos en la detección. Para minimizar estos efectos se aplican filtros de imagen, algoritmos robustos y técnicas de aprendizaje adaptativo para hacer que el sistema sea más robusto a condiciones hostiles (González, 2023).

2.5 Capa de Aplicación: Monitoreo y respuesta en zonas agrícolas

La capa de aplicación de sistemas IoT para seguridad agrícola se define como el nexo entre la persona usuaria y la infraestructura tecnológica del campo, ya que en ella se conectan las funcionalidades de monitoreo, de alerta, de análisis y de control para la protección de los cultivos y de los recursos. De este modo, la capa de aplicación, ya sea a través de plataformas web o de aplicaciones de telefonía móvil, permite la visualización de datos en tiempo real, la gestión de las cámaras y de los sensores, la producción de reportes de seguridad, entre otros, lo que mejora la protección del campo agrícola.

Además, la capa de aplicación permite emitir alertas de modo automático a través de SMS, de correo electrónico o de notificaciones push de eventos sospechosos e incluso controlar dispositivos como alarmas y luces para inmediatamente atender esas alertas. En general, dado que en zonas rurales la conectividad suele ser limitada, esto quiere decir que las aplicaciones suelen operar en modo híbrido, es decir que almacenan los datos localmente y una vez restablecida la conectividad los sincronizan. Las conexiones e integraciones de inteligencia artificial y de machine learning contribuyen a la detección de patrones anómalos y a optimizar la eficacia del sistema cada vez más, todo lo cual contribuye a una gestión más sostenible y segura de las plantaciones agrícolas (Szeliski, 2011)

2.5.1 Sistemas de videovigilancia integrados a IoT en fincas y zonas rurales

La incorporación de sistemas de videovigilancia aplicados a la arquitectura IoT ha supuesto un auténtico cambio radical en la manera de llevar la seguridad a fincas y zonas rurales. Gracias a la posibilidad de llevar a cabo una monitorización remota, continua y en tiempo real de grandes superficies que antes eran muy difíciles de vigilar por su dispersión geográfica o su localización en entornos muy dificultosos. En el caso de las plantaciones agrícolas como las de aguacate del San José Mira, dichos sistemas de videovigilancia configuraban una solución tecnológica avanzada para proteger el patrimonio agrícola habitual frente a robos, vandalismo o intrusiones de animales.

Gracias a la conexión de cámaras inteligentes con diferentes sensores IoT como los de movimiento, apertura o vibraciones soportan no solo la obtención de imágenes y vídeos, sino también otros tipos de parámetros ambientales susceptibles de capturar la existencia de las actividades anómalas superiores. Las cámaras de las que se habla suelen ser equipos con capacidades de procesamiento en el borde (edge computing), lo que quiere decir que permiten procesos iniciales de tratamiento de vídeos en la misma unidad, lo que minimiza el tratamiento de grandes volúmenes de los mismos y maximiza el uso del ancho de banda en el caso de zonas rurales en las que la conectividad pueda ser corta o limitada, algo que Shi et al. (2016) transforma en un problema grave.

También es habitual que estos sistemas se comuniquen a través de redes inalámbricas con bajo consumo, por ejemplo LoRa o redes celulares 4G/5G cuando están disponibles, la cual debería tener cobertura suficiente también en zonas remotas. La placa de IoT actúa como la plataforma receptora de la información recabada. Los administradores de este tipo de sistemas pueden acceder desde dispositivos móviles o computadoras personales a una interfaz gráfica, la cual muestra en tiempo real, el estado de las cámaras y sensores, y las cuales incluyen también reproducciones de vídeo y alertas generadas automáticamente ante detección de algunos eventos sospechosos (Muthuraj, 2016).

La capacidad de transmitir mensajes de modo instantáneo vía SMS, por correo electrónico o, incluso, por aplicaciones móviles asegura una rápida respuesta a la eventualidad de incidentes; con ello se logra recortar el tiempo de respuesta y potencialmente prevenir pérdidas económicas o daños en la plantación. También la posibilidad de guardar su vídeo y de almacenar datos históricos invita a revisar los eventos y a realizar análisis forenses, así como ajustar los parámetros del sistema y perfeccionar su precisión.

La escalabilidad y la flexibilidad de los sistemas también deben considerarse. Dicha arquitectura IoT permite unir o eliminar dispositivos con facilidad, logrando una flexibilidad

relativa a requisitos específicos para cada finca o zona agrícola. Esto es muy importante en el medio rural, donde normalmente existe poca disponibilidad de recursos y la infraestructura en cuestión debe ir cambiando lenta y sosteniblemente.

2.5.2 Almacenamiento de datos: soluciones locales y en la nube adaptadas al entorno rural

El almacenamiento seguro y eficiente de la información es un aspecto clave en el desarrollo de soluciones de monitoreo y seguridad basadas en dispositivos de IoT para las zonas rurales, como son las plantaciones de aguacate en San José, Mira. La combinación de almacenamiento local, en la nube e híbrido es determinada por varios factores, tales como la conectividad, la latencia en el acceso a los datos, la seguridad, el coste que supondrá, la escalabilidad en la solución, etcétera.

Las soluciones de almacenamiento a nivel local suponen apoyarse en dispositivos físicos de almacenamiento situados en la finca o el área agrícola en cuestión, bien un servidor de ficheros local, bien un disco duro externo o bien un sistema NAS (Network Attached Storage). Tal opción tiene sus ventajas en relación a la autonomía ya que para el almacenamiento de la información y acceder a la misma de acuerdo a los usos siempre se puede usar la propia red local y no la conexión a internet, la cual puede ser deficiente o escasa en muchos entornos rurales. Por el contrario, permite acceder rápidamente a la información y procesarla en tiempo real y reduce también los riesgos pugnados en el acceso a los datos sensibles por las propias redes públicas (Botta et al., 2016).

Por otro lado, al tratarse de emplear soluciones en la nube, aportan flexibilidad y escalabilidad, ya que permite almacenar grandes volúmenes de datos a la vez que no hay que invertir en infraestructura física local. Existen plataformas como Amazon Web Services (AWS), Microsoft Azure o Google Cloud que ofrecen servicios específicos para IoT que permiten la integración de dispositivos, el análisis avanzado de los datos y la gestión de manera

remota, entre otras. Sin embargo, en zonas rurales, es habitual encontrar limitaciones de ancho de banda y disponibilidad de red lo que puede condicionar la sincronización y el acceso en tiempo real, así como, en general, la toma de decisiones (Dinh et al. 2017).

El modelo híbrido, que asocia el almacenamiento local para el procesamiento de datos "in situ" y como almacenamiento temporal y de sincronización a la nube, puede ser considerado la alternativa más adecuada, siempre y cuando las condiciones de red así lo permitan. Al optar por este tipo de modelo, se significa la mejora de la resiliencia del sistema, la continuidad operativa de éste y el uso más eficiente de los recursos de red (Botta et al., 2016).

De igual modo, se han de considerar aspectos de seguridad y privacidad en el almacenamiento de datos, incorporando cifrado y mecanismos de acceso controlado que protejan la información sensible relacionada con la operación y la Seguridad de la explotación.

2.5.3 Tipos de alertas útiles para propietarios agrícolas

En el diseño de sistemas de monitorización y seguridad para zonas agrícolas se tiene que buscar la generación de alertas efectivas, oportunas y veraces, ya que esto contribuirá a una respuesta rápida y adecuada a eventos adversos debidos a la detección de intrusos, fallos del sistema o condiciones ambientales. Por su parte, los propietarios agrícolas pueden requerir la utilización de diferentes canales de comunicación de acuerdo con sus hábitos y las particularidades de las zonas rurales, donde la conectividad y la accesibilidad en el desarrollo de los planes para la seguridad de los cultivos puede contrastar muchísimo.

El correo electrónico como alerta es ampliamente extendido por su universalidad y facilidad de integración en plataformas IoT. Permite el envío de reportes con imágenes o vídeos como adjuntos, ofreciendo un registro formal de un evento que puede consultarse posteriormente. No obstante, puede ser limitado en su uso debido a depender de unos requisitos de conexión establecidos (Botta, 2016).

Las aplicaciones de mensajería instantánea, como la de Telegram y WhatsApp, han sabido convertirse en medios sencillos, rápidos y accesibles mediante los cuales se pueden recibir notificaciones. Es el caso que Telegram, por su parte, tiene una buena API, lo que permite la asociación con los sistemas de control a través del envío de mensajes, imágenes en tiempo real, llegando a permitir incluso el uso de comandos para gestionar el control remoto de objetos, entre otros (Szeliski, 2011). En este sentido, WhatsApp se ha convertido en una aplicación, también empleada frecuentemente, aunque cuando se desea hacer su integración normalmente también se deben considerar otro tipo de soluciones que se producen por sus limitaciones en la API.

Además, las alertas sonoras mediante buzzer o sirenas proporcionan una señal inmediata y directa en el lugar, disuadiendo posibles intrusos y alertando a personas cercanas sin depender de redes de comunicación. Estos dispositivos son especialmente útiles para fincas con baja conectividad o cuando se requiere una respuesta local rápida.

La fusión de diversos canales de alerta mejora la redundancia, logrando que el propietario o responsable reciba la información de alerta, ya sea que existan problemas de red o de disponibilidad de dispositivos, p. ej., un sistema puede desplegar la alerta sonora en el campo que a su vez consista en una notificación Telegram o de correo electrónico con el fin de que el propietario pueda actuar incluso a distancia.

2.5.4 Comunicación en tiempo real desde zonas alejadas o con baja conectividad

La comunicación en tiempo real, sin dudas, es la clave en los sistemas de vigilancia y seguridad aplicados a lugares rurales distantes o con limitada conectividad, por ejemplo, en las plantaciones de aguacate de San José, Mira. Asegurar la transmisión veloz y fiable de datos, alertas y video en estos ambientes representa un problema técnico que debe solucionarse mediante el uso de tecnologías y arquitecturas adecuadas a las características del entorno.

Una de las principales estrategias para poder superar las limitaciones del acceso a la conectividad lo constituyen las tecnologías de larga distancia y de bajo consumo energético, por ejemplo, LoRaWAN (Long Range Wide Area Network) y/o tecnologías de comunicaciones celulares 4G y 5G en caso de estar disponibles. En este sentido, LoRaWAN permite la transmisión de paquetes pequeños de datos a grandes distancias con bajo consumo para sensores IoT que envían alertas o medidas periódicas, aunque LoRaWAN no es apropiada para la transmisión de datos de forma continua en vídeo, debido a que su tasa de datos es muy baja (Akyildiz, 2005).

Simultáneamente, la puesta en práctica del edge computing permite el procesamiento y análisis de datos de forma local, en los dispositivos próximos a la fuente, minimizando así la obligación de tener que transmitir grandes volúmenes de información hacia la nube y permitiendo responder rápidamente a eventos críticos (Sahoo, 2018). Es un enfoque que resulta de gran utilidad en lugares rurales, en los cuales la conectividad presenta una intermitencia o bien, latencia alta.

Para la transmisión de vídeo y datos en tiempo real, es posible hacer uso de técnicas avanzadas de compresión como H.265, o bien el empleo de protocolos optimizados para redes inestables, que minimizan la pérdida de paquetes y que se adaptan a la calidad de acuerdo a la capacidad del canal (Schwarz et al., 2018).

2.5.5 Protocolos de respuesta ante intrusos o emergencias en terrenos agrícolas

Los protocolos de respuesta ante intrusos o emergencias en zonas agrícolas son necesarios desde el punto de vista de prevenir cualquier tipo de daño, proteger a los cultivos y proteger a los trabajadores y a la infraestructura. Los protocolos de respuesta ante intrusos o emergencias en zonas agrícolas están constituidos por un conjunto de acciones coordinadas y automáticas que se ejecutan desde que el sistema de monitorización detecta una amenaza, por ejemplo, la presencia de personas no autorizadas, un incendio o un evento natural adverso.

De entrada, la detección temprana resulta fundamental. Los sistemas han de incluir tecnología como sensores de desplazamiento; cámaras de videovigilancia con análisis en tiempo real; sistemas de reconocimiento de matrículas o identificación que confirmen la presencia de automóviles que tengan autorización (Redmon et al., 2016); una vez se detecte un evento sospechoso, se puedan enviar alertas en tiempo real a los titulares o gestores por medio de SMS, e-mail, aplicaciones de mensajería instantánea (Telegram, WhatsApp) o incluso alarmas acústicas en sitio (Botta et al., 2016).

Luego, los protocolos implican la activación automática de mecanismos de respuesta, como por ejemplo encender las luces, activar sirenas o activar barreras electrónicas que dificulten el avance de los intrusos y alerten a personas cercanas. Así mismo, aconseja mantener comunicación con servicios de seguridad o con los cuerpos de seguridad locales que permitan reaccionar con mayor rapidez y facilitar la intervención en situaciones de emergencia grave y/o de riesgo de vida (Maanak Gupta, 2020).

El protocolo también debe incluir el registro detallado de todos los eventos detectados, almacenando imágenes, videos y logs para análisis forense o judicial. Esto facilita la investigación y prevención de incidentes futuros, además de mejorar continuamente la configuración del sistema de seguridad.

2.6 Análisis comparativo de tecnologías de hardware para el sistema de seguridad

Se desarrollará un proceso de enseñanza activo y participativo, en el que los estudiantes aprenderán mediante actividades prácticas, dinámicas y cercanas a su realidad. Se fortalecerán las habilidades semánticas, léxicas, sintácticas y fonológicas con apoyo de imágenes, materiales concretos, alfabetos móviles y ejercicios de reconocimiento de sonidos y palabras. También se favorecerá la lectura, la escritura y la expresión oral mediante actividades individuales, en parejas y grupales. Se incorporarán juegos y recursos digitales para reforzar

los aprendizajes, mientras que la evaluación será permanente para acompañar el progreso de cada estudiante y brindar apoyo cuando sea necesario.

El objetivo es determinar qué componentes ofrecen el mejor equilibrio entre rendimiento y autonomía para la detección de vehículos y lectura de matrículas en tiempo real.

2.6.1 Comparativa de unidades de procesamiento (Edge Computing){

La capa de procesamiento requiere un dispositivo capaz de ejecutar algoritmos de visión por computadora (como YOLO) localmente, reduciendo la dependencia de la nube. La Tabla 2 compara las características de los microcontroladores y microordenadores más utilizados en IoT agrícola. (Raspberry Pi, 2023)

Tabla 2.
Comparativa de placas de procesamiento para visión artificial en el borde

Característica	ESP32-CAM	Raspberry Pi 4B (4GB/8GB)	Raspberry Pi 5 (8GB)
Arquitectura	Microcontrolador (Dual-core 240MHz)	Microordenador (Quad-core A72 1.5GHz)	Microordenador (Quad-core A76 2.4GHz)
Capacidad de IA (YOLO)	Muy Baja (Solo clasificación básica o streaming)	Media (3-8 FPS en modelos ligeros)	Alta (15-30+ FPS, ideal para tiempo real)
Memoria RAM	~520 KB (Muy limitada)	4GB / 8GB LPDDR4	8GB LPDDR4X (Alta velocidad)
Gestión de Video	Baja resolución / Streaming MJPEG	Hasta 4K, codificación H.265 por software	Dual 4K, Arquitectura de ISP mejorada

Conectividad	Wi-Fi / Bluetooth (Integrado)	Wi-Fi 5 / BT 5.0 / Gigabit Ethernet	Wi-Fi 802.11ac / BT 5.0 / PCIe para SSD/4G
Efectividad para el Proyecto	Insuficiente para reconocimiento de placas (OCR) local.	Funcional, pero con limitaciones de latencia en detección compleja.	Óptima. Permite ejecutar SO completo, base de datos local y IA simultánea.

Nota: Elaboración propia basada en especificaciones técnicas de fabricantes.

El análisis pone de manifiesto que, pese a la eficiencia del ESP32, tiene una falta de potencia para el reconocimiento de matrículas. La Raspberry Pi 5 está por delante con su anterior modelo (Pi 4) gracias a su procesador A76 y memoria LPDDR4X, lo que asegura que la detección de intrusos no quede entorpecida.

2.6.2 Comparativa de módulos de adquisición de imagen

La efectividad del reconocimiento de placas (LPR/ANPR) va ligado de manera directa a la calidad del sensor de imagen y la calidad de la óptica utilizada. En la Tabla 3 se comparan muchos módulos de cámara compatibles con la CSI (Camera Serial Interface) de la Raspberry Pi, conforme a su rendimiento al aire libre (Raspberry Pi, 2023).

Tabla 3.

Comparativa de módulos de cámara para sistemas de seguridad basados en Raspberry Pi

Característica	Cámara Oficial V2 (8MP)	Webcam USB Estándar (1080p)	Arducam IMX477 HQ (12.3MP)
Sensor	Sony IMX219	Varios (CMOS genérico)	Sony IMX477 (Alta fidelidad)

Resolución	8 Megapíxeles	2 Megapíxeles (1920x1080)	12.3 Megapíxeles (4056x3040)
Tipo de Lente	Fijo (No intercambiable)	Fijo / Enfoque manual limitado	Intercambiable (Montura CS/C) con enfoque y zoom ajustable
Interfaz de Conexión	MIPI CSI-2	USB 2.0 / 3.0	MIPI CSI-2 (Baja latencia directa a CPU)
Sensibilidad (Poca Luz)	Baja (Mucho ruido visual)	Media	Alta (Mejor rango dinámico para exteriores)
Efectividad para LPR	Media. Dificultad para leer placas a distancia (>5m).	Baja. Latencia alta por bus USB y compresión.	Alta. Permite ajustar el lente para enfocar la zona de paso vehicular.
Adaptabilidad Externa	Requiere adaptación compleja.	Generalmente plástico simple.	Compatible con carcasas industriales IP65.

Nota: Datos comparativos de cámaras compatibles

Se puede concluir que la cámara Arducam IMX477 es la mejor solución para este trabajo, ya que la posibilidad de utilizar lentes de montura CS permite realizar un ajuste óptico exacto hacia el camino de entrada y maximizar la densidad de píxeles en la matrícula del vehículo, lo que es un requisito indispensable que fue destacado en el apartado de los desafíos tecnológicos. Aparte de eso, esta cámara además es nativamente compatible con la Raspberry Pi 5 a través del puerto CSI, lo que va a garantizar un procesamiento de imagen libre de estrangulamientos.

3.CAPÍTULO III: DISEÑO DEL SISTEMA DE MONITOREO

3.1. Metodología

La metodología utilizada para la elaboración de este trabajo de fin de titulación ha sido la cascada (Waterfall), la cual se encuentra caracterizada para un tipo de desarrollo de proyectos a través de su estructura secuencial y por fases. Partiendo de ser un modelo que nos da la posibilidad de establecer que cada fase del desarrollo tiene que estar finalizada antes de comenzar la siguiente, además de permitir que el avance del proyecto sea controlado al poder garantizar que cada una de las fases se basa en el nivel de análisis y de planificación necesario (Sommervill, 2014).

El modelo de proceso ideal en cascada es uno de los más utilizados en proyectos de ingeniería y de desarrollo de sistemas tecnológicos, puesto que permite especificar de manera clara y precisa los requerimientos desde el comienzo del proyecto y permite preparar la planificación de diseño, la planificación de implementación y la planificación de evaluación del sistema. En este tipo de proceso, se descompone el proceso de desarrollo mediante fases claramente definidas, tales como la de análisis de requisitos, la fase de diseño de sistema, la fase de implementación de prototipos y la fase de pruebas y validación.

La aplicación de esta metodología resulta adecuada para el caso del presente proyecto, ya que los requerimientos del sistema de monitoreo y seguridad, basado en arquitectura IoT, para el sector San José, cantón Mira, fueron identificados desde las etapas iniciales del estudio, lo cual permitió que se pueda establecer una planificación técnica clara para el desarrollo del prototipo, la selección del hardware, así como la integración de los componentes requeridos para su funcionamiento.

Este enfoque asegurará un buen paso entre las diferentes etapas del desarrollo, pues facilita permitir una evaluación progresiva del cumplimiento del sistema respecto al

cumplimiento de los objetivos previstos. La Figura 5 presenta el esquema general de la metodología en cascada en el desarrollo del sistema de monitoreo que se establece.

Figura 5. Metodología en cascada aplicada al sistema de monitoreo IoT



Nota: La figura ilustra las fases metodológicas utilizadas para el desarrollo del sistema de monitoreo y seguridad para la detección de vehículos en la vía agrícola del sector San José, Mira. Fuente: Autoría

Este modelo se compone de cuatro fases principales, las cuales se describen a continuación:

- **Analizar:** En esta fase se realizó un diagnóstico del estado actual del sector San José, correspondiente al cantón de Mira, mediante el cual se identificaron los problemas de la seguridad asociados al ingreso de vehículos ajenos a las vías del sector agrícola. Con ese objetivo, se llevó a cabo el levantamiento de la información del entorno, considerándose la ubicación geográfica, condiciones de la vía, iluminación y necesidades los predios. Con el fin de caracterizar la realidad en la que se quiere desarrollar un sistema de monitoreo, se determinaron los requerimientos técnicos necesarios para el desarrollo de tal sistema.
- **Diseñar:** En esta fase se definió la arquitectura general del sistema (para un sistema basado en IoT) de la manera en que la forma en la que interactuarán entre sí los dispositivos que capturan información, el sistema de procesado y los mecanismos de alarmas. Además, se procesaron estos criterios para la selección del hardware principal del sistema, es decir, para la Raspberry Pi 5 como unidad de procesado y el módulo de cámara Arducam para la captura de imágenes, teniendo en cuenta criterios de

rendimiento, precisión y resistencia a las condiciones medioambientales del entorno agrícola.

- **Implementar:** En esta fase se implementó el prototipo del sistema a través de la instalación y configuración del hardware que fue elegido. También se llevó a cabo la integración de los diferentes elementos que forman parte de la arquitectura IoT. A su vez, se implementó el software que se encarga de todo el procesamiento de las imágenes capturadas por la cámara, utilizando algoritmos sobre el reconocimiento de vehículos y el desarrollo de alertas automáticas a los usuarios. Finalmente, se realizaría la configuración del sistema operativo y de las herramientas que son necesarias para el funcionamiento del sistema de monitoreo.
- **Comprobar:** Por último, se realizaron comprobaciones de funcionamiento en las condiciones reales del sector San José con el objetivo de comprobar la funcionalidad del sistema relacionado con la detección de las personas y los vehículos que circulan por la vía agrícola; en esta etapa se corroboró la adecuada captura de imágenes, el funcionamiento del algoritmo de detección y de alerta hacia los usuarios; y se obtuvieron resultados que validaron la funcionalidad del prototipo y que permitieron realizar los ajustes oportunos con la finalidad de optimizar su funcionalidad.

3.2. Etapa de Análisis

En esta etapa, se realiza un análisis de la situación actual respecto a la seguridad y monitoreo en las plantaciones de aguacate en el sector San José, identificando las vulnerabilidades, los desafíos logísticos y los factores que facilitan el ingreso de intrusos. Este análisis permite establecer los requerimientos del sistema de monitoreo con arquitectura IoT y procesamiento de video, delimitando el alcance del proyecto y sentando las bases para el diseño e implementación de una solución tecnológica autónoma que optimice la vigilancia y garantice la protección de los cultivos.

3.2.1. Situación Actual y problemática en el sector San José

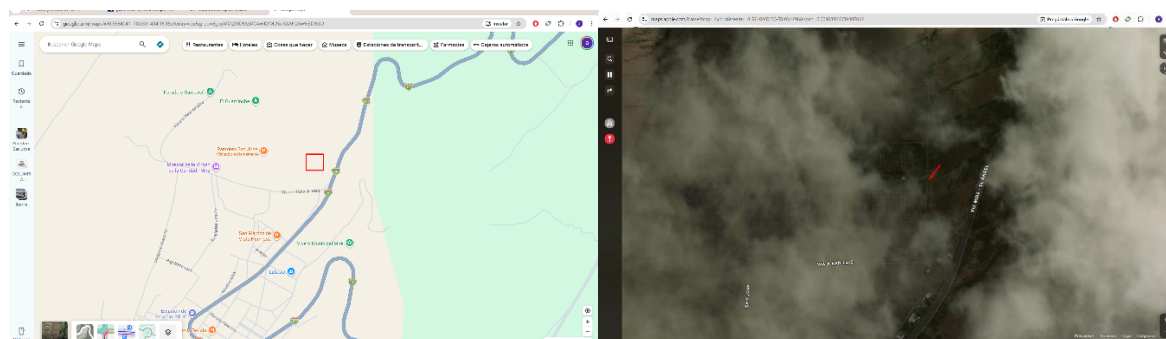
El sector San José (Pueblo Viejo), perteneciente al cantón Mira en la provincia del Carchi, se encuentra a una altitud de 2,100 msnm. La economía y el desarrollo de esta zona dependen en gran medida de la agricultura, destacándose el cultivo de aguacate como una de las principales fuentes de ingresos para los productores locales. Su ubicación estratégica, situada sobre la Vía a el Hato de Mira y utilizando como referencia principal el Paradero San José, convierte a estas plantaciones en un punto clave para la actividad agrícola y comercial de la región.

Sin embargo, en la actualidad, la seguridad en las plantaciones de aguacate del sector enfrenta importantes limitaciones. El control de acceso y la vigilancia se realizan, en su mayoría, de manera manual o mediante recorridos esporádicos por parte de los cuidadores. Este método resulta ineficiente para cubrir grandes extensiones de terreno, especialmente durante la noche o en zonas perimetrales de difícil acceso. Esta falta de monitoreo continuo ha incrementado la vulnerabilidad del sector ante el robo de las cosechas y el ingreso de personas o vehículos no autorizados.

El problema primordial se encuentra en la falta de un sistema automatizado que avise a tiempo real sobre intrusiones. El hecho de no poder detectar con antelación lo que está pasando, significa que los propietarios no tienen un tiempo para reaccionar ante un posible acontecimiento. Dicho proyecto se llevará a cabo en una zona representativa de este sector. Por su parte, como la topografía concreta de la zona y los linderos de las plantaciones no quedan expresados con grado suficiente en mapas satelitales, el posicionamiento exacto del prototipo y su entorno quedan expresados en el croquis correspondiente en la figura. El entorno rural que se ha escogido presenta dificultades para la operación como una conectividad reducida; en este sentido, se hace aún más necesario un sistema de IoT en el

borde (Edge Computing) que detecte movimiento, que procese el vídeo a partir de una localización concreta y que realice un envío de alertas adecuadas para proteger la producción.

Figura 6. Ubicación de la zona de implementación del prototipo en el Sector San José



Nota: Ubicación aproximada del entorno agrícola donde se desplegará el nodo de monitoreo

IoT. Fuente: Google Maps, Apple Maps

Este contexto rural va acompañado de dificultades operativas, destacando la escasa conectividad, que aumenta la imperiosa necesidad de un sistema IoT en el borde (Edge Computing) que permita la detección de movimiento, procese el vídeo de forma local y envíe todo tipo de alertas para salvar la producción. Las características físicas de la vías de acceso principal, al estar rodeadas de vegetación abundante y variación topográfica, se muestran en la Figura 7, justificando la implementación de un sistema de lentes varifocales y de sensores de movimiento con alta sensibilidad que eviten falsos positivos derivados del entorno natural.

Figura 7. Sector San José.



Nota: Condiciones topográficas y visuales de la vía de acceso principal a las plantaciones de aguacate en el sector San José.

Fuente: Autoría

3.3. Etapa de Planificar

De acuerdo con el proceso estructurado que abarca la metodología en cascada utilizada en el presente trabajo de titulación, la etapa de planificación establece el marco metodológico y las estrategias técnicas necesarias para realizar la implementación del sistema de monitoreo en el sector San José. A través de su estudio, se determinan los requerimientos técnicos y operativos que marcarán la guía del diseño de la solución de arquitectura IoT y procesamiento de video.

Con ello, se define un enfoque que permite asegurar la coherencia entre los objetivos de seguridad planteados y la solución tecnológica desarrollada; ello incluye la redacción y organización estructurada de los requisitos, definiendo una nomenclatura clara en la que se contraponen requerimientos funcionales, de usuario y de arquitectura. Esta fase permite garantizar que cada fase posterior del diseño responda a las verdaderas necesidades realistas que se hayan podido identificar en el entorno agrícola de las plantaciones de aguacate, asegurando que el sistema sea eficiente, autónomo y cumpla con la finalidad de alertar sobre la posibilidad de las intrusiones.

3.3.1. Identificación de Actores y Stakeholders

Para el diseño y la implementación de un sistema de monitoreo de las plantaciones de aguacate del sector San José con arquitectura IoT y procesamiento de video para la seguridad, se identificaron los principales stakeholders que tienen un papel directo o indirecto en el desarrollo del proyecto. Los stakeholders involucrados van desde los propietarios o encargados

de plantaciones hasta los académicos, que son los que dirigen y asesoran el proyecto. La correcta identificación de los propios stakeholders implica también una buena comunicación, entender las expectativas con respecto a la seguridad y asegurarse que el sistema automatizado entregue los objetivos según son requeridos, detección en forma oportuna de intrusos y vehículos. En el cuadro siguiente podemos observar la tabla de los stakeholders involucrados en el presente proyecto.

A continuación, en la Tabla 4, se detallan los actores clave identificados durante la investigación de campo:

Tabla 4.

Identificación de Stakeholders

No	Rol	Nombres
1	Representante del Sector San José	Edgar Ortega
2	Encargado de Seguridad	Blanca Lopez
3	Director	Ing. Jaime Roberto Michilena Calderón, MSc
4	Asesor	Ing. Suárez Zambrano Luis Edilberto, MSc
5	Desarrollador	Alder Stalin Navarrete Tipas

Nota: La tabla presenta la lista de stakeholders involucrados en el proyecto, incluyendo sus roles y nombres. Fuente: Autoría

3.3.2. Nomenclatura de Requerimientos

A fin de garantizar que exista la organización y trazabilidad de los requerimientos técnicos propuestos para el desarrollo del sistema de monitoreo IoT, se presenta una nomenclatura que clasifica los requerimientos técnicos conforme el origen de los mismos y la naturaleza. Esta codificación permite distinguir entre aquellos requerimientos realizados por los stakeholders, aquellos que representan requisitos operacionales del sistema de visión

artificial y aquellos cuya naturaleza está relacionada con la arquitectura hardware-software. La tabla 5 presenta la nomenclatura utilizada.

Tabla 5.

Nomenclatura de Requerimientos

No	Abreviatura	Descripción
1	StSR	Requerimientos planteados por los stakeholders.
2	SySR	Requerimientos técnicos del sistema de monitoreo y procesamiento de video.
3	ArQR	Requerimientos relacionados con la arquitectura general IoT e infraestructura de hardware.

Nota: Nomenclatura de Requerimientos para ser utilizado. Fuente: Autoría

Así pues, una vez que se ha realizado la definición de los requerimientos que se quiere incluir en el sistema, es necesario establecer un orden de prioridad para cada uno de ellos de forma que se puedan dar respuesta a las características más importantes en la red de la red de monitorización. La utilización de prioridades permite orientar mejor los recursos tanto computacionales como energéticos de forma que se puedan tomar decisiones más ajustadas en el momento de llevar a cabo el desarrollo. Proceso que resulta bastante interesante y adecuado, por el hecho que se pueden establecer las prioridades en la definición de los requerimientos que son los que realmente afectan a la detección de intrusos. En la tabla siguiente vemos como establecemos la clasificación de los requerimientos en función de su grado de importancia.

3.3.2.1. Priorización de los Requerimientos del Sistema.

Una vez que se han definido los requerimientos del sistema, es clave dar una prioridad a cada uno. Así se asegura que se atiendan primero los puntos más importantes para que la red de monitoreo funcione bien. Priorizar los objetivos ayuda a usar mejor los recursos de cómputo

y energía. También permite tomar decisiones claras durante el desarrollo. Esto resulta muy útil al trabajar con sistemas embebidos, porque ayuda a enfocar el trabajo en los procesos que afectan de forma directa la detección de intrusos. En la siguiente tabla se muestra cómo se agrupan los requerimientos según su nivel de importancia.

Tabla 6.

Priorización de los Requerimientos del Sistema

No	Prioridad	Descripción
1	Alta	Indispensable para el funcionamiento adecuado del sistema. Su ausencia impactaría directamente en la detección de intrusos y la operatividad general de la seguridad.
2	Media	Puede ser pospuesto bajo ciertas circunstancias. Si se omite, puede afectar parcialmente la eficiencia del monitoreo, pero no impide las alertas básicas del sistema.
3	Baja	Su ausencia no afecta significativamente el funcionamiento principal de seguridad. Es un aspecto que puede ser implementado de forma opcional o posterior.

Nota: La tabla muestra la priorización de los requerimientos del sistema, clasificándolos en alta, media y baja según su impacto en el funcionamiento general del sistema. Fuente: Autoría

3.3.2.2. Requerimientos Operacionales y de Usuario.

La definición de los requerimientos del sistema constituye un paso fundamental para garantizar tanto su viabilidad técnica como su utilidad práctica. A continuación, se clasifican estas directrices en dos categorías principales: los requerimientos operacionales (Tabla 7), que establecen las condiciones técnicas necesarias para la captura, el procesamiento en el nodo local (Edge Computing) y la autonomía energética del hardware; y los requerimientos del usuario (Tabla 8), enfocados en la usabilidad, la recepción oportuna de alertas y el acceso a la

evidencia visual, asegurando que la solución tecnológica responda de manera eficaz a las necesidades de seguridad en el entorno agrícola.

Tabla 7.

Requerimientos Operacionales

No	Requerimiento	Descripción	Alta	Media	Baja
1	Detección de movimiento por hardware	El sistema debe integrar un sensor PIR (HC-SR501) para activar el procesamiento de video solo cuando exista movimiento, ahorrando energía.	X		
2	Captura de video de alta resolución	El sistema debe capturar imágenes nítidas que permitan el análisis posterior de personas y placas vehiculares.	X		
3	Procesamiento de video local (Edge Computing)	El sistema debe ejecutar los algoritmos de detección de objetos localmente para reducir la dependencia de la red.	X		
4	Conectividad a red de datos	El sistema debe permitir la conexión a Internet para el envío de alertas mediante un bot de mensajería.	X		
5	Gestión de energía eficiente	El sistema debe operar 24/7 de forma autónoma, requiriendo un dimensionamiento adecuado de paneles solares y baterías.	X		
6	Almacenamiento local temporal	Los eventos detectados deben almacenarse localmente de forma temporal en caso de pérdida de conexión a la red.		X	

Nota: Requerimientos fundamentales para su desarrollo e implementación. Fuente: Autoría

Tabla 8.

Requerimientos del Usuario

No	Requerimiento	Descripción	Alta	Media	Baja
1	Alertas en tiempo real	El usuario debe recibir notificaciones inmediatas (vía Telegram) cuando se detecte una intrusión de personas o vehículos.	X		
2	Evidencia visual	Las alertas deben incluir un identificador claro del evento detectado para su validación por parte del usuario.		X	
3	Operación autónoma sin mantenimiento frecuente	El sistema no debe requerir intervención manual constante del		X	

		usuario para funcionar en el día a día.	
4	Interfaz de consulta de estado	El usuario debe poder consultar si el sistema está activo mediante comandos sencillos.	X
5	Control de flujo de notificaciones	El sistema no debe saturar al usuario con mensajes repetitivos si un intruso permanece en la misma zona.	X
6	Historial de eventos	El usuario debe poder acceder a un registro de los eventos detectados previamente.	X

Nota: La tabla presenta los requerimientos del usuario, detallando las necesidades y expectativas específicas que deben cumplirse. Fuente: Autoría

3.3.3. Requerimiento de Stakeholders

El sistema propuesto debe alinearse estrictamente con los requerimientos establecidos por las partes interesadas, considerando de forma prioritaria la necesidad de los propietarios de asegurar las plantaciones de aguacate en el sector San José. Estos requerimientos, detallados en la Tabla 9, abarcan aspectos operativos clave y expectativas funcionales, siendo elementos esenciales para la evaluación del nodo de monitoreo. Su cumplimiento permitirá determinar si la solución IoT desarrollada responde de manera efectiva al problema de intrusiones no autorizadas.

Tabla 9.

Requerimiento de Stakeholders

StSR	Requerimiento del Stakeholder	Prioridad (Alta)	Prioridad (Media)	Prioridad (Baja)
	REQUERIMIENTOS DE FUNCIONAMIENTO			
StSR1	El sistema debe instalarse a la intemperie, soportando las condiciones climáticas del sector San José (sol, lluvia, polvo, humedad).	X		
StSR2	El sistema debe ser capaz de discriminar entre el movimiento del follaje y el movimiento real de personas o vehículos para evitar falsas alarmas.	X		
StSR3	El sistema debe asegurar el envío de alertas fotográficas mediante una	X		

	conexión inalámbrica estable (red celular o Wi-Fi de largo alcance).	
StSR4	El sistema debe basarse en arquitectura IoT para permitir la futura escalabilidad (añadir más nodos de cámaras en el perímetro).	X
StSR5	El sistema debe optimizar el consumo de energía, operando únicamente cuando se detecte una posible intrusión.	X
StSR6	El sistema debe tener un diseño discreto y robusto para evitar el vandalismo o robo del propio equipo de monitoreo.	X
REQUERIMIENTO DE USUARIO		
StSR7	Las notificaciones en la interfaz (Telegram) deben indicar claramente el tipo de objeto detectado (persona o vehículo) y la hora exacta del evento.	X
StSR8	El sistema debe ser sostenible económicamente, minimizando los costos de mantenimiento y operación mensual.	X
StSR9	La instalación y configuración inicial debe estar documentada, permitiendo que componentes averiados puedan ser reemplazados con partes del mercado local.	X
StSR10	El nodo principal (caja estanca, panel y cámara) debe ser fácilmente transportable en caso de requerir reubicación a otra zona de la plantación.	X

Fuente: Autoría

3.3.4. Requerimientos del sistema

Los requerimientos del sistema abarcan las funcionalidades y especificaciones técnicas necesarias para que el sistema automatizado de monitoreo con procesamiento de video funcione de manera eficiente en el entorno agrícola del sector San José. Estos requerimientos incluyen aspectos relacionados con la latencia de las alertas, la capacidad de inferencia local (Edge Computing), la gestión de la alimentación ininterrumpida, así como las condiciones de resguardo de los componentes electrónicos. El cumplimiento de estos requisitos es fundamental para asegurar que el sistema propuesto cumpla con los objetivos de vigilancia perimetral autónoma. La siguiente tabla 8 detalla estos requerimientos.

Tabla 10.

Requerimientos del sistema (SySR)

SySR	Requerimientos del Sistema	Prioridad
REQUERIMIENTO DE USO		
SySR1	El sistema debe mantenerse en un estado de bajo consumo (reposo) hasta que el sensor de movimiento (PIR) detecte actividad térmica.	Alta
SySR2	El sistema debe contar con comunicación remota bidireccional mediante un bot conversacional para el envío de alertas y consulta de estado.	Alta
SySR3	El sistema debe garantizar una fuente de energía continua y estable (solar) para su funcionamiento ininterrumpido 24/7.	Alta
SySR4	La operación diaria del sistema debe ser completamente autónoma, sin requerir intervención manual del agricultor.	Alta
SySR5	El sistema debe registrar localmente (en una tarjeta microSD) un historial de eventos en formato CSV en caso de pérdida temporal de conectividad.	Media
REQUERIMIENTO DE PERFORMANCE		
SySR6	El tiempo de respuesta entre la detección de movimiento y el envío de la alerta con evidencia visual no debe superar los 15 segundos.	Alta
SySR7	El algoritmo de visión artificial debe ejecutar la inferencia de forma local, procesando un mínimo de 5 fotogramas por segundo (FPS) durante una alerta.	Alta
SySR8	El sistema debe implementar una lógica de "cooldown" (tiempo de enfriamiento) para no emitir alertas repetitivas de un mismo objetivo estacionario.	Alta
REQUERIMIENTOS DE INTERFACES		
SySR9	El sistema debe utilizar la interfaz MIPI CSI para la conexión de la cámara, garantizando el ancho de banda necesario para video de alta definición.	Alta
SySR10	El sensor de movimiento debe generar interrupciones por hardware a través de los pines GPIO del microprocesador.	Alta
REQUERIMIENTOS FÍSICOS		
SySR11	Los componentes electrónicos deben estar resguardados en una carcasa con certificación IP65 o superior, protegidos contra agua y polvo.	Alta
SySR12	La cámara debe estar posicionada estratégicamente en una estructura elevada para evitar obstrucciones visuales por el crecimiento del aguacate.	Alta
REQUERIMIENTOS DE MODOS Y ESTADOS		
SySR13	El sistema debe contar con un mecanismo de reinicio automático (Watchdog) en caso de fallos críticos del software o del sistema operativo.	Alta
SySR14	El sistema debe reanudar automáticamente la ejecución del script de detección de intrusos tras un corte y posterior restablecimiento de energía.	Alta

Nota: La tabla muestra los requerimientos esenciales para el funcionamiento eficiente del sistema de seguridad. Fuente: Autoría

La propuesta de sistema ha sido formulada en base a las necesidades operativas reales de las plantaciones en el sector San José, donde hoy la vigilancia de perímetro se realiza de manera manual o esporádica, lo que resulta ineficaz para cubrir tal extensión y deja los cultivos expuestos al robo. Con la intención de acometer esta situación se fijaron una serie de requerimientos técnicos orientados en poder lograr un monitoreo automatizado y proactivo.

Uno de los requisitos principales es la ejecución de edge computing. Esto se debe a que en las zonas rurales es complicado disponer de internet de calidad; por tanto, no se puede estar transmitiendo vídeo continuamente a una nube. El sistema debe implementar los algoritmos de IA (YOLOv8, EasyOCR) en la propia placa principal para realizar análisis de fotogramas, llevar a cabo la detección de amenazas (personas o vehículos) y la interpretación de datos (placas).

Dado que la monitorización debe ser continua pero el consumo energético mínimo, se adopta un sensor IR pasivo (PIR) como disparador de la captura y así, se asegura de que la cámara y los algoritmos de IA se activen sólo en el caso que exista una probabilidad real de intrusión. Para asegurar que el agricultor reciba la información de forma inmediata, se plantea realizar la solución utilizando la API de Telegram, lo que permite, al agricultor, recibir alertas con imágenes directamente en su teléfono móvil. La Tabla 9 recopila todos los requerimientos técnicos que se consideran necesarios para el correcto uso de la arquitectura.

Tabla 11.

Requerimientos Técnicos Consolidados

Necesidad Identificada	Requerimiento Técnico
Vigilancia continua sin intervención humana	Integración de algoritmos de detección de objetos y OCR ejecutados en un microprocesador local.

Reducción del consumo de ancho de banda	Procesamiento local (Edge AI) que solo transmite metadatos e imágenes clave, no video continuo.
Ahorro de energía en estado de reposo	Uso de interrupciones de hardware mediante sensor PIR para despertar el módulo de captura de video.
Notificación remota e inmediata al usuario	Implementación de un bot conversacional (Telegram API) para el envío de alertas y fotografías.
Operación ininterrumpida sin red comercial	Diseño de un sistema de alimentación off-grid mediante paneles solares y baterías de respaldo.
Protección contra condiciones ambientales	Ensamblaje del nodo en cajas estancas (IP65) y uso de interfaces de hardware robustas (CSI/GPIO).
Tolerancia a fallos de conectividad	Almacenamiento local temporal de registros y evidencias fotográficas (Data Logging).

Nota: La tabla presenta los requerimientos técnicos fundamentales para asegurar el correcto funcionamiento del sistema de seguridad IoT. Fuente: Autoría

3.3.5. Requerimientos de Arquitectura

Los requerimientos de arquitectura definen las condiciones técnicas necesarias para que el sistema de monitoreo IoT funcione de manera eficiente, autónoma y coherente. Estos requisitos incluyen aspectos relacionados con el software de visión artificial, el hardware de procesamiento, los componentes eléctricos solares y el diseño físico. Estos parámetros guían la selección definitiva de dispositivos y herramientas para el sistema. La Tabla 12 muestra los requerimientos establecidos, asegurando que cada componente cumpla con los estándares operacionales para garantizar una detección efectiva y un manejo seguro del equipo a la intemperie.

Tabla 12.

Requerimientos de Arquitectura

ArQR	Requerimiento de Arquitectura	Prioridad
REQUERIMIENTOS LÓGICOS		
ArQR 1	El nodo debe contar con puertos GPIO para la lectura de interrupciones digitales provenientes del sensor de movimiento.	Alta
ArQR 2	El sistema debe utilizar la interfaz MIPI CSI-2 para garantizar el ancho de banda necesario en la transmisión de video desde la cámara al procesador.	Alta
REQUERIMIENTO DE DISEÑO		
ArQR 3	El diseño del sistema debe ser estanco (certificación IP65 o superior), protegiendo la electrónica del polvo y la lluvia.	Alta
ArQR 4	El sistema debe ser modular, permitiendo separar la estructura del panel solar de la caja principal de procesamiento si la topografía lo requiere.	Media
REQUERIMIENTO DE HARDWARE		
ArQR 5	El microcomputador debe poseer suficiente memoria RAM y capacidad de CPU para ejecutar algoritmos de inferencia (Edge AI) en tiempo real.	Alta
ArQR 6	La cámara debe admitir lentes varifocales para ajustar el campo de visión a las dimensiones específicas de las vías de acceso en la plantación.	Alta
REQUERIMIENTO DE SOFTWARE		
ArQR 7	El sistema operativo debe ser compatible con entornos de ejecución en Python y librerías de visión artificial (OpenCV, PyTorch, Ultralytics).	Alta
ArQR 8	Se debe implementar un control lógico (cooldown) para evitar la saturación de mensajes de alerta ante un mismo objetivo estacionario.	Alta
REQUERIMIENTO ELÉCTRICOS		
ArQR 9	Contar con alimentación eléctrica totalmente autónoma mediante un arreglo fotovoltaico y baterías de respaldo para funcionamiento 24/7.	Alta

Nota: La tabla presenta los requerimientos de arquitectura necesarios para asegurar un monitoreo perimetral efectivo. Fuente: Autoría

3.3.6. Selección de Hardware

La selección de hardware abarca la evaluación de componentes esenciales para el funcionamiento del nodo Edge, como sensores de movimiento, captura de imagen, microcomputadores y sistemas de alimentación. La correcta elección de estos elementos es crucial para garantizar la efectividad del modelo y la durabilidad del equipo a la intemperie.

3.3.6.1. Selección de Sensor de Movimiento.

Para evitar que el sistema procese video de forma ininterrumpida y agote la batería, es necesario un sensor que despierte al microprocesador. El sensor debe ser capaz de discriminar elementos térmicos (personas/vehículos) en un entorno agrícola, cumpliendo con los requerimientos operativos de la arquitectura.

Tabla 13.

Selección de Sensor de Movimiento

Sensor Evaluado	StSR2	StSR5	SySR1	SySR10	Valoración Total
Sensor PIR (HC-SR501)	1	1	1	1	4
Radar de Microondas (RCWL-0516)	0	1	1	1	3
Sensor Ultrasónico (HC-SR04)	0	1	0	1	2

Fuente: Autoría

Tras el análisis de los datos de la Tabla 13, se determinó que el Sensor PIR HC-SR501 es la mejor opción. A diferencia del radar de microondas, que puede traspasar el follaje y generar falsas alarmas de ramas moviéndose (incumpliendo StSR2), el PIR sólo detecta la firma térmica de los cuerpos. Además, se integra directamente mediante los pines GPIO de la

placa (SySR10) y mantiene el sistema en reposo (SySR1) optimizando al máximo la energía (StSR5).

3.3.6.2. Selección de Cámara de Video.

La cámara es el dispositivo de entrada principal para el algoritmo de inteligencia artificial. Debe proporcionar imágenes de alta calidad para la detección de objetos y reconocimiento de placas vehiculares, adaptándose a la óptica necesaria en las vías de la plantación.

Tabla 14.

Selección de Cámara de Video

Cámara Evaluada	StSR3	SySR7	SySR9	SySR12	Valoración Total
Arducam IMX477 HQ (12MP)	1	1	1	1	4
Raspberry Pi Camera V2 (8MP)	1	1	1	0	3
Cámara Web USB Genérica	0	0	0	0	0

De acuerdo con la evaluación que se encuentra en la Tabla 14, se ha decidido optar por la Arducam IMX477 HQ al tratarse de la adecuada, tal y como se puede observar en la figura 8. Esta cámara cumple todos los requisitos técnicos exigidos; su principal característica es que tiene una montura tipo CS apta para un acoplamiento de lentes varifocales (SySR12) necesario para ajustar el enfoque a ciertas distancias. Asimismo, esta cámara hace uso de la interfaz nativa MIPI CSI (SySR9) que permite asegurar que el ancho de banda sea suficiente

para que el algoritmo que implementa YOLO lleve a cabo la inferencia de forma adecuada (SySR7) y para que envíe alertas visuales claras (StSR3).

Figura 8. Arducam IMX477 HQ (12MP)



Fuente: Amazon

3.3.6.3. Selección de Microcomputadora.

El microordenador es el centro del sistema, que se encarga de realizar los algoritmos de visión artificial de forma local (Edge Computing) sin depender de la nube, por lo que necesita notable capacidad de procesamiento y memoria RAM.

Tabla 15.

Selección de Microcomputadora

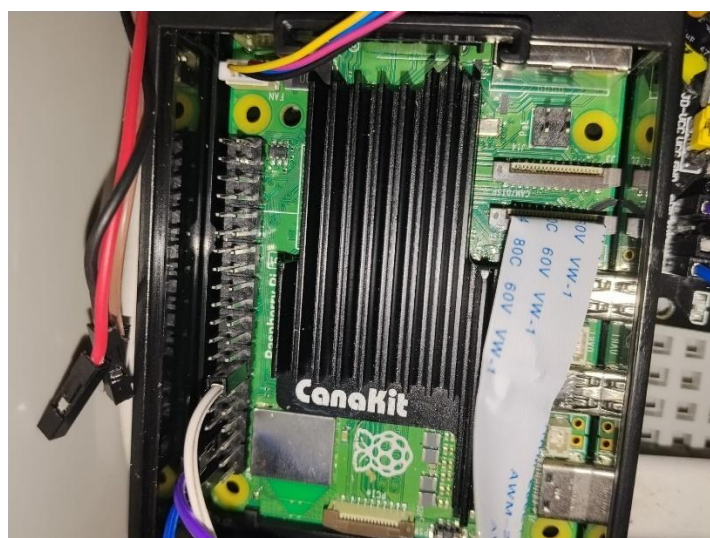
Placa Evaluada	StSR4	SySR2	SySR5	SySR7	Valoración Total
Raspberry Pi 5 (8GB)	1	1	1	1	4
Raspberry Pi 4 (4GB)	1	1	1	0	3
Nvidia Jetson Nano	1	1	0	1	3

Fuente: Autoría

De conformidad con las especificaciones, la Raspberry Pi 5 (modelo de 8GB de RAM) es la placa seleccionada (ver Figura 9). Como se observa en la Tabla 13, su salto generacional

en tonelada de CPU permite alcanzar tasas de fotogramas adecuadas para tiempo real con los modelos YOLOv8n y EasyOCR (SySR7), con un rendimiento superior a la Raspberry Pi 4. Le proporciona, además, la conectividad y la RAM necesaria para gestionar el bot conversacional de alertas (SySR2) y almacenar eventos localmente en el caso de desconexión (SySR5), sin dejar de integrarse en el resto de la arquitectura IoT (StSR4).

Figura 9. Raspberry pi 5 placa



Fuente: Autoría

3.3.6.4. Selección de Fuente de Alimentación Autónoma.

Para garantizar el funcionamiento ininterrumpido del nodo de seguridad en el sector agrícola, donde no existe conexión a la red eléctrica comercial, se debe elegir un sistema de generación y almacenamiento de energía confiable y de bajo mantenimiento.

Tabla 16.

Selección de Fuente de Alimentación

Fuente Evaluada	StSR1	StSR8	SySR3	SySR4	Valoración Total
Sistema Solar Fotovoltaico	1	1	1	1	4
Microturbina Eólica	1	0	0	1	2
Banco de Baterías Portátil	1	0	0	0	1

Fuente: Autoría

El sistema solar fotovoltaico (combinación de panel y batería de ciclo profundo) es el tipo de sistema elegido al obtener el mayor valor en la tabla 16. A diferencia de las turbinas eólicas que dependen de ráfagas de viento invariables y que acaban sufriendo un desgaste mecánico, el panel solar se adapta perfectamente al clima de la zona y permite que la planta se mueva de forma autónoma.

Para que el sistema de visión artificial (YOLOv8 y easyOCR) pueda funcionar adecuadamente durante la noche es necesario tener algún tipo de iluminación artificial.

Para que el nodo central de Edge AI funcione deben generarse dos niveles de tensión diferentes en continuo (DC): 12V para el reflector LED nocturno y 5V de alta tensión (3-5A) para la Raspberry Pi 5. En este sentido, queda desechado el uso de un inversor de tensión a 110V porque esta alternativa genera unas pérdidas de energía muy considerables. Como alternativa se opta por una topología exclusivamente en corriente continua.

Figura 10. Reflector LED de 12V DC para iluminación nocturna.



Fuente: Autoría

Figura 11. Módulo de relé para control del reflector desde la Raspberry Pi.



Fuente: Autoría

Para resolver este desafío, se ha optado por el uso de un sistema fotovoltaico con baterías recargables de ciclo profundo, que permiten mantener el sistema en funcionamiento sin interrupciones. La selección de la batería se realiza considerando los dispositivos que serán alimentados, como la Raspberry Pi 5, la cámara Arducam, el sensor PIR y el reflector LED nocturno, evaluando los requerimientos energéticos específicos de cada componente. En la Tabla 14, se detallará el consumo de cada dispositivo referenciado a un sistema de 12V, lo que permitirá realizar un cálculo preciso mediante un promedio ponderado y elegir la batería adecuada.

Tabla 17.

Consumo Eléctrico Nodo de Seguridad y Componentes

Elemento	Consumo en Funcionamiento (Inferencia + Luz)	Consumo en modo Sleep (Reposo)
Raspberry Pi 5 + Arducam (con pérdidas DC-DC)	850 mA	300 mA
Sensor PIR HC-SR501	1 mA	1 mA
Reflector LED 12V + Relé	1666 mA	0 mA
TOTAL	2517 mA	301 mA

Fuente: Autoría

De acuerdo con la tabla, el amperaje máximo para alimentar el nodo sensor cuando detecta un intruso es de 2517 mA, mientras que en estado de reposo es de 301 mA. Para el cálculo del consumo promedio de la batería se utiliza la ecuación (1).

Ecuación 1.

Formula consumo promedio de la batería.

$$Consumo_{Total} = \frac{((T_{cn} * I_{cn}) + (T_{cd} * I_{cd}))}{(T_{cn} + T_{cd})}$$

De acuerdo con la tabla, el amperaje máximo para alimentar el nodo sensor cuando detecta un intruso es de 2517 mA, mientras que en estado de reposo es de 301 mA. Para el cálculo del consumo promedio de la batería se utiliza la ecuación (1).

Donde:

- T_{cn} = Tiempo de consumo en funcionamiento
- I_{cn} = Intensidad de consumo de corriente en funcionamiento
- T_{cd} = Tiempo de consumo de corriente en modo sleep
- I_{cd} = Intensidad de consumo de corriente en modo sleep

Los parámetros seleccionados para este nodo se basan en un ciclo de una hora (3600 segundos), asumiendo de manera conservadora que el sistema podría detectar intrusos o falsas alarmas durante un total de 5 minutos por hora, permaneciendo en reposo el resto del tiempo. Los valores se detallan a continuación:

A continuación, se sustituyen estos valores en la ecuación (1) para calcular el consumo de corriente promedio requerido por el nodo en el entorno de aplicación.

Ecuación 2.

Cálculo con datos teóricos

$$Consumo_{Total} = \frac{(300 \times 2517) + (3300 \times 301)}{300 + 3300}$$

$$Consumo_{Total} = \frac{755100 + 993300}{3600} = 485.66_{mA}$$

Utilizando el cálculo de consumo energético promedio de 485.66 mA (0.485 A), podemos determinar el consumo total en un día (24 horas) y proceder a dimensionar la batería.

Donde:

- Consumo Diario = 11.64_{Ah}
- Días de Autonomía = 2
- PD (Profundidad de descarga) = 0.5-0.8

Se obtiene lo siguiente:

Ecuación 3.

Cálculo de dimensión Promedio Batería

$$Bateria = \frac{\text{Consumo Diario} * \text{Dias de autonomia}}{\text{Profundidad de descarga}} = \frac{11.64 * 2}{0.5} = 46.56 Ah$$

Para satisfacer este requerimiento y contar con un margen de seguridad para el sistema fotovoltaico en el entorno rural, se opta por una batería de Gel de ciclo profundo de 12V y 50Ah. Estas baterías son eficaces debido a su alta resistencia a descargas profundas y cambios de temperatura, lo que las hace ideales para sistemas que necesitan operar de forma continua en exteriores.

Figura 12.Batería de Gel Ciclo Profundo 12V 50Ah



Fuente: Autoría

Para la recarga del banco de baterías, se implementó un panel solar monocristalino acoplado a un controlador de carga PWM (Pulse Width Modulation). Este controlador gestiona

la energía proveniente del panel, protegiendo la batería contra sobrecargas y sobredescargas. Las mediciones en campo (Figura 13) demuestran que el controlador estabiliza la tensión de carga, asegurando la restitución de la energía consumida durante las fases operativas e inactivas del sistema.

Figura 13.Controlador de carga PWM registrando los parámetros de tensión.



Fuente: Autoría

Para garantizar la recarga diaria de esta batería, considerando un promedio de 4 horas de Sol Pico (HSP) en el sector de Mira, se selecciona un Panel Solar Monocristalino de 100W, el cual es capaz de inyectar la energía necesaria a través de un controlador de carga MPPT para restituir lo consumido durante los ciclos de funcionamiento y sleep.

Figura 14.Panel Solar 100 w



Fuente: Autoría

3.3.7. Selección de Software

La selección del ecosistema de software es un pilar fundamental para el éxito del "Sistema de Monitoreo con Arquitectura IoT y Procesamiento de Video". Al tratarse de una solución basada en Edge Computing (procesamiento en el borde), el software debe garantizar un equilibrio óptimo entre la precisión de los algoritmos de inteligencia artificial y la eficiencia en el consumo de recursos (CPU y RAM) de la Raspberry Pi 5. Una selección adecuada asegura la detección en tiempo real, la reducción de falsas alarmas y una comunicación fluida con el usuario final.

3.3.7.1. Selección del Lenguaje de Programación.

Para el desarrollo de la lógica de control, captura de video y ejecución de modelos de IA en el nodo sensor, es necesario establecer un lenguaje de programación robusto y compatible con librerías científicas.

Tabla 18.

Selección del Lenguaje de Programación

Lenguaje Evaluado	SySR5	SySR7	SySR10	SySR13	Valoración Total
Python	1	1	1	1	4
C++	1	1	1	0	3
Node.js	0	0	1	1	2

Fuente: Autoría

Tras evaluar las opciones en la Tabla 17, se seleccionó Python como el lenguaje principal del sistema. Aunque C++ ofrece una velocidad de ejecución ligeramente superior, Python destaca abrumadoramente en el cumplimiento del requerimiento SySR7, ya que posee integración nativa y soporte oficial para los frameworks de visión artificial requeridos (OpenCV, PyTorch, Ultralytics). Además, su ecosistema facilita la lectura directa de los pines

GPIO (SySR10) para la interrupción del sensor PIR y el manejo de excepciones para reiniciar procesos en caso de fallos (SySR13), agilizando el ciclo de desarrollo e implementación.

3.3.7.2. Selección del Algoritmo de Detección de Objetos

La tarea crítica del sistema es discriminar entre el movimiento ambiental y amenazas reales (personas y vehículos). Esto requiere un modelo de detección de objetos preentrenado que pueda operar eficazmente bajo recursos limitados.

Tabla 19.

Selección de Algoritmo de Detección

Algoritmo Evaluado	StSR2	SySR5	SySR7	SySR8	Valoración Total
YOLOv8n (Nano)	1	1	1	1	4
SSD MobileNet V2	0	1	1	1	3

Fuente: Autoría

La evaluación detallada en la Tabla 18 determina que YOLOv8n (You Only Look Once, versión 8 nano) es la arquitectura óptima. Los métodos tradicionales como Haar Cascades presentan demasiadas falsas alarmas y no distinguen vehículos. Si bien SSD MobileNet es ligero, YOLOv8n (desarrollado por Ultralytics) ofrece una precisión significativamente mayor (mAP) discriminando intrusos del entorno agrícola (StSR2), manteniendo una velocidad de inferencia superior a 5 fotogramas por segundo (FPS) en la CPU de la Raspberry Pi 5 (SySR7).

3.3.7.3. Selección de Software de Reconocimiento Óptico de Caracteres (OCR)

Para elevar el nivel de seguridad y proveer evidencia útil, el sistema debe ser capaz de extraer caracteres alfanuméricos de las placas de los vehículos detectados (ALPR - Automatic License Plate Recognition).

Tabla 20.

Selección de Software OCR

Software Evaluado	StSR7	SySR5	ArQR7	Valoración Total
-------------------	-------	-------	-------	------------------

EasyOCR	1	1	1	3
Tesseract OCR	0	1	0	1
OpenALPR (Comercial)	1	0	1	2

Fuente: Autoría

Se seleccionó EasyOCR debido a su alto rendimiento en la lectura de texto en entornos naturales . A diferencia de Tesseract OCR, que fue diseñado para documentos escaneados y falla frecuentemente con la distorsión, ángulos y condiciones de luz variables de las placas vehiculares (incumpliendo StSR7), EasyOCR se basa en redes neuronales profundas (PyTorch) y proporciona lecturas mucho más certeras. Al procesar únicamente las regiones de interés (ROI) previamente recortadas por YOLO, su impacto en la memoria del microprocesador (SySR5) se mantiene dentro de los márgenes permitidos.

3.3.7.4. Selección de Plataforma de Notificación y Alertas

El usuario final debe recibir notificaciones visuales e inmediatas cuando ocurre un evento, sin incurrir en altos costos de mantenimiento por el uso de servidores web o aplicaciones de terceros de pago.

Tabla 21.

Selección de Plataforma de Notificaciones

Plataforma Evaluada	StSR3	StSR7	StSR8	SySR2	Valoración Total
API de Telegram (Bot)	1	1	1	1	4
WhatsApp Business API	1	1	0	1	3
Servidor SMTP (Correo)	1	0	1	0	2

Fuente: Autoría

La API de Telegram (API de los Bots) ha logrado nada menos que la máxima puntuación. El correo electrónico no es un mecanismo eficiente para los avisos de emergencias

por su escasa inmediatez y porque, además, no permite establecer alertas adecuadas. En cambio, aunque WhatsApp es una aplicación con una buena presencia, su API empresarial tiene un coste por cada mensaje enviado y permite utilizar políticas muy estrictas para aumentar el coste de operación (StSR8).

En Telegram, se pueden crear bots de forma gratuita para producir bots conversacionales que habilitan un canal bidireccional encriptado (SySR2) donde el servidor remite alarmas por medio de texto acompañado de la grabación fotográfica empleando una buena calidad (StSR7). Además, se permite poner en el código Python una lógica de "cooldown" (tiempo de enfriado en un periodo de 15 segundos) para evitar el spam de notificaciones cuando la persona o vehículo está en la posición estacionaria delante de la cámara.

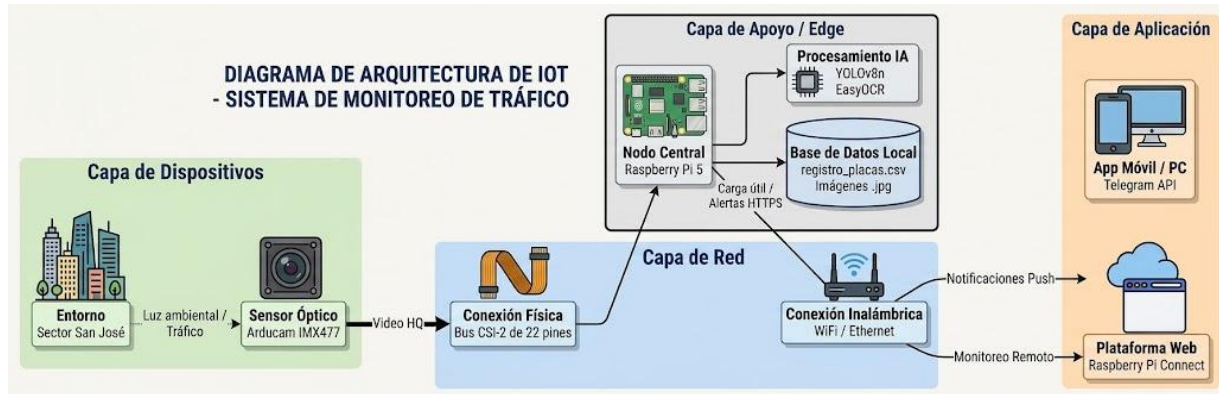
3.4. Etapa de Diseño

En esta etapa se tratan todos los aspectos relativos al diseño del sistema de control del sistema con arquitectura IoT y procesamiento de vídeo, que va desde la elaboración de la planificación de la arquitectura de alto nivel (Edge Computing), la integración del hardware de captura y la alimentación, el cómputo y programación del flujo de control con Python, etc., garantizando que el sistema fuese lo más autónomo, eficiente y seguro para las plantaciones de aguacate.

3.4.1. Diagrama de bloques del Sistema

En este apartado se presenta el diagrama de bloques del sistema desarrollado. En la Figura 15 se muestra la estructura general, donde se detallan los diferentes bloques que lo componen: Alimentación (Panel y Batería), Sensores (Cámara Arducam y PIR), Procesamiento (Raspberry Pi 5) y Actuadores/Comunicación (Relé, LED y Módulo de Red). Cada uno cumple una función esencial, integrando los componentes para asegurar la detección de intrusos.:

Figura 15:.*Diagrama de bloques*



Nota: capas de la arquitectura del proyecto

A. Capa de Dispositivos

- Es el punto de entrada del sistema, encargado de la adquisición de señales del entorno.
- Entorno: Sector San José, donde se requiere la vigilancia de tráfico y detección de personas.
- Sensor Óptico: Sensor Arducam IMX477 (High Quality Camera), conectado físicamente a la placa mediante un cable flex de 22 pines.
- Función: Captura de flujo de video de alta definición (Video HQ) para su posterior análisis.

B. Capa de Red

- Gestiona el transporte de datos entre el hardware de captura y las unidades de procesamiento y aplicación.
- Conexión Física: Interfaz CSI-2 de alta velocidad que enlaza el sensor con la Raspberry Pi 5.
- Conexión Inalámbrica: Uso de protocolos WiFi / Ethernet para el despacho de alertas vía HTTPS y el acceso remoto al sistema.

C. Capa de Apoyo / Edge

- Es el núcleo de inteligencia del sistema, donde reside el hardware de procesamiento y los modelos de IA.

- **Nodo Central:** Computador monoplaca Raspberry Pi 5, equipado con refrigeración activa para mantener el rendimiento durante la inferencia.
- **Procesamiento IA:** Implementación de modelos YOLOv8n para la detección de objetos y EasyOCR para el reconocimiento de matrículas vehiculares.
- **Almacenamiento Local:** Generación de una base de datos persistente compuesta por el archivo registro_placas.csv e imágenes de evidencia en formato .jpg.

D. Capa de Aplicación

- **Interfaz final** donde el usuario interactúa con los resultados del sistema.
- **App Móvil / PC:** Integración con la API de Telegram a través del bot @MonitorSanJoseBot para la recepción de notificaciones push en tiempo real.
- **Plataforma Web:** Uso de Raspberry Pi Connect para la gestión técnica, configuración y mantenimiento remoto del nodo desde cualquier navegador.

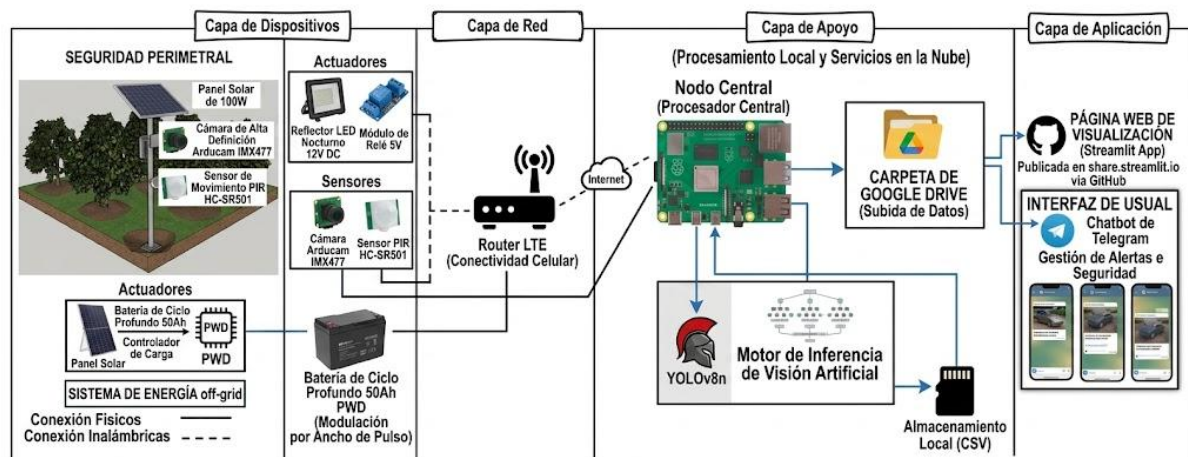
3.4.2. Diseño y Descripción del Sistema de Monitoreo

La arquitectura del sistema para el sector San José está diseñada bajo un enfoque de Edge Computing (Computación en el Borde). A diferencia de los sistemas IoT tradicionales que envían datos crudos a la nube, este diseño concentra el poder de procesamiento localmente.

Los datos del entorno son captados inicialmente por el sensor de movimiento PIR, el cual actúa como un interruptor de bajo consumo. Al detectar una firma térmica, envía una interrupción por hardware a la Raspberry Pi 5, la cual actúa como el cerebro central. La Raspberry enciende el reflector LED nocturno mediante un módulo de relé y activa la cámara Arducam IMX477 para capturar la escena. Los fotogramas son analizados localmente por los modelos de IA (YOLOv8n y EasyOCR).

Finalmente, la comunicación al exterior se realiza a través de la red de datos (Wi-Fi/4G), enviando únicamente las alertas confirmadas (fotografías y texto) mediante la API de Telegram, lo que ahorra un ancho de banda masivo y reduce los costos operativos:

Figura 16. Topología del Sistema de Monitoreo Edge IoT



Nota: Estructura de capas del sistema IoT para detección de intrusos. Fuente: Autoría

- A. **Capa de Dispositivos (Percepción y Actua-ción).** Esta capa constituye la interacción física del sistema con el medio agrícola. Está levantada por un sistema de energía off-grid completamente autónomo (Panel Solar de 150W y Batería de Ciclo Profundo 50Ah). A nivel de sensórica, incluye un sensor de movimiento PIR HC-SR501 actuando como el 'gatillo' del sistema para tener un bajo consumo, y una cámara de alta definición Arducam IMX477 que se encarga de captar la escena. Como actuador, incluye un reflector LED 12V DC con control a través de un módulo de relé, y que proporciona la iluminación para asegurar la operatividad de la cámara por la noche.
- B. **Red de Capa (Conectividad).** Determina los dispositivos de comunicación físicos, así como los inalámbricos. En el ámbito local (conexiones físicas recogidas mediante líneas continuas), los sensores y actuadores se comunican de forma directa y dando uso a una conexión cableada con el microcontrolador usando interfaces de alta velocidad (por ejemplo MIPI CSI-2 para video) o con pines GPIO (interrupciones digitales). En lo que respecta al exterior (señales con líneas entrecortadas), el sistema usa un enrutador local al que tiene acceso a Internet usando redes para móviles (4G LTE), logrando con

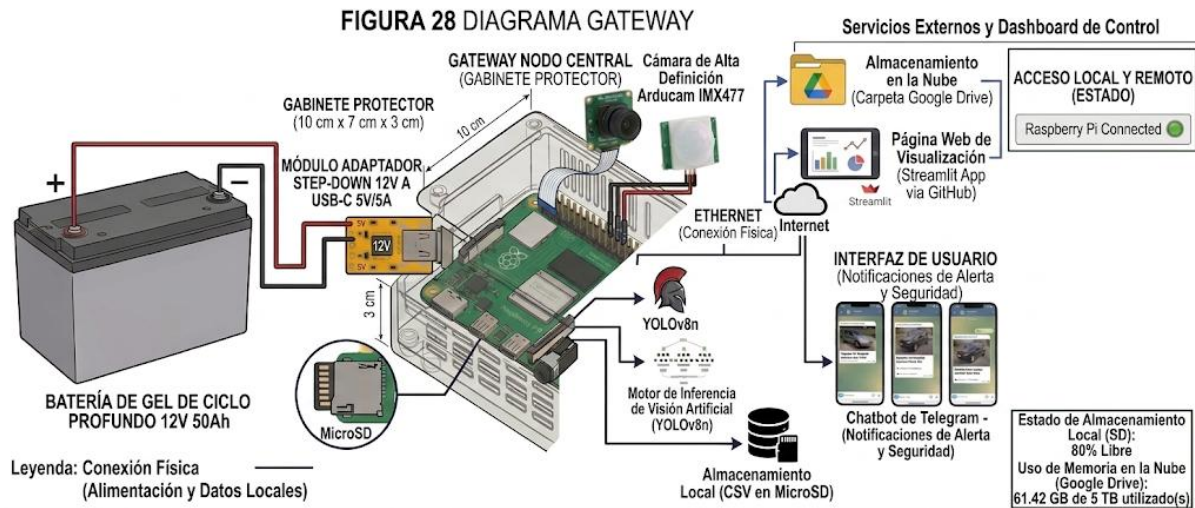
ello una transmisión de baja latencia, condición indispensable para que el sistema pueda realizar el envío inmediato de las alertas de seguridad provenientes de la zona rural.

- C. **Capa de Soporte (Borde de Procesamiento).** El núcleo de esta capa es la Raspberry Pi 5, la cual funge como el Nodo Central. Aquí es donde se materializa el concepto de Edge AI. Los fotogramas capturados no viajan a internet; en su lugar, son analizados localmente por el motor de inferencia de visión artificial compuesto por la arquitectura YOLOv8n (para detectar personas y vehículos) y la herramienta EasyOCR (para la extracción alfanumérica de matrículas). Esta capa también gestiona el almacenamiento local temporal en formatos CSV, respaldando datos en el drive en caso de necesitar un respaldo de datos en la red.
- D. **Capa de Aplicación (Interfaz de Usuario)** Es el punto de contacto final con el propietario o administrador. Para dotar al sistema de interfaces accesibles sin requerir servidores locales costosos, esta capa se resolvió mediante una estrategia dual que incluye, por un lado, un Chatbot bidireccional en Telegram para la recepción de alertas "Push" en tiempo real con la prueba fotográfica y el nivel de confianza de la red neuronal, y por otro, un panel de control web interactivo alojado en la nube (dashboard en Streamlit) que permite la navegación histórica y estructurada por fechas de las evidencias del repositorio, todo ello protegido mediante autenticación de credenciales.

3.4.3. Diagrama eléctrico y esquemático de conexiones

En este apartado se presenta el diseño detallado de la infraestructura eléctrica y las interconexiones físicas del nodo central Edge AI. Este diagrama es fundamental para garantizar la integridad de las señales digitales de los sensores y el suministro de potencia estable hacia el microprocesador y los actuadores, asegurando la operatividad continua del sistema de seguridad en el sector rural San José. En la Figura (17) se detalla la esquemática completa del sistema.

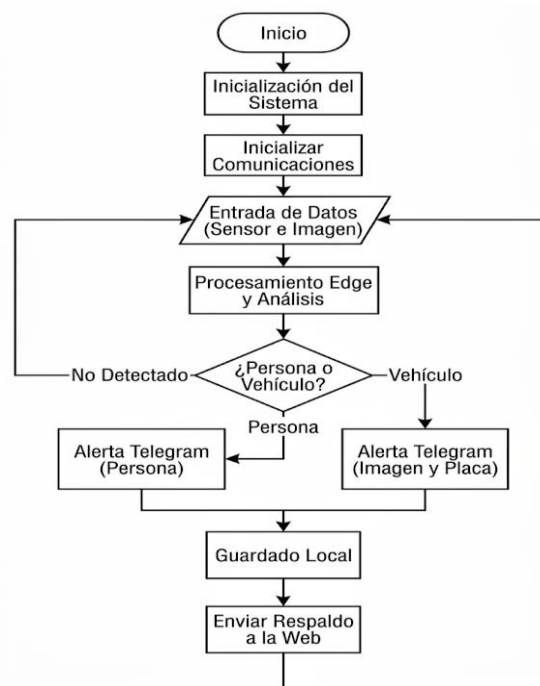
Figura 17. Diagrama de Conexiones Eléctricas y Esquemático del Nodo Central



Nota: La figura muestra las interconexiones entre la Raspberry Pi 5, la Arducam IMX477, el sensor PIR HC-SR501, la Batería de Gel 12V 50Ah, el Relé y el Reflector LED. Fuente: Autoría.

3.4.3.1 Diagrama de flujo del Gateway.

Tal y como se ilustra en la Figura 17, el flujo de ejecución en el software del Gateway (Raspberry Pi 5) comienza con la inclusión de las librerías de visión artificial y mensajería (OpenCV, Ultralytics y Telebot), la creación de variables (tokens de la API) y la configuración de los pines GPIO, características cruciales en la interacción física que se establece con los sensores y los actuadores. A continuación, se realiza la inicialización de la conexión de red (Wi-Fi/4G) y de la autorización de la API del Telegram; si no se consigue establecer una conexión estable, entra en un bucle de stock, y se notifican las pérdidas de la conexión. Si se establece, entra en estado de poca energía y espera a recibir datos sobre el entorno mediante interrupciones de hardware que son generadas a partir del sensor de movimiento PIR.

Figura 18.Diagrama de Flujo Funcionamiento

Fuente: Autoría propia

Ahora, la Figura 18 describe el proceso de inferencia y actuación ejecutado por la Raspberry Pi 5, la cual actúa como nodo de Edge Computing principal dentro del sistema de seguridad. En este caso, toda la lógica operativa se desarrolla directamente mediante un script en Python, garantizando un procesamiento eficiente sin depender de plataformas intermedias. Al recibir la señal de activación del sensor PIR, el sistema activa el reflector LED mediante el módulo de relé y captura la escena.

Inmediatamente, estos datos visuales son procesados de forma local utilizando los modelos de inteligencia artificial YOLOv8n (para la detección de personas o vehículos) y EasyOCR (para el reconocimiento de placas vehiculares). Si la detección resulta positiva, la información se deriva para su almacenamiento en una base de datos local (archivo CSV) a modo de respaldo o bitácora. Paralelamente, el sistema activa una alerta remota y automatizada, enviando la evidencia fotográfica y los detalles del evento al usuario mediante el bot de

Telegram, garantizando así una respuesta oportuna y eficiente ante cualquier intrusión en la plantación.

3.4.3.2 Subsistema de Potencia de Entrada y Regulación.

La alimentación principal proviene de la batería de Gel de ciclo profundo de 12V y 100Ah. Debido a que la Raspberry Pi 5 opera estrictamente a 5V DC con una corriente de hasta 5A (25W) a través de su puerto USB-C, se implementó un módulo adaptador DC-DC .

Como se observa en la esquemática, el terminal positivo (+) de la batería se conecta a la entrada (VIN+) del módulo reductor, y el terminal negativo (-) a la entrada común (GND). La salida del módulo (VOOUT+ de 5V) y (GND) se derivan mediante un cable USB-C modificado hacia el puerto de alimentación de la Raspberry Pi 5. Esta conexión garantiza la estabilidad y potencia necesarias para ejecutar los modelos de inteligencia artificial YOLOv8n y EasyOCR sin interrupciones.

3.4.3.3 Subsistema de Captura de Imagen de Alta Velocidad.

Para garantizar la transmisión fluida y de alta definición de los fotogramas necesarios para el algoritmo ALPR (reconocimiento de placas), la cámara Arducam IMX477 de 12MP no utiliza interfaces USB. Se conecta mediante un cable plano flexible (FPC - Flat Flexible Cable) directamente al puerto MIPI CSI-2 nativo de dos líneas de la Raspberry Pi 5. Esta interfaz es crucial para manejar el ancho de banda requerido para la inferencia local sin latencia perceptible.

3.4.3.4 Subsistema de Potencia Controlada.

La conexión de potencia para la iluminación nocturna se realiza mediante un circuito de conmutación. El terminal positivo (+) de la batería de 12V 50Ah se conecta al terminal común (COM) del contacto de salida del relé. El terminal normalmente abierto (NO) del relé se dirige hacia el polo positivo (+) del reflector LED de 12V y 20W. Finalmente, el polo

negativo (-) del reflector LED se cierra directamente al terminal negativo (-) de la batería de 12V. De esta manera, cuando la Raspberry activa el relé, este cierra el circuito de 12V y enciende el foco para permitir la visión de la cámara en la oscuridad.

3.5. Desarrollo de Software y Algoritmos de Procesamiento

El desarrollo lógico del sistema se centraliza en el nodo de Edge Computing, utilizando Python como lenguaje principal de programación para orquestar el hardware, ejecutar la inteligencia artificial y gestionar las telecomunicaciones. La arquitectura del código se divide en módulos funcionales que operan de manera concurrente para garantizar la seguridad perimetral ininterrumpida.

3.5.1. Inicialización y Control

El desarrollo del software en el nodo Edge comienza con la importación y configuración de las dependencias necesarias para la operación del sistema. Como se detalla en la Figura 19, se incluyen módulos fundamentales para el procesamiento de matrices de imágenes y temporización (cv2, time), gestión de concurrencia y subprocessos del sistema operativo (threading, subprocess), así como para las peticiones de red (requests). De igual manera, se incorporan los frameworks de Inteligencia Artificial (ultralytics, easyocr) y las bibliotecas nativas para el control de interfaces físicas en la placa (picamera2, gpiozero).

Figura 19. Importación de Librerías y Dependencias del Sistema

```
import cv2 #procesamiento de imagen
import os # crear archivos
import time # maneja reloj para tareas
import requests # peticiones tipo post get
import easyocr # motor de reconociminto
import threading # permite multitarea
import subprocess #
from datetime import datetime # fecha
from ultralytics import YOLO #ia
from picamera2 import Picamera2
from gpiozero import MotionSensor, OutputDevice
```

Fuente: Autoría propia

Posteriormente, el código establece los parámetros globales y las variables de configuración operativa, tal como se muestra en la Figura 20. En este segmento se definen las credenciales y tokens de acceso para la API de Telegram, la ruta de los directorios de respaldo local (BASE_DIR), y las constantes operacionales para el hardware, tales como la rotación de la imagen, la cantidad de fotografías por ráfaga y el tiempo mínimo de iluminación del reflector LED.

Figura 20 . Configuración de Variables Globales y Credenciales

```
TOKEN = "8626887025:AAFdYomeBzMKQyYkmnFVxpKtaXqVtfajP0g"
CHAT_ID = "-5575877445 " # grupo-5575877445 individual 5682386608
BASE_DIR = "detecciones"
TIEMPO_SYNC_MINUTOS = 10

# --- CONFIGURACIÓN DE CÁMARA Y RÁFAGA ---
ROTACION = cv2.ROTATE_180
NUM_FOTOS_RAFAGA = 3
TIEMPO_MINIMO_LUZ = 5
```


Nota. Definición de los tokens de acceso para la API de Telegram y parámetros operacionales de temporización y ráfaga fotográfica. Fuente: Autoría propia.

Una vez definidas las variables, inicia la fase de inicialización de los modelos de visión computacional y el control físico. Como se evidencia en la Figura 21, la interacción con el entorno se establece asignando el sensor de movimiento PIR al pin GPIO 17 y el módulo de relé al pin GPIO 27. Este último se declara explícitamente en estado bajo (`active_high=False`) para asegurar un inicio seguro y sin consumo energético. Simultáneamente, se cargan los modelos de IA (YOLOv8n y EasyOCR) en memoria y se configura la cámara con una resolución de 640x480 píxeles en formato de color RGB888, optimizando el rendimiento del procesador al evitar conversiones de color posteriores.

Figura 21. Inicialización de Hardware y Modelos de Inteligencia Artificial

```
print("Iniciando Hardware...")
# [SENSOR PIR]: Se declara el pin 17 para el sensor que despierta el sistema.
pir = MotionSensor(17)
# [RELÉ / FOCO]: Se declara el pin 27 para el reflector LED. Inicia apagado (False).
foco = OutputDevice(27, active_high=False, initial_value=False)

# =====
# INICIALIZACIÓN DE MODELOS E IA
# =====
print("Cargando Inteligencia Artificial (YOLOv8 y EasyOCR)...")
model = YOLO('yolov8n.pt')
reader = easyocr.Reader(['es'])

# [INICIALIZACIÓN DE CÁMARA]: Se levanta la cámara usando el bus MIPI CSI-2.
# El formato RGB888 evita que la CPU pierda tiempo convirtiendo colores para la IA.
print("Iniciando Cámara...")
picam2 = Picamera2()
config = picam2.create_preview_configuration(main={"format": "RGB888", "size": (640, 480)})
picam2.configure(config)
picam2.start()
```

Nota. Asignación de pines para el sensor PIR y el módulo relé, y configuración directa de la cámara a formato RGB888. Fuente: Autoría propia.

Previo a la ejecución continua, se estructuran funciones auxiliares que garantizan la modularidad del código. Como se aprecia en la Figura 22, se definen rutinas para la creación dinámica de directorios locales (obtener_directorios), la inyección de notificaciones y fotografías hacia la API de Telegram (enviar_telegram), la validación condicional del horario nocturno (es_denoche) y la transferencia cifrada de respaldos a Google Drive ejecutada mediante comandos del sistema hacia la herramienta rclone (sincronizar_drive).

Figura 22. Funciones Auxiliares de Almacenamiento, Notificación y Sincronización

```
# =====
# FUNCIONES AUXILIARES
# =====

def obtener_directorios():
    ahora = datetime.now()
    mes = ahora.strftime("%Y-%m")
    dia = ahora.strftime("%Y-%m-%d")

    dir_personas = os.path.join(BASE_DIR, mes, dia, "personas")
    dir_vehiculos = os.path.join(BASE_DIR, mes, dia, "vehiculos")

    os.makedirs(dir_personas, exist_ok=True)
    os.makedirs(dir_vehiculos, exist_ok=True)

    return dia, dir_personas, dir_vehiculos

def enviar_telegram(path, msg):
    # [ENVÍO A TELEGRAM]: Petición HTTP para enviar la evidencia (foto) y el texto de alerta.
    url = f"https://api.telegram.org/bot{TOKEN}/sendPhoto"
    try:
        with open(path, 'rb') as f:
            requests.post(url, data={'chat_id': CHAT_ID, 'caption': msg}, files={'photo': f}, timeout=10)
            print("📷 Notificación de Telegram enviada.")
    except Exception as e:
        print(f"⚠️ Error Telegram: {e}")

def es_denoche():
    hora_actual = datetime.now().hour
    return hora_actual >= 18 or hora_actual < 6

def sincronizar_drive():
    # [CONEXIÓN AL DRIVE]: Esta función corre en segundo plano continuamente.
    while True:
        time.sleep(TIEMPO_SYNC_MINUTOS * 60)
        print(f"\n[🔄 Sincronización] Subiendo archivos a Google Drive...")
        try:
            # Invoca a rclone (sistema operativo Linux) para empujar los archivos a la nube.
            subprocess.run(["rclone", "copy", BASE_DIR, "gdrive_tesis:detecciones"], check=True)
            print("[✅ Sincronización] Respaldo en la nube completado con éxito.")
        except Exception as e:
            print(f"[❌ Sincronización] Error al subir a Drive: {e}")
```

Nota. Declaración de los métodos de soporte para la gestión de archivos, comunicación externa y evaluación de estado ambiental. Fuente: Autoría propia.

La lógica central de vigilancia y captura se concentra en el bucle principal del sistema (ver Figura 23). El flujo inicia delegando la función de respaldo a un hilo en segundo plano (threading.Thread) para evitar bloqueos. A continuación, el procesador entra en estado de suspensión hasta que el sensor PIR dispara una interrupción por detección de calor (pir.wait_for_motion()). Tras este estímulo, el sistema evalúa la luminosidad, activa el reflector LED si es necesario, y captura una ráfaga de fotogramas. Cada imagen es rotada espacialmente y convertida al espacio de color RGB antes de ser sometida a la inferencia de la red neuronal YOLOv8, aplicando un umbral de confianza del 40% para descartar falsos positivos iniciales.

Figura 23. Bucle Principal de Monitoreo, Activación de Hardware e Inferencia

```
# BUCLE PRINCIPAL DEL SISTEMA
# =====
# [HILOS / THREADING]: Lanza la sincronización a Drive en paralelo para no congelar la vigilancia.
hilo_drive = threading.Thread(target=sincronizar_drive, daemon=True)
hilo_drive.start()

print("\n--- Sistema San José Activo y Monitoreando ---")

try:
    while True:
        # [DESPERTAR]: El sistema duerme consumiendo lo mínimo hasta que detecta calor/movimiento.
        pir.wait_for_motion()
        print("\n 🚨 ¡Movimiento detectado! Iniciando ráfaga...")
        necesita_luz = es_denoche()
        tiempo_inicio_rafaga = time.time()
        # [ENCIENDE EL RELÉ]: Si es de noche, manda la señal al pin 27 para prender el foco de 12V.
        if necesita_luz:
            print(" 🌙 Es de noche. Encendiendo reflector...")
            foco.on()
            time.sleep(1)
            dia_actual, ruta_personas, ruta_vehiculos = obtener_directorios()
            placas_vistas_ahorita = set()
            persona_vista_ahorita = False
            detectado = False
            for i in range(NUM_FOTOS_RAFA):
                print(f" 📷 Capturando foto {i+1} de {NUM_FOTOS_RAFA}...")
                img_bgr_cruda = picam2.capture_array()

                # 1. Rotamos la imagen cruda
                img_bgr = cv2.rotate(img_bgr_cruda, ROTACION)
                # 2. Convertimos a RGB puro
                img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
                # [INFERENCIA IA]: Se evalúa la imagen en YOLO con un 40% de confianza mínima.
                results = model(img_rgb, conf=0.4, verbose=False)
                for r in results:
                    boxes = r.boxes
```

Nota. Secuencia operativa de despertar del sistema, control de iluminación y procesamiento primario de las matrices de imagen a través del modelo YOLOv8. Fuente: Autoría propia.

Si el modelo clasifica un objeto dentro de las categorías de vehículos (auto, camión, motocicleta o bus), el flujo ingresa a un bloque de procesamiento avanzado de visión computacional. De acuerdo con la Figura 24, el algoritmo aísla el vehículo de mayor tamaño y aplica un recorte geométrico inteligente sobre el 70% inferior de la caja delimitadora. Para maximizar la precisión de la lectura óptica, esta sección (la placa) es sometida a un escalado digital cúbico (2.5x), conversión a escala de grises, ecualización de histograma (CLAHE) y filtrado bilateral. Finalmente, el motor EasyOCR extrae los caracteres alfanuméricos, descartando lecturas inválidas antes de registrar la placa en la bitácora local.

Figura 24. Procesamiento Digital de Vehículos y Extracción de Caracteres (OCR)

```
# --- PROCESAMIENTO DE VEHÍCULOS (Con mejoras OCR) ---
# [DECISIÓN 1]: YOLO filtra y decide si en la foto hay algún tipo de vehículo.
vehiculos = [b for b in boxes if model.names[int(b.cls[0])] in ['car', 'truck', 'motorcycle', 'bus']]
if vehiculos:
    detectado = True
    vehiculo_principal = max(vehiculos, key=lambda b: (b.xyxy[0][2] - b.xyxy[0][0]) * (b.xyxy[0][3] - b.xyxy[0][1]))
    x1, y1, x2, y2 = map(int, vehiculo_principal.xyxy[0])
    label = model.names[int(vehiculo_principal.cls[0])]
    ts = datetime.now().strftime("%H%M%S_%f")[:10]
    fecha_hora = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    # 1. Recorte Inteligente (70% superior y sin 20% laterales)
    ancho_carro = x2 - x1
    margen_lateral = int(ancho_carro * 0.20)
    x1_crop = x1 + margen_lateral
    x2_crop = x2 - margen_lateral
    y_inicio = y1 + int((y2 - y1) * 0.70)
    placa_crop = img_rgb[y_inicio:y2, x1_crop:x2_crop]
    # 2. Zoom digital Cúbico (2.5x)
    if placa_crop.size != 0:
        placa_crop = cv2.resize(placa_crop, None, fx=2.5, fy=2.5, interpolation=cv2.INTER_CUBIC)
        # 3. Escala de grises, corrección de luz (CLAHE) y Alto Contraste
        placa_gray = cv2.cvtColor(placa_crop, cv2.COLOR_RGB2GRAY)
        clahe = cv2.createCLAHE(clipLimit=4.0, tileGridSize=(8,8))
        placa_gray = clahe.apply(placa_gray)
        placa_gray = cv2.convertScaleAbs(placa_gray, alpha=2.5, beta=-15)
        placa_gray = cv2.bilateralFilter(placa_gray, 11, 17, 17)
        # 4. Lectura OCR
        ocr_res = reader.readtext(placa_gray, allowlist='ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789-')
        textos_validos = []
        for res in ocr_res:
            texto = res[1].upper().replace(" ", "")
            if "ECUADOR" not in texto and len(texto) >= 6:
                textos_validos.append(texto)
        txt_placa = "-".join(textos_validos) if textos_validos else "INDETERMINADA"
        if txt_placa not in placas_vistas_ahorita:
            f_name = f"Vehiculo_{txt_placa}_{ts}.jpg"
            f_path = os.path.join(ruta_vehiculos, f_name)
            # Dibujar recuadro verde en la foto principal para Telegram
            img_final = img_bgr.copy()
            cv2.rectangle(img_final, (x1, y1), (x2, y2), (0, 255, 0), 3)
            # [ALMACENAMIENTO LOCAL 1]: Guarda la fotografía en la memoria SD de la placa.
            cv2.imwrite(f_path, img_final)
            txt_log_path = os.path.join(ruta_vehiculos, f"Registro_Placas_{dia_actual}.txt")
            # [ALMACENAMIENTO LOCAL 2]: Escribe la placa extraída en la bitácora de texto (CSV).
            with open(txt_log_path, "a") as txt_file:
                txt_file.write(f"[{fecha_hora}] Tipo: {label} | Placa: {txt_placa}\n")
```

Nota. Algoritmo de recorte inteligente, mejora de contraste dinámico y lectura de placas vehiculares mediante redes neuronales secundarias. Fuente: Autoría propia.

Para mantener la seguridad perimetral contra intrusos a pie, el código implementa una segunda validación lógica (ver Figura 25). Si YOLOv8 no identifica vehículos, el escaneo se redirige hacia la detección de personas (person). De ser positivo, se guarda la evidencia y se alerta al usuario. Si ninguna de las dos validaciones resulta afirmativa, el sistema cataloga el evento como una falsa alarma ambiental, evitando la saturación del ancho de banda y apagando el reflector tras una pausa de seguridad de 5 segundos. Este bloque incluye también el manejo de interrupciones de teclado para garantizar un cierre de procesos limpio (KeyboardInterrupt).

Figura 25. Filtrado de Personas, Manejo de Falsas Alarmas y Excepciones

```
# --- PROCESAMIENTO DE PERSONAS ---
# [DECISIÓN 2]: Si YOLO no vio ningún vehículo, entonces busca personas.
if not detectado:
    personas = [b for b in boxes if model.names[int(b.cls[0])] == 'person']
    if personas:
        detectado = True
        if not persona_vista_ahorita:
            ts = datetime.now().strftime("%H%M%S")
            fecha_hora = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
            f_name = f"Persona_{ts}.jpg"
            f_path = os.path.join(ruta_personas, f_name)

            # [ALMACENAMIENTO LOCAL]: Guarda foto de la persona en la SD.
            cv2.imwrite(f_path, img_bgr)
            enviar_telegram(f_path, f"👤 ALERTA: Persona detectada en San José\n🕒 {fecha_hora}")
            persona_vista_ahorita = True
            print("👤 Persona guardada y enviada.")

    tiempo_transcurrido = time.time() - tiempo_inicio_rafaga

    if necesita_luz:
        tiempo_restante = TIEMPO_MINIMO_LUZ - tiempo_transcurrido
        if tiempo_restante > 0:
            time.sleep(tiempo_restante)
        # [APAGA EL RELÉ]: Corta la energía del foco LED para ahorrar batería tras la captura.
        foco.off()
        print("💡 Reflector apagado tras iluminar la zona.")

    if not detectado:
        print("🚫 Falsa alarma o sujeto no clasificado por la IA en ninguna foto.")

    print("⌚ Pausa de seguridad (5 seg)...")
    time.sleep(5)

except KeyboardInterrupt:
    print("\n🛑 Finalizando sistema de forma segura...")
```

Nota. Lógica de descarte de falsos positivos térmicos, detección de intrusos humanos y rutinas de finalización segura de scripts. Fuente: Autoría propia.

Para salvaguardar la integridad de los actuadores y evitar errores críticos por interrupción de procesos, la ejecución principal está protegida por una estructura de manejo de excepciones. Tal y como se observa en la Figura 26, el bloque de seguridad (finally) garantiza que, frente a una caída del sistema o finalización forzada, se despache una alerta de detención al usuario por Telegram, se corte el suministro eléctrico del relé (foco.off()) y se liberen de manera segura los recursos de hardware de la cámara.

Figura 26. Bloque de Seguridad y Cierre Controlado

```
finally:
    # [BLOQUE DE SEGURIDAD]: Si el programa se cae por un error,
    # garantiza apagar el foco y liberar la cámara además de mandar una alerta a telegram.
    enviar_telegram(f_path, f"🚨 ALERTA: Se detuvo el sistema")
    foco.off()
    picam2.stop()
```

Nota. Estructura de manejo de excepciones implementada para garantizar el apagado del reflector LED y liberación de la cámara ante paradas no programadas. Fuente: Autoría propia.

Finalmente, el éxito de todo este proceso secuencial desde la inicialización de las librerías hasta la ejecución del bucle de inferencia se verifica a través de los registros del sistema operativo. La Figura 27 ilustra la consola de la Raspberry Pi durante el arranque del servicio de Python, donde se evidencia el correcto despliegue de las dependencias, el reconocimiento del bus MIPI CSI-2 por parte de la librería libcamera, y los mensajes de estado que confirman que el nodo Edge se encuentra activo, analizando el entorno y discriminando falsas alarmas de manera autónoma.

Figura 27.Registro de Arranque y Operación del Sistema en Consola

```

alder@raspberrypi: ~
Archivo  Editar  Pestañas  Ayuda
alder@raspberrypi:~ $ source entorno_tesis/bin/activate
(entorno_tesis) alder@raspberrypi:~ $ python monitoreo_yolo5.py
Iniciando Hardware...
Cargando Inteligencia Artificial (YOLOv8 y EasyOCR)...
Neither CUDA nor MPS are available - defaulting to CPU. Note: This module is much faster with a GPU.
Iniciando Cámara...
[2:08:27.532863703] [75512] INFO Camera camera_manager.cpp:330 libcamera v0.5.2+99-bfd68f78
[2:08:27.539919064] [75532] INFO RPI pisp.cpp:720 libpisp version v1.2.1 981977ff21f3 29-04-2025 (14:13:50)
[2:08:27.542712718] [75532] INFO IPAProxy ipa_proxy.cpp:180 Using tuning file /usr/share/libcamera/ipa/rpi/pisp/imx477.json
[2:08:27.549544722] [75532] INFO Camera camera_manager.cpp:220 Adding camera '/base/axi/pcie@1000120000/rp1/i2c@800000/imx477@1a' for pipeline handler rpi/pisp
[2:08:27.549570500] [75532] INFO RPI pisp.cpp:1179 Registered camera /base/axi/pcie@1000120000/rp1/i2c@800000/imx477@1a to CFE device /dev/media0 and ISP device /dev/media2 using PiSP variant BCM2712_D0
[2:08:27.552659254] [75512] INFO Camera camera.cpp:1215 configuring streams: (0) 640x480-RGB888/sRGB (1) 1332x990-BGGR_PISP_COMP1/RAW
[2:08:27.552765812] [75532] INFO RPI pisp.cpp:1483 Sensor: /base/axi/pcie@1000120000/rp1/i2c@800000/imx477@1a - Selected sensor format: 1332x990-SBGGR12_1X12/RAW - Selected CFE format: 1332x990-PC1B/RAW
--- Sistema San José Activo y Monitoreando ---
[0:01] ¡Movimiento detectado!
[0:16] Es de noche. Encendiendo reflector...
[0:21] Reflector apagado.
[0:26] Analizando imagen...
[0:27] Falsa alarma o sujeto no clasificado por la IA.
[0:32] Pausa de seguridad (5 seg)...

```

Nota. Salida de la terminal del sistema operativo evidenciando el acoplamiento del hardware de visión y el rastreo de eventos en tiempo real. Fuente: Autoría propia.

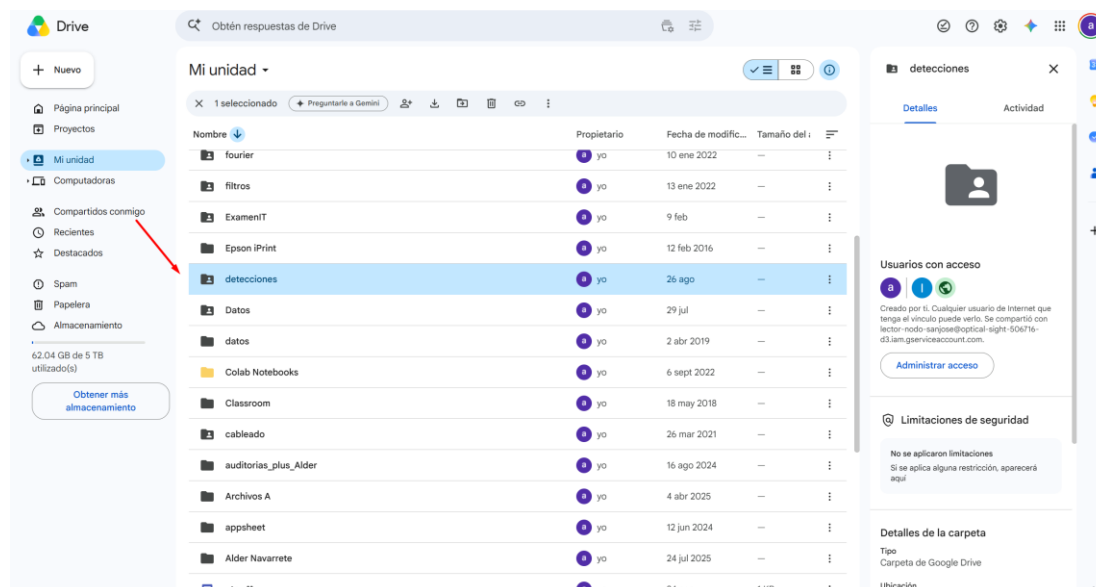
3.5.2. Integración en la Nube y Desarrollo de la Interfaz Web

Para que el usuario final pueda consumir las evidencias fotográficas generadas por el nodo *Edge* sin depender exclusivamente de las notificaciones de Telegram, se diseñó una arquitectura de consulta remota. El primer componente de esta arquitectura es el almacenamiento en la nube. Como se observa en la Figura 28, se estableció una carpeta central

denominada "detecciones" en Google Drive, la cual recibe los respaldos sincronizados periódicamente por la Raspberry Pi.

Figura 28.

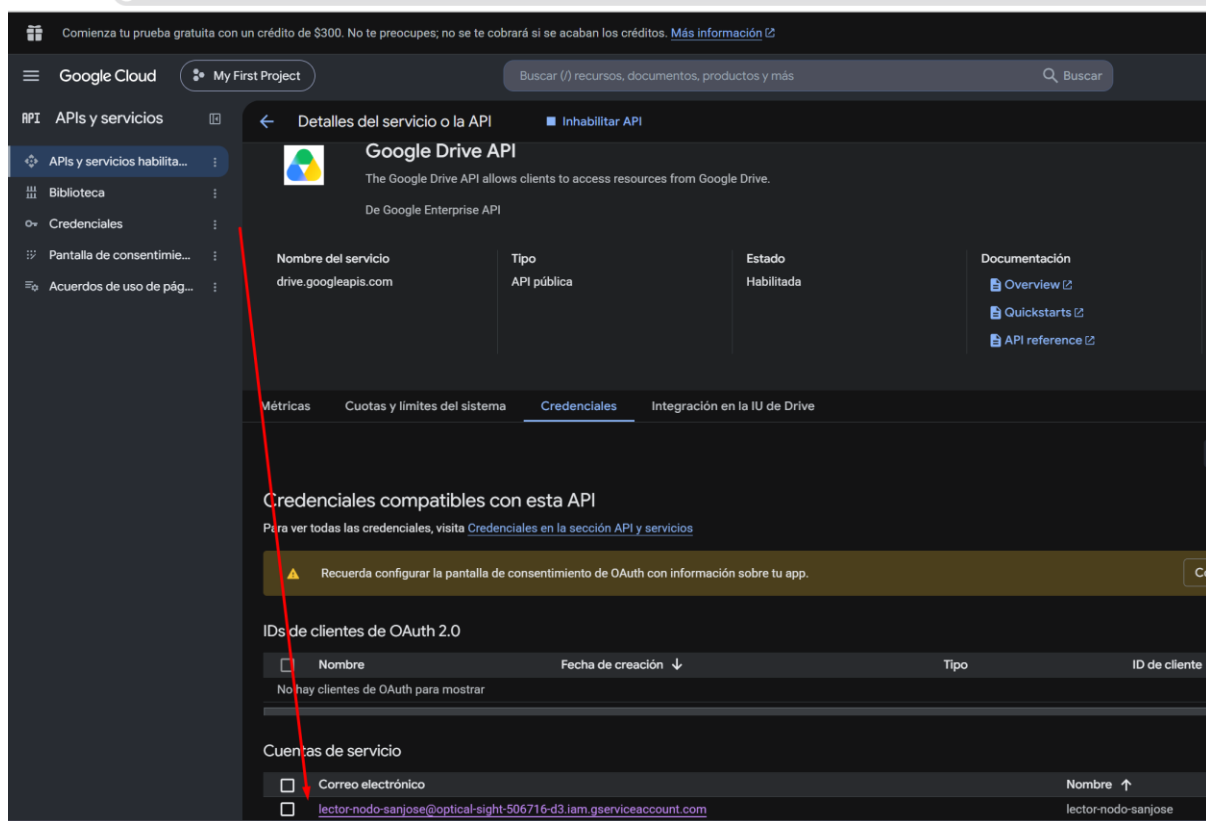
Directorio Principal de Respaldo en Google Drive



Nota. Interfaz de almacenamiento en la nube mostrando la carpeta compartida que actúa como base de datos centralizada de las evidencias. Fuente: Autoría propia.

Para que la aplicación web pueda acceder a estos archivos de forma segura sin exponer credenciales personales, se configuró una Cuenta de Servicio (*Service Account*) en Google Cloud Platform (ver Figura 29). Al habilitar la API de Google Drive, se generó un correo electrónico virtual con permisos exclusivos de solo lectura (*reader*), garantizando que la aplicación web pueda listar y descargar las imágenes, pero no alterarlas ni eliminarlas.

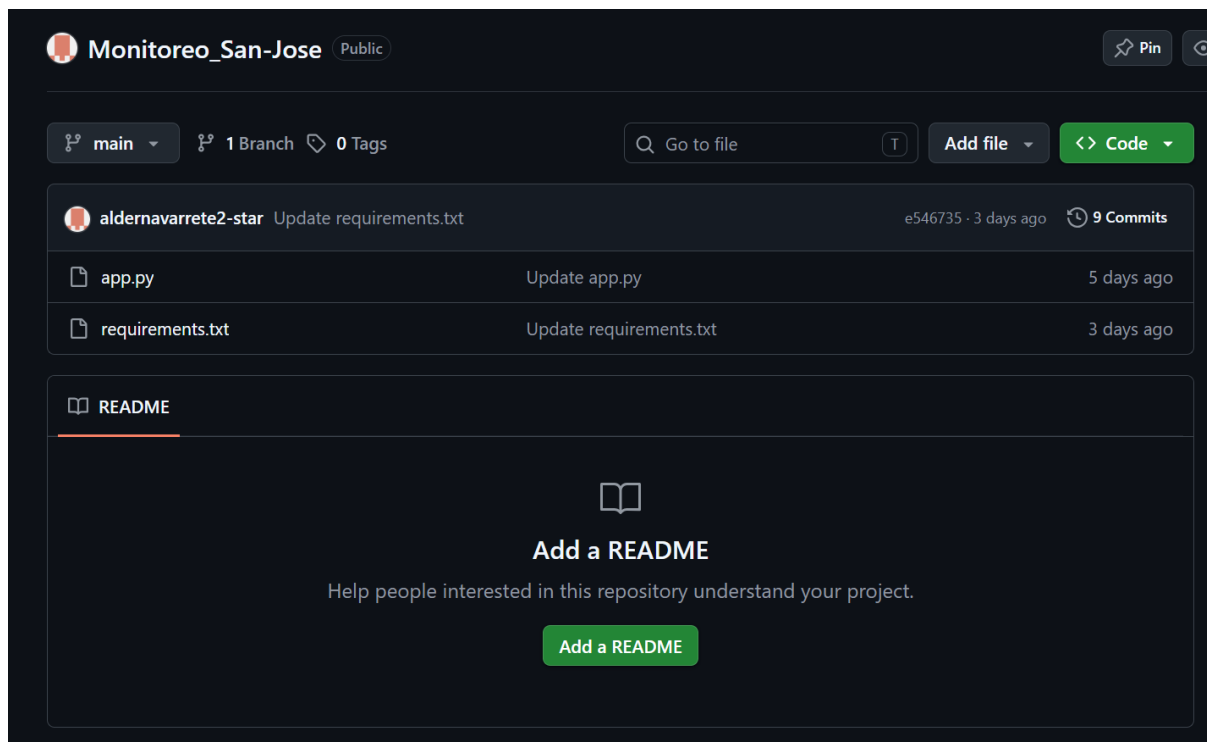
Figura 29. Configuración de la Cuenta de Servicio en Google Cloud



Nota. Panel de administración de credenciales de Google Cloud Platform evidenciando la habilitación de la API de Drive y el agente de servicio creado. Fuente: Autoría propia.

El código fuente de la interfaz web fue desarrollado en Python utilizando el framework Streamlit. Para garantizar un control de versiones adecuado y facilitar su despliegue continuo, los archivos del proyecto se alojaron en un repositorio público en GitHub, tal y como se ilustra en la Figura 30.

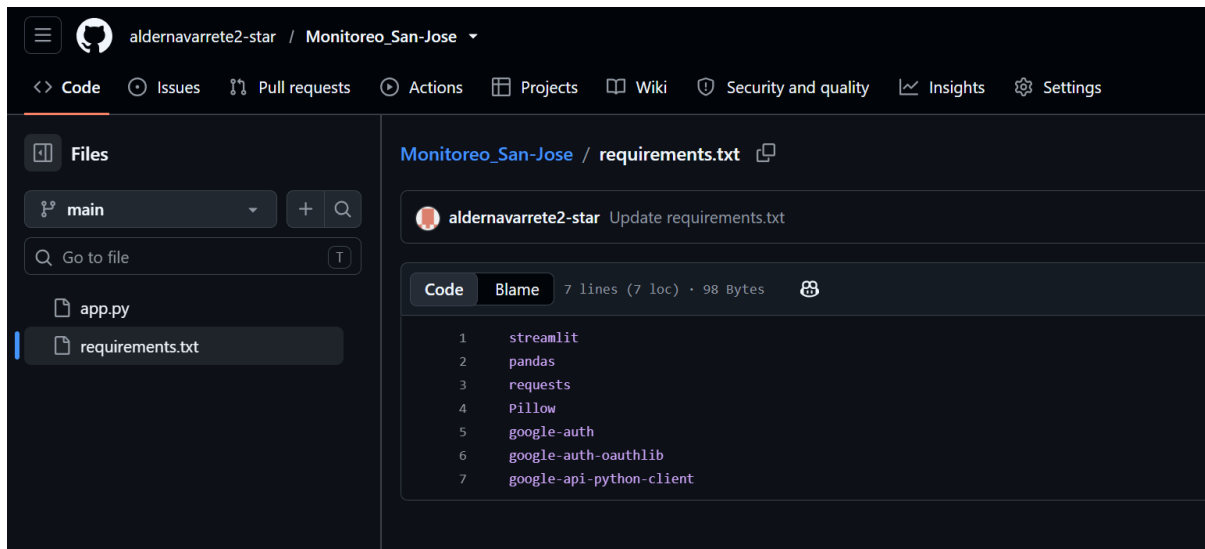
Figura 30.Repositorio del Proyecto Web en GitHub



Nota. Alojamiento en la nube del código fuente (app.py) y del archivo de dependencias necesarios para el despliegue del panel de control. Fuente: Autoría propia.

Dentro de este repositorio, el entorno virtual del servidor requiere dependencias específicas para operar. La Figura 31 detalla el contenido del archivo requirements.txt, donde se listan las librerías necesarias, incluyendo los paquetes de google-auth para la gestión de tokens criptográficos y requests para las peticiones de red directas.

Figura 31. Archivo de Dependencias del Servidor (requirements.txt)



Nota. Listado de librerías de Python requeridas para la ejecución de la aplicación web, el manejo de imágenes y la autenticación de Google. Fuente: Autoría propia.

A nivel de código (archivo app.py), el desarrollo se dividió en dos capas. La primera capa, correspondiente al *backend*, se encarga de la seguridad y comunicación. Según se muestra en la Figura 32, el sistema extrae las credenciales cifradas de la cuenta de servicio mediante st.secrets y genera tokens temporales OAuth 2.0. Estos tokens se inyectan en las cabeceras de peticiones HTTP puras (requests.get) hacia la API REST de Google Drive, evitando errores de *sockets* y reduciendo la latencia de respuesta.

Figura 32. Lógica de Autenticación y Peticiones REST a Google Drive

```

import streamlit as st
import pandas as pd
import requests
import io
from PIL import Image
from google.oauth2 import service_account
import google.auth.transport.requests

st.set_page_config(page_title="Seguridad San José", page_icon="🚗", layout="centered")

CARPETA_RAIZ_ID = "1HRA_2wC9sEUHonaw_uCRe9R0YxACPZ6j"

# --- AUTENTICACIÓN SEGURA POR TOKEN HTTP ---
def obtener_token():
    credenciales = service_account.Credentials.from_service_account_info(
        st.secrets["gcp_service_account"],
        scopes=["https://www.googleapis.com/auth/drive.readonly"]
    )
    auth_request = google.auth.transport.requests.Request()
    credenciales.refresh(auth_request)
    return credenciales.token

def consultar_drive(url, params=None):
    """Realiza peticiones HTTP directas evitando el fallo de SSL sockets."""
    token = obtener_token()
    headers = {"Authorization": f"Bearer {token}"}
    response = requests.get(url, headers=headers, params=params)
    if response.status_code == 200:
        return response.json()
    else:
        return None

def listar_archivos(parent_id):
    url = "https://www.googleapis.com/drive/v3/files"
    params = {
        "q": f"'{parent_id}' in parents and trashed=false",
        "orderBy": "name desc",
        "fields": "files(id, name, mimeType)"
    }
    data = consultar_drive(url, params)
    if data:
        return data.get('files', [])

```

Nota. Segmento de código mostrando la generación de tokens temporales de acceso y la estructuración de peticiones HTTP seguras. Fuente: Autoría propia.

La segunda capa lógica corresponde al *frontend* o interfaz de usuario. Como se expone en la Figura 33, se implementó una estructura de navegación en cascada mediante menús desplegables (selectbox). Esta lógica permite al usuario filtrar las evidencias por mes, día y categoría (vehículos o personas). Una vez seleccionada la ruta, el código itera sobre los

archivos encontrados, descarga los paquetes de bytes temporalmente en memoria mediante la librería io y renderiza las imágenes en pantalla sin saturar el almacenamiento del servidor web.

Figura 33.Lógica de Navegación en Cascada y Renderizado de Interfaz

```

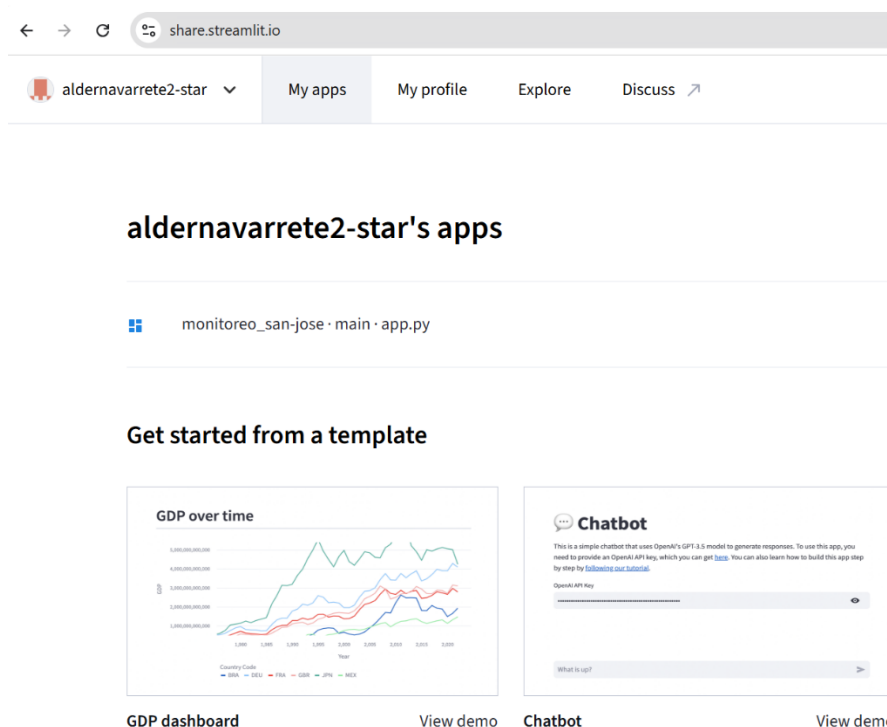
55 def check_password():
83     st.write("Navegación optimizada mediante API REST y tokens cifrados.")
84
85     # 1. Seleccionar Mes
86     meses = listar_archivos(CARPETA_RAIZ_ID)
87     if meses:
88         mes_seleccionado = st.selectbox("📅 Seleccione el Mes:", meses, format_func=lambda x: x['name'])
89
90     # 2. Seleccionar Día
91     dias = listar_archivos(mes_seleccionado['id'])
92     if dias:
93         dia_seleccionado = st.selectbox("📅 Seleccione el Día:", dias, format_func=lambda x: x['name'])
94
95     # 3. Seleccionar Categoría
96     categorias = listar_archivos(dia_seleccionado['id'])
97     if categorias:
98         cat_seleccionada = st.selectbox("📁 Seleccione la Categoría:", categorias, format_func=lambda x: x['name'])
99
100    # 4. Mostrar fotos
101    fotos = listar_archivos(cat_seleccionada['id'])
102
103    if fotos:
104        st.info(f"📄 Se encontraron {len(fotos)} registros en esta carpeta.")
105        for foto in fotos:
106            if "image" in foto['mimeType']:
107                st.markdown(f"***Archivo:** {foto['name']}")
108                with st.spinner('Descargando evidencia...'):
109                    img = descargar_imagen(foto['id'])
110                    if img:
111                        st.image(img, use_container_width=True)
112                    else:
113                        st.error("No se pudo descargar la imagen.")
114                st.divider()
115            else:
116                st.warning("No hay detecciones en esta carpeta.")
117        else:
118            st.write("No hay categorías para este día.")
119    else:
120        st.write("No hay días registrados en este mes.")
121    else:
122        st.warning("El robot no encuentra la carpeta principal. Verifique permisos de compartición en Drive.")

```

Nota. Algoritmo de filtrado cronológico para la exploración estructurada de las fotografías y su visualización dinámica. Fuente: Autoría propia.

Completado el código, el repositorio de GitHub se vinculó directamente a Streamlit Community Cloud para su puesta en producción. La Figura 34 demuestra el éxito del despliegue en la plataforma, donde el servidor virtual asume la ejecución continua de la aplicación, asignándole un dominio público accesible desde cualquier navegador web.

Figura 34. Despliegue de la Aplicación en Streamlit Community Cloud



Nota. Panel de administración del servidor en la nube confirmando el estado activo de la aplicación principal. Fuente: Autoría propia.

Finalmente, para resguardar la privacidad de los datos agrícolas, se implementó una barrera de seguridad en la capa de presentación. Tal y como se observa en la Figura 35, al ingresar a la dirección web del panel, el sistema intercepta la navegación mostrando una pantalla de "Acceso Restringido". El usuario final debe ingresar credenciales de autorización válidas antes de que el script ejecute las llamadas a la API de Google Drive y revele las evidencias del perímetro.

Figura 35. Interfaz Final de Usuario y Barrera de Acceso Restringido

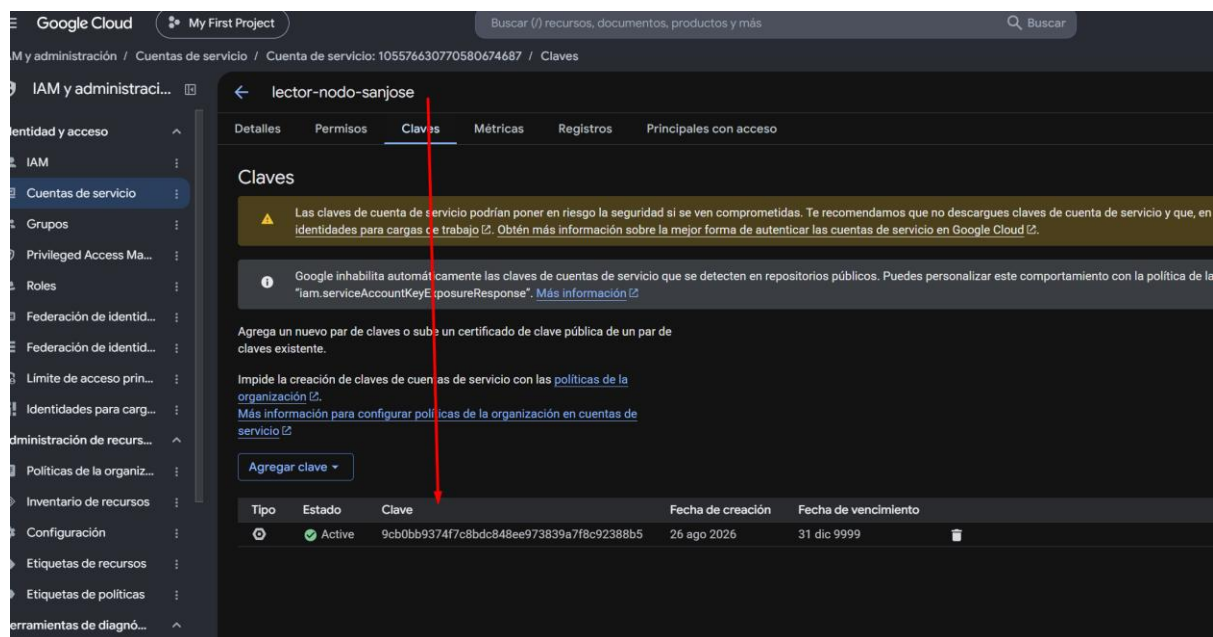


Nota. Vista principal de la aplicación web en producción, evidenciando el formulario de seguridad requerido para acceder al panel de control. Fuente: Autoría propia.

3.5.3. Gestión Criptográfica y Variables de Entorno (Secrets)

Una vez habilitada la cuenta de servicio en Google Cloud Platform, es necesario establecer un mecanismo de autenticación autónoma para la aplicación web. Para ello, se generó un par de claves criptográficas asociadas a dicha cuenta de servicio. Como se observa en la Figura 36, la plataforma emite y registra esta clave de acceso asimétrica, mostrando su estado activo, fecha de creación y opciones de administración para permitir su revocación en caso de vulnerabilidades.

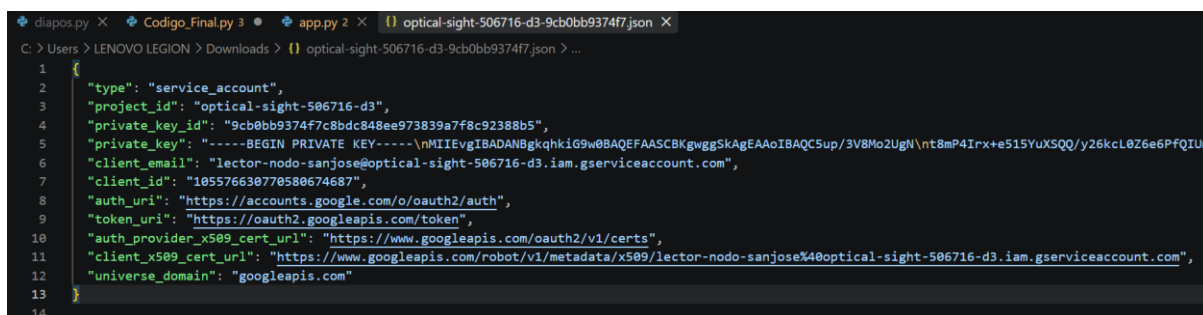
Figura 36. Generación de Claves de Autenticación en Google Cloud



Nota. Panel de administración de IAM y Cuentas de servicio donde se gestionan las claves criptográficas de acceso. Fuente: Autoría propia.

El sistema de Google Cloud exporta esta clave mediante un archivo estructurado en formato JSON. Según se detalla en la Figura 37, este documento contiene parámetros críticos para la autorización, tales como el `project_id`, el identificador de la clave (`private_key_id`), la clave privada de encriptación (`private_key`) y la dirección de correo electrónico del agente de servicio (`client_email`). Por principios estrictos de ciberseguridad, este documento confidencial se aísla del control de versiones público (GitHub) para evitar la exposición de accesos no autorizados a la infraestructura.

Figura 37. Estructura Interna del Archivo de Credenciales JSON



```

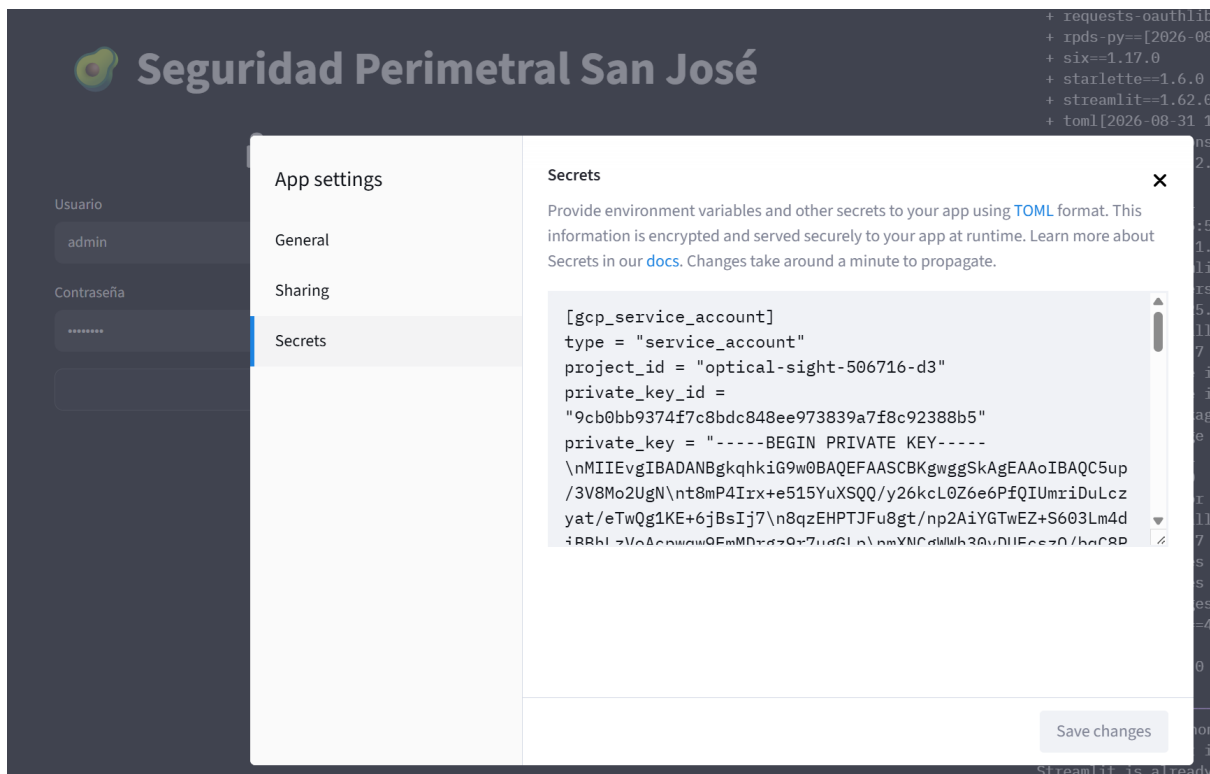
1 {
2   "type": "service_account",
3   "project_id": "optical-sight-506716-d3",
4   "private_key_id": "9cb0bb9374f7c8bdc848ee973839a7f8c92388b5",
5   "private_key": "-----BEGIN PRIVATE KEY-----\nMIIEvgIBADANBgkqhkiG9w0BAQEFAASCBAgEAAoIBAQC5up/3V8Mo2UgN\nt8mP4Irx+e51SYuXSQQ/y26kcL0Z6e6PfQIU\n6   "client_email": "lector-nodo-sanjose@optical-sight-506716-d3.iam.gserviceaccount.com",
7   "client_id": "105576630770580674687",
8   "auth_uri": "https://accounts.google.com/o/oauth2/auth",
9   "token_uri": "https://oauth2.googleapis.com/token",
10  "auth_provider_x509_cert_url": "https://www.googleapis.com/oauth2/v1/certs",
11  "client_x509_cert_url": "https://www.googleapis.com/robot/v1/metadata/x509/lector-nodo-sanjose%40optical-sight-506716-d3.iam.gserviceaccount.com",
12  "universe_domain": "googleapis.com"
13 }
14

```

Nota. Estructura del archivo de credenciales descargado, mostrando los parámetros y llaves requeridos por la API de Google Drive. Fuente: Autoría propia.

Para que el servidor de producción en Streamlit Community Cloud pueda consumir estas credenciales de forma segura, se utilizó el gestor nativo de variables de entorno (*Secrets*). Como se evidencia en la Figura 38, la información del archivo JSON fue transcrita a formato TOML e inyectada directamente en la configuración interna de la aplicación web. Esta arquitectura permite que el código invoque la autorización mediante el atributo `st.secrets`, manteniendo el repositorio de GitHub completamente sanitizado y aislando los datos sensibles del entorno de desarrollo público.

Figura 38. Inyección de Variables de Entorno en Streamlit Community Cloud



Nota. Panel de configuración avanzada (Secrets) de la aplicación web, donde se almacenan las credenciales encriptadas en formato TOML para su consumo en tiempo de ejecución.

Fuente: Autoría propia.

3.5.4. Automatización de Arranque y Supervisión de Servicios

Para garantizar una operación autónoma ante fallos de suministro eléctrico o caídas de tensión por baja radiación solar, el script de Python no se ejecuta de forma manual. Se encapsula dentro de un servicio gestionado por el administrador del sistema operativo (*systemd*). Esta configuración obliga al nodo a reiniciar el código de vigilancia automáticamente tras cada ciclo de encendido, manteniendo la operatividad 24/7 sin intervención del usuario.

3.6. Almacenamiento de Datos

El diseño del sistema de seguridad perimetral implementa una arquitectura de almacenamiento híbrida, estructurada en dos niveles complementarios: físico (local) y remoto (en la nube). Esta dualidad tecnológica se justifica por los desafíos inherentes de operar un nodo IoT en un entorno agrícola aislado. Por un lado, la retención local garantiza que el sistema siga operando y guardando evidencias sin interrupciones, incluso frente a caídas temporales de la red celular LTE/4G. Por otro lado, la sincronización paralela hacia un servidor remoto actúa como un respaldo de seguridad crítico que protege la información en caso de daño físico, vandalismo o robo del equipo en campo, siendo además el pilar que permite al usuario final consultar los eventos de forma remota y asíncrona.

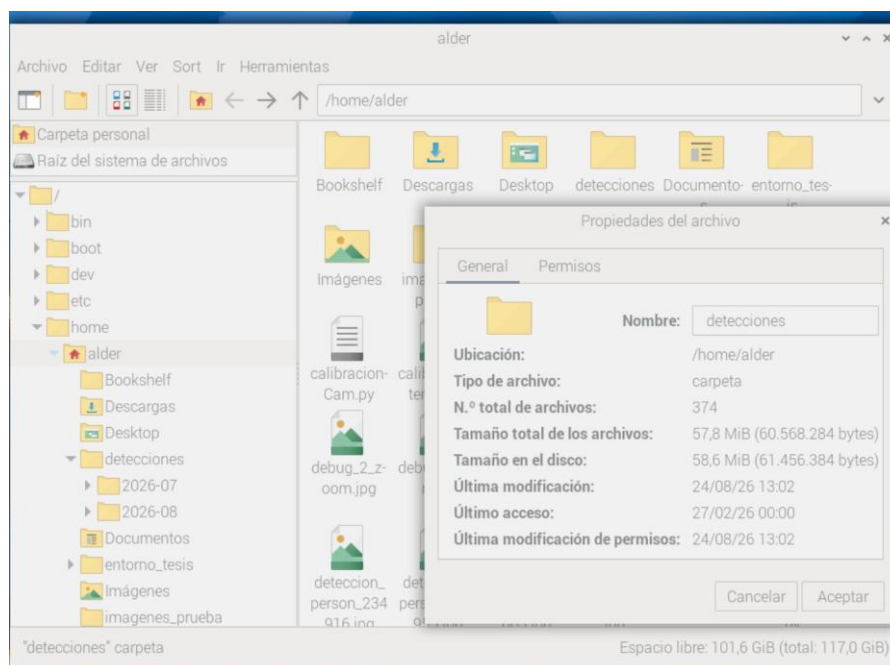
3.6.1 Almacenamiento de Datos Local

El registro de los datos obtenidos tras la detección de movimiento es una fase fundamental dentro del sistema de seguridad perimetral, ya que permite acceder a un historial detallado sobre las intrusiones, las horas de mayor actividad y los vehículos detectados. Este mecanismo de *Data Logging* local facilita al propietario de la plantación o administrador la supervisión de los eventos sin depender de conectividad a la nube o servidores externos, garantizando que la evidencia (fotografías y metadatos) no se pierda ante posibles caídas de la red 4G/LTE. De esta manera, se mejora la trazabilidad de los incidentes y se respalda la toma de decisiones para la seguridad del sector.

La gestión de los datos generados por el nodo *Edge* requiere un enfoque híbrido para garantizar la integridad de las evidencias frente a posibles intermitencias o caídas en la red celular. En primera instancia, todas las fotografías capturadas y clasificadas positivamente por los modelos de inteligencia artificial se guardan de manera local en la memoria SD de la Raspberry Pi. Como se detalla en la Figura 39, el sistema crea dinámicamente un árbol de

directorios jerárquico dentro de la carpeta raíz del usuario, organizando los eventos automáticamente por año, mes, día y categoría. Esta estructura modular facilita la administración del espacio en disco del microordenador y asegura un respaldo físico *in situ* inalterable.

Figura 39. Estructura de Directorios para el Almacenamiento Local en la Raspberry Pi

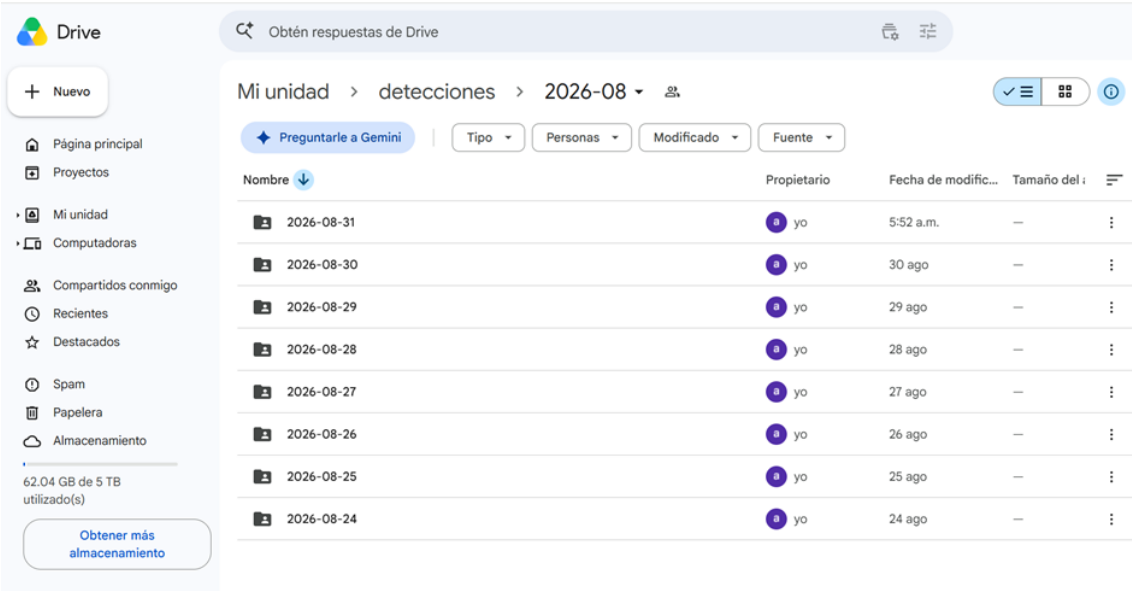


Fuente: Autoría propia

3.6.2. Almacenamiento de Datos Web (Nube)

Para asegurar la disponibilidad remota de las evidencias fotográficas y permitir su consumo a través de la interfaz web, el sistema implementa una réplica exacta del directorio local hacia la nube. Mediante la ejecución en segundo plano de la herramienta de sincronización rclone, los archivos locales se transfieren de forma cifrada a través de la red LTE/4G hacia una cuenta de servicio de Google Drive. Tal y como se ilustra en la Figura 40, el repositorio web mantiene la misma arquitectura cronológica generada por el nodo físico (desglosando las carpetas por mes y día), lo que resulta fundamental para que la aplicación web pueda indexar, buscar y presentar los registros históricos al usuario final de manera eficiente y estructurada.

Figura 40. Sincronización Cronológica de Evidencias en Google Drive



Nota. Interfaz web de Google Drive mostrando las subcarpetas sincronizadas diariamente desde el nodo *Edge*, manteniendo la misma jerarquía del almacenamiento local. Fuente: Autoría propia.

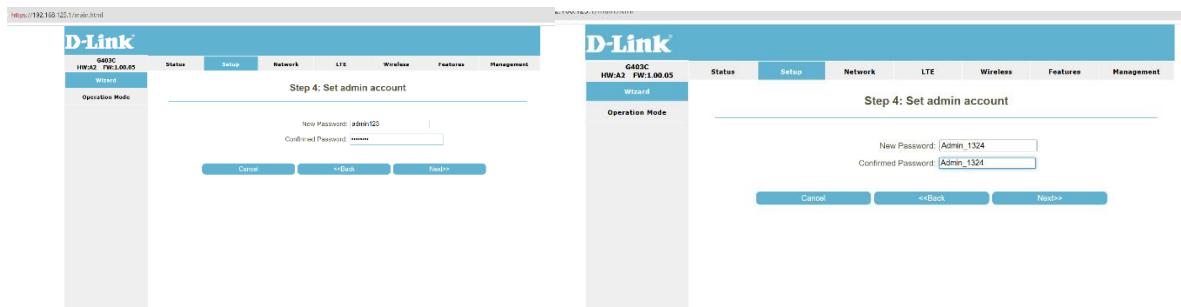
3.7. Integración y pruebas locales

En esta fase se llevó a cabo la integración final del hardware del nodo Edge Computing (Raspberry Pi 5, cámara Picamera2 y etapa de potencia) con la nueva infraestructura de telecomunicaciones. Tras descartar el uso de un dispositivo móvil como Access Point debido a su inestabilidad,

3.7.1. Configuración del enrutador LTE

Para proveer salida a Internet y comunicación de área local (LAN), se desplegó un enrutador LTE D-Link G403C. El proceso de inicialización consistió en establecer credenciales de acceso seguro para la administración del equipo, tal como se detalla en la configuración inicial de la Figura 41.

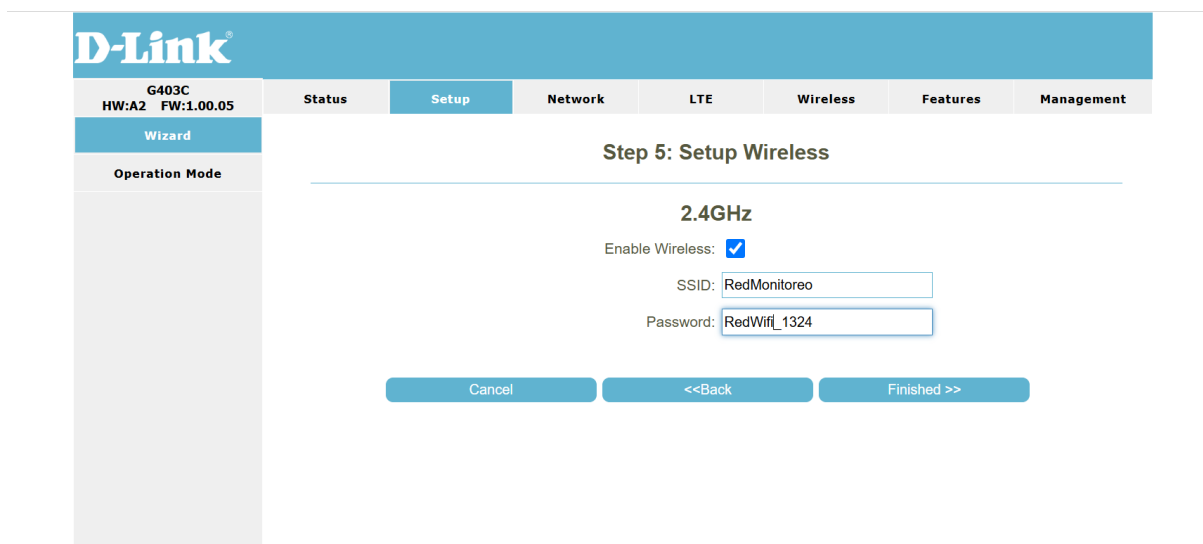
Figura 41. Configuración de credenciales de administrador en enrutador D-Link



Fuente: Autoría propia

Posteriormente, se levantó la red inalámbrica local asignando el identificador de conjunto de servicios (SSID) "RedMonitoreo" como se observa en la figura 42, la cual centraliza la conexión inalámbrica de los equipos de calibración y el nodo IoT de forma aislada para evitar intrusiones

Figura 42. Configuración de red WLAN SSID y seguridad



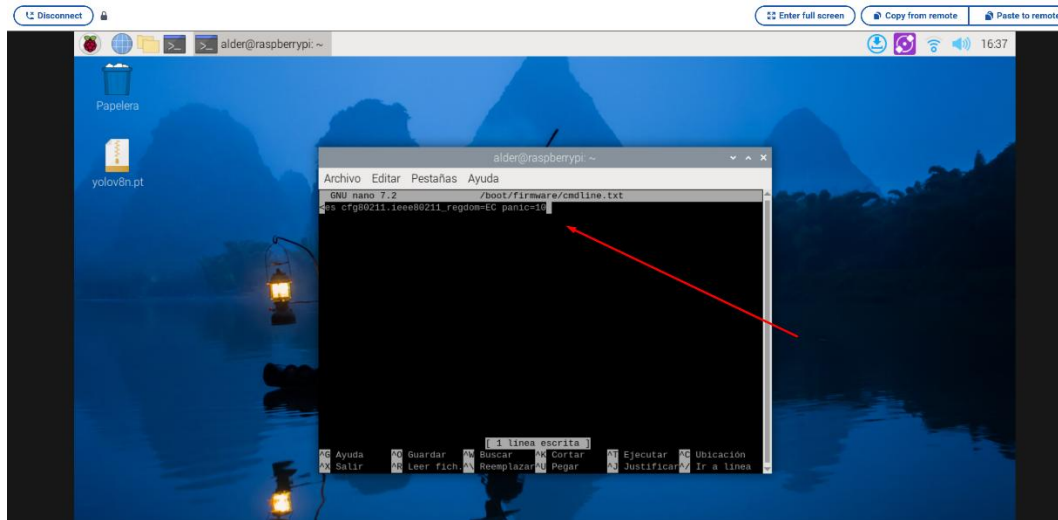
Fuente: Autoría propia

3.7.2. Depuración de arranque de la Raspberry Pi 5

Al integrar la placa en la carcasa metálica final, el sistema operativo presentó bloqueos críticos de arranque (*kernel panic*) vinculados al módulo de dominio regulatorio inalámbrico (cfg80211). Para forzar al núcleo de Linux a ignorar este conflicto y asegurar el reinicio

autónomo del equipo, se editó el archivo de configuración de arranque `/boot/firmware/cmdline.txt` que se observa en la Figura 43.

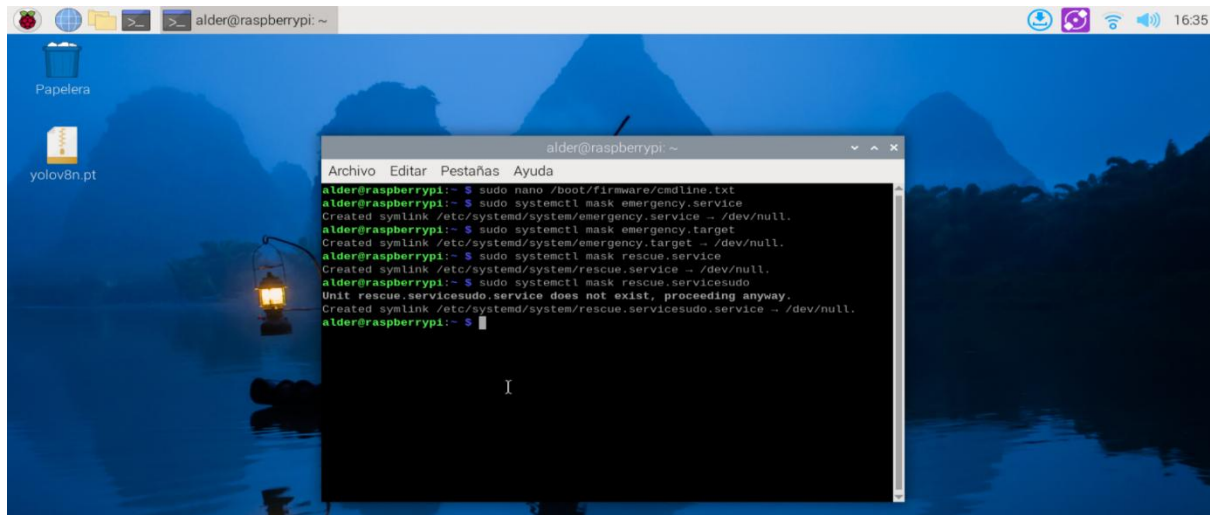
Figura 43. Edición de parámetros de arranque en `cmdline.txt`



Fuente: Autoría propia

Finalmente, para evitar que el sistema operativo se detenga esperando interacción del usuario ante fallos menores de red o hardware, se procedió a enmascarar los servicios de emergencia (`emergency.service`) y rescate (`rescue.service`) del gestor de sistema `systemd` (ver Figura 44). Esta configuración garantiza que el script de visión artificial opere de manera perpetua en el entorno de producción.

Figura 44. *Enmascaramiento de servicios de emergencia mediante systemctl*



Fuente: Autoría propia

3.7.3. Validación del sistema de alertas y visualización de datos.

Para comprobar el correcto funcionamiento de la tubería de datos (data pipeline), se ejecutó un entorno de pruebas controlado utilizando imágenes pregrabadas de vehículos y personas. El objetivo consistió en validar la latencia de inferencia de los modelos YOLOv8 y EasyOCR, así como la correcta transmisión de la información hacia las plataformas de usuario final.

3.7.3.1. Inferencia y notificaciones en tiempo real

Al iniciar el script de reconocimiento, la consola del sistema operativo evidenció el procesamiento secuencial de las imágenes. El algoritmo logró clasificar exitosamente la presencia de personas y vehículos, extrayendo las cadenas de texto de las matrículas (por ejemplo, "FARM02") e invocando la sincronización forzada hacia la nube mediante la herramienta rclone (ver Figura 45).

Figura 45 .Ejecución del script de inferencia y registro de eventos en consola

```

alder@raspberrypi: ~
Archivo Editar Pestañas Ayuda
der.py:775: UserWarning: 'pin_memory' argument is set as true but no accelerator
is found, then device pinned memory won't be used.
  super().__init__(loader)
[11] Notificación de Telegram enviada.
[11] Vehículo (Placa: LpI) detectado y guardado.

[11] Procesando imagen: imagenes_prueba/carro_1.jpg
[11] Notificación de Telegram enviada.
[11] Vehículo (Placa: FARM02) detectado y guardado.

[11] Procesando imagen: imagenes_prueba/carro_2.jpg
[11] Notificación de Telegram enviada.
[11] Vehículo (Placa: INDETERMINADA) detectado y guardado.

[11] Procesando imagen: imagenes_prueba/carro_3.jpg
[11] No se detectaron personas ni vehículos en esta imagen.

[11] Procesando imagen: imagenes_prueba/personas_1.jpg
^ Error Telegram: HTTPConnectionPool(host='api.telegram.org', port=443): Read
timed out. (read timeout=10)
[11] Persona detectada y guardada.

[11] Procesando imagen: imagenes_prueba/personas_4.jpg
[11] Notificación de Telegram enviada.
[11] Persona detectada y guardada.
[11] Notificación de Telegram enviada.
[11] Persona detectada y guardada.
[11] Notificación de Telegram enviada.
[11] Persona detectada y guardada.
[11] Notificación de Telegram enviada.
[11] Persona detectada y guardada.

[11] Procesando imagen: imagenes_prueba/personas_2.jpg
[11] Notificación de Telegram enviada.
[11] Persona detectada y guardada.

[▲ Sincronización] Forzando subida de archivos a Google Drive para prueba...
[✓ Sincronización] Respaldo en la nube completado con éxito.

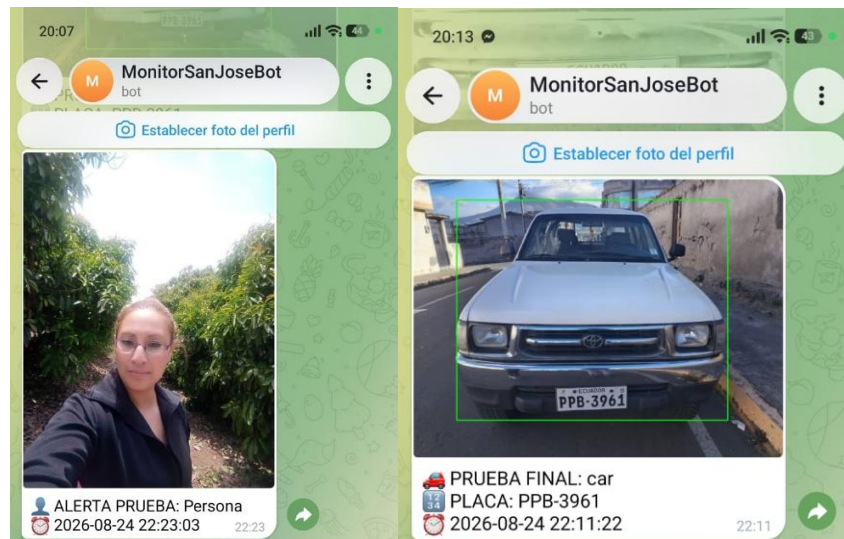
✓ ¡Prueba finalizada! Revisa tu Google Drive para confirmar que las fotos y el
TXT subieron correctamente.
(entorno_tesis) alder@raspberrypi:~ $

```

Fuente: Autoría propia

Simultáneamente, el sistema estableció conexión con la API de Telegram, transmitiendo las alertas estructuradas al dispositivo móvil del administrador. Se verificó la recepción de notificaciones inmediatas con la fotografía del evento, marca de tiempo y clasificación para la detección de transeúntes (ver Figura 46) y para el reconocimiento de vehículos con su respectiva lectura OCR (ver Figura 46).

Figura 46. Notificación , Reconocimiento vehicular y lectura de placa a través de Telegram



Fuente: Autoría propia

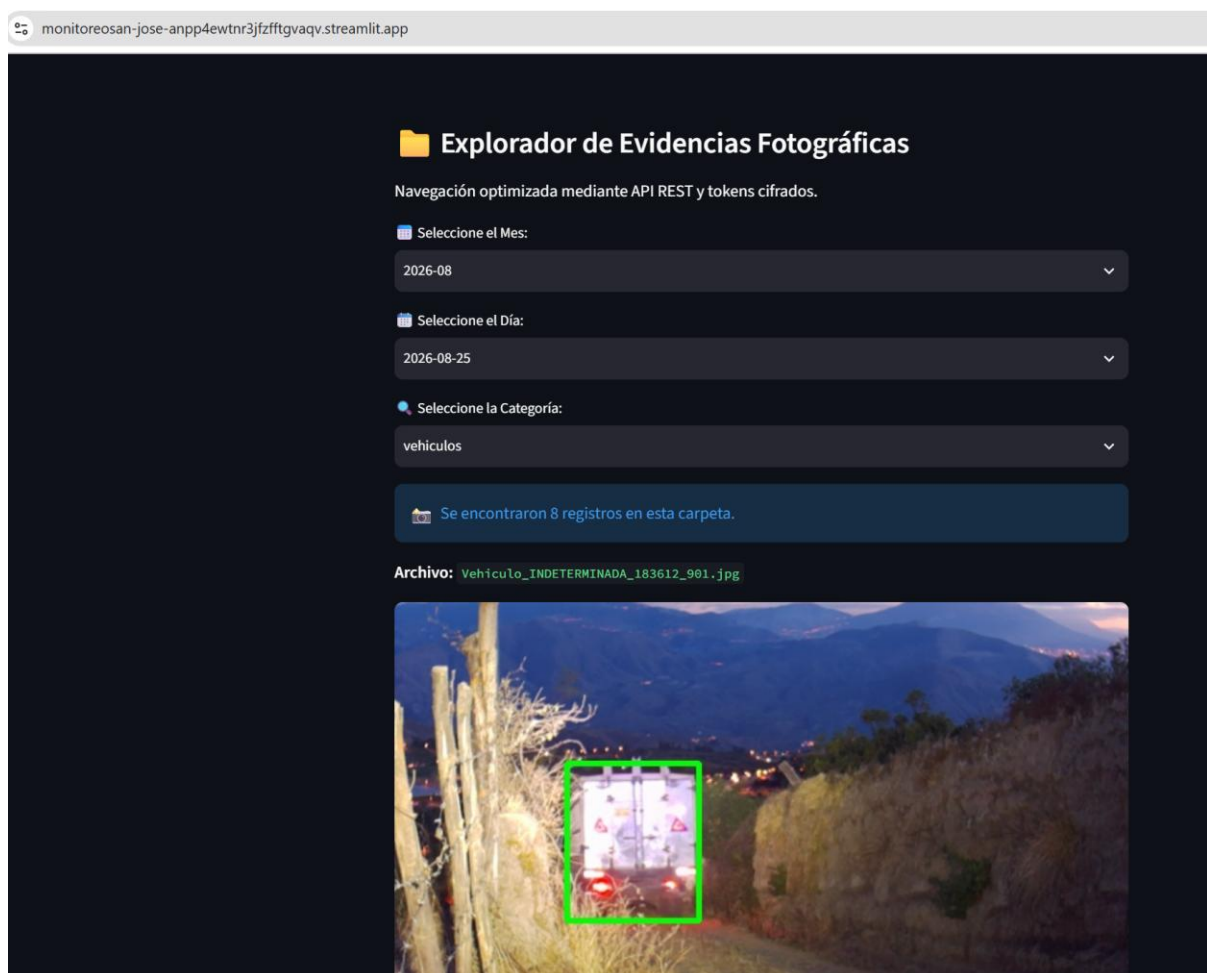
3.7.4 Despliegue del panel de control web

Para dotar al sistema de una interfaz gráfica accesible sin requerir conocimientos técnicos por parte del usuario final, se desarrolló un panel de control (dashboard) web personalizado utilizando Python y el marco de trabajo Streamlit. Esta plataforma se desplegó en la nube mediante un servicio de alojamiento continuo (Cloud Hosting), garantizando una disponibilidad operativa 24/7 sin necesidad de infraestructura local adicional.

A nivel de arquitectura de software, el sistema opera mediante un esquema desacoplado: el repositorio en Google Drive actúa como servidor de almacenamiento documental (Backend), mientras que la interfaz web se comunica de forma cifrada mediante tokens de acceso temporal y llamadas a la API REST de Google. Para proteger la información sensible del sector San José (registros históricos en formato CSV y evidencias fotográficas clasificadas por categorías), se implementó una capa de autenticación basada en credenciales de acceso restringido en el extremo de la aplicación. Como resultado, la interfaz consolida de

manera automática la telemetría y las carpetas de respaldo, permitiendo una navegación fluida y segura desde cualquier dispositivo con acceso a internet (ver Figura 47).

Figura 47. Panel de control web para la visualización de registros almacenados en la nube



Fuente: Autoría propia

4.CAPÍTULO IV: IMPLEMENTACION Y PRUEBAS FUNCIONALES

Los resultados de la materialización y la instalación del sistema de monitoreo Edge IoT para las plantaciones de aguacates en el sector San José, cantón Mira, se hacen presentes en este capítulo. Se expone el proceso de implementación de la parte estructural, del área electrónica y de la comunicación que hicieron posible la adaptación del diseño teórico a un escenario agrícola complejo, en el que se registran obstáculos topográficos, de clima y de conectividad. Con la puesta en escena de este despliegue, se establece la viabilidad técnica para la operación de un nodo de seguridad perimetral completamente autónomo, alimentado por energía solar y que puede llevar a cabo tareas complejas sin depender de infraestructura externa.

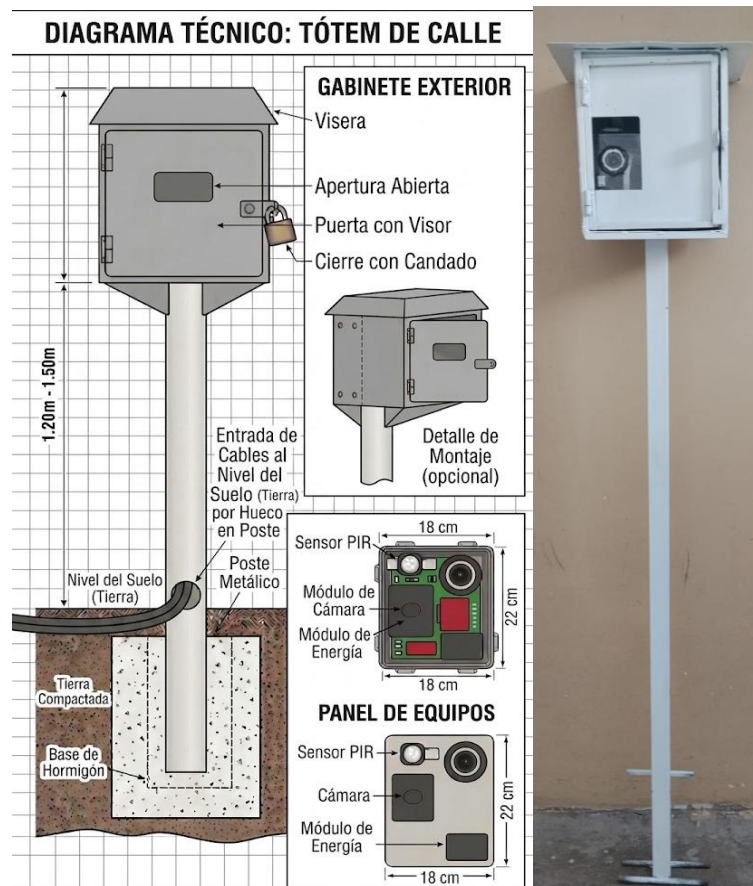
Adicionalmente, este capítulo expone un análisis exhaustivo de las pruebas de hardware y software ejecutadas en campo, evaluando el rendimiento del procesamiento local con visión artificial para la detección de personas y el reconocimiento de matrículas vehiculares. Se contrastan los datos empíricos recolectados por el sistema automatizado frente a los métodos de vigilancia tradicional, midiendo parámetros críticos como la precisión de los modelos de inteligencia artificial, la latencia en el envío de alertas remotas y la eficiencia del consumo energético. Finalmente, se consolida la viabilidad económica del proyecto detallando los costos de inversión y se discuten los beneficios operativos directos que esta solución tecnológica aporta a los productores locales.

4.1. Implementación Estructural del Sistema

La implementación estructural del sistema de monitoreo se llevó a cabo en la vía de acceso principal de las plantaciones de aguacate del sector San José, cantón Mira. Para garantizar la máxima cobertura visual y proteger los componentes de intervenciones no autorizadas o fauna local, el nodo principal requirió una estructura metálica elevada a una altura aproximada de 2.5 a 3 metros (ver Figura 48). Esta elevación proporciona a la cámara Arducam

un ángulo de picado óptimo, permitiendo capturar el frente de los vehículos para la correcta extracción de las matrículas, la detección de personas y la reducción de los puntos ciegos generados por el crecimiento del follaje.

Figura 48. Estructura metálica elevada para la protección del nodo principal



Fuente: Elaboración propia, con asistencia de inteligencia artificial (2026).

El panel solar monocristalino de 100W fue anclado en la parte superior de la estructura de soporte, con una inclinación orientada hacia la máxima incidencia solar, aprovechando el promedio de 4 horas de Sol Pico (HSP) características de la zona (ver Figura 49).

Figura 49. Instalación del panel solar monocristalino en la estación de energía



Fuente: Autoría propia

Para salvaguardar la electrónica frente a las condiciones climáticas (lluvia, humedad y polvo), los elementos sensibles como la Raspberry Pi 5, el controlador de carga PWM, el módulo reductor de voltaje y los relés, fueron confinados dentro de un gabinete de protección transparente con certificación IP65 (ver Figura 50). Asimismo, el sensor de movimiento PIR y la lente de la cámara se ajustaron milimétricamente en la cubierta frontal, orientados estratégicamente hacia el área de detección térmica y la focalización del camino empedrado.

Figura 50. Componentes electrónicos y ópticos confinados en el gabinete IP65



Fuente: Autoría propia

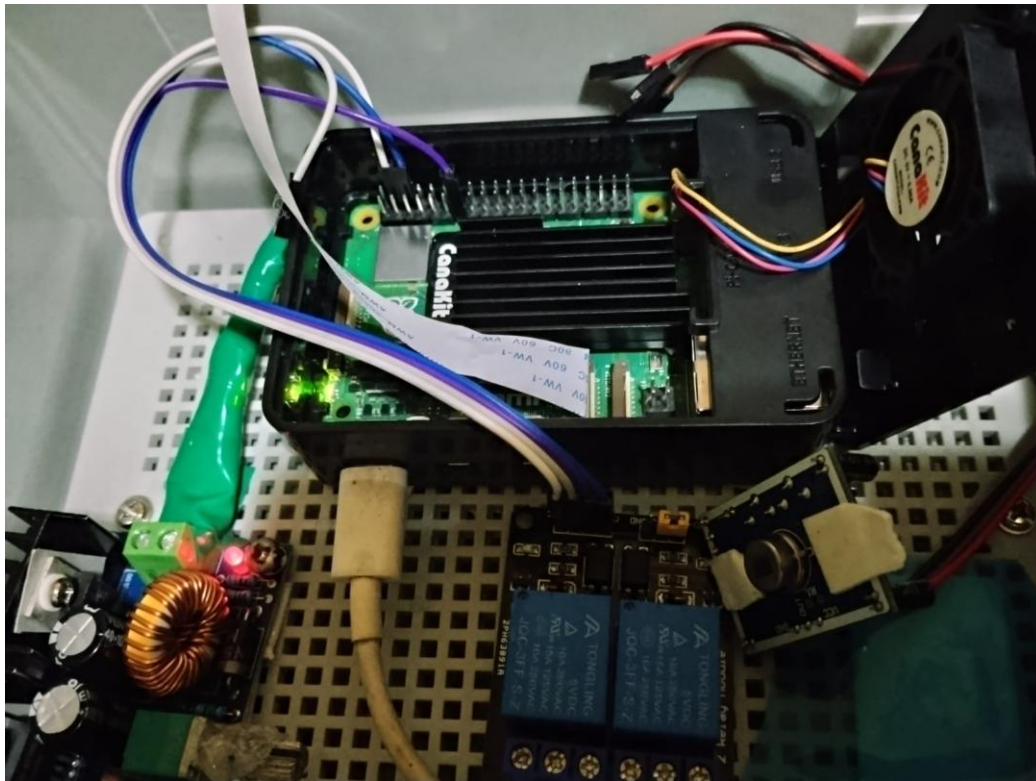
4.2. Implementación Electrónica del Sistema

La integración electrónica del nodo Edge AI se fundamenta en una topología de corriente continua (DC), eliminando el uso de inversores AC para maximizar la eficiencia energética del sistema autónomo. El circuito principal inicia en el controlador de carga PWM, el cual recibe la energía captada por el panel solar y la regula para cargar el banco de almacenamiento primario, compuesto por la batería de Gel de ciclo profundo de 12V y 50Ah. A partir de la salida de carga (Load) de 12V, el circuito se divide en dos subsistemas críticos:

4.2.1. Subsistema de Procesamiento

La tensión de 12V es dirigida a un módulo reductor de voltaje (Buck Converter Step-Down), el cual estabiliza y transforma la corriente a 5V y 5A. Esta salida se conecta directamente al puerto USB-C de la Raspberry Pi 5, garantizando que el microprocesador reciba la potencia sostenida de 25W necesaria para ejecutar los modelos de visión artificial (YOLOv8n) sin sufrir caídas de tensión (Under-voltage throttling).

Figura 51. Conexión del módulo reductor de voltaje a la Raspberry Pi 5 para alimentación de 25W.

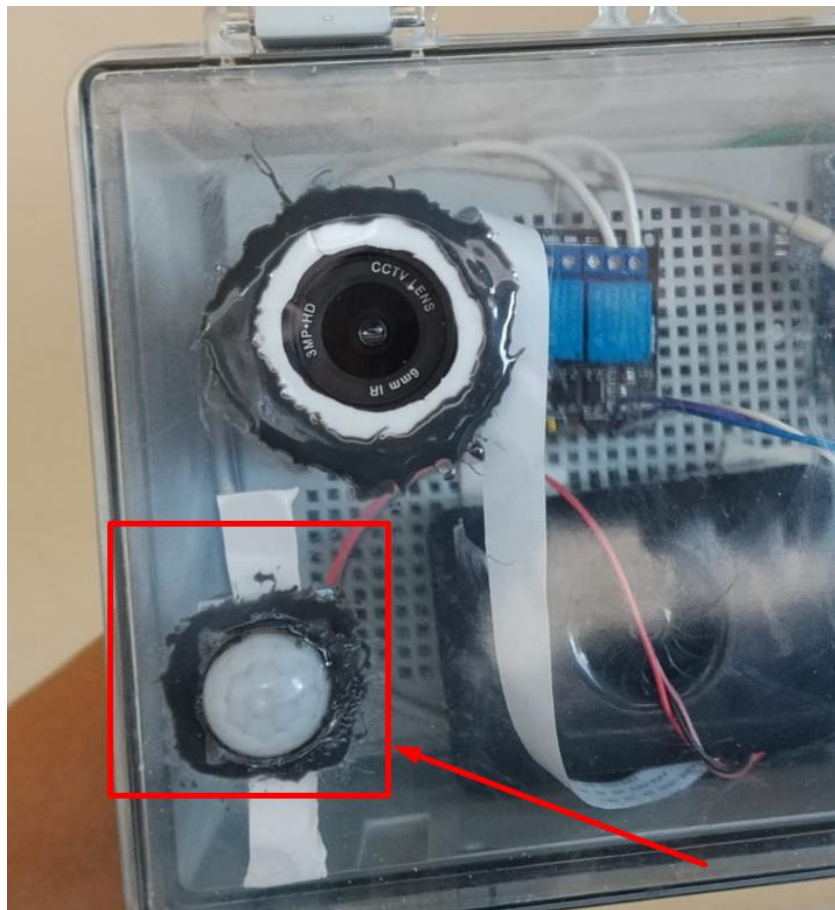


Fuente: Autoría propia

4.2.2. Subsistema de Detección y Actuación

El sensor PIR HC-SR501 se alimenta de los pines de 5V de la Raspberry Pi, conectando su pin de datos a un puerto GPIO configurado para generar interrupciones por hardware. Paralelamente, el módulo de relé optoacoplado de 5V recibe la señal lógica de actuación de la placa, cerrando el circuito de 12V que alimenta directamente al reflector LED de 20W. Esta configuración aísla galvánicamente el circuito de alta potencia (iluminación) del circuito lógico, protegiendo el procesador de posibles picos inductivos.

Figura 52. Sensor PIR HC-SR501

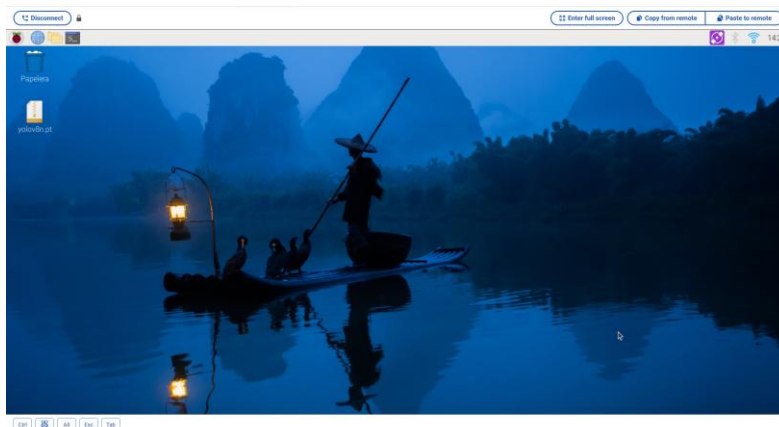


Fuente: Autoría propia

4.3. Implementación del Sistema Tecnológico y de Comunicaciones

Para facilitar el despliegue del sistema en el entorno rural y evitar la necesidad de conectar periféricos físicos (monitor, teclado y ratón) a la intemperie, se habilitó el acceso remoto seguro a la Raspberry Pi 5. Como se evidencia en la Figura 53, mediante el enlace de red (Wi-Fi/4G) se estableció un túnel de control remoto hacia el entorno de escritorio del microprocesador. Esta interfaz de gestión técnica permitió transferir los pesos del modelo de inteligencia artificial (yolov8n.pt) directamente a la memoria de la placa, así como compilar el código fuente, monitorear el estado del sistema operativo y verificar la conexión a internet en tiempo real desde cualquier navegador externo.

Figura 53. Acceso remoto al entorno de escritorio de la Raspberry Pi 5 para la configuración del sistema y gestión del modelo YOLOv8n.



Fuente: Autoría propia

4.4. Pruebas de Hardware y Software

Esta sección detalla la fase de validación experimental del sistema de monitoreo perimetral. El proceso de pruebas se estructuró para comprobar de manera metódica el correcto funcionamiento de los componentes físicos (hardware) y de los algoritmos desarrollados (software) bajo condiciones reales de operación en el sector San José.

4.4.1. Cronograma de Pruebas

Las pruebas de campo se ejecutaron durante el mes de agosto de 2026. En la Tabla 22 se describen de manera organizada las actividades desarrolladas, los componentes involucrados y los resultados esperados durante el proceso de validación.

Tabla 22.

Cronograma de Pruebas

Nº	Detalle de la prueba	Involucrados	Hardware / Software	Resultados Esperados	Fechas de Ejecución
1	Verificación de autonomía energética del panel y batería.	Autor	Controlador PWM, Batería 12V, Panel Solar	Mantenimiento de voltaje estable por encima de 11.8V	Del 15 de junio al 30 de junio de 2026

2	Prueba de acoplamiento y arranque de la Raspberry Pi 5.	Autor	Buck Converter, Raspberry Pi 5	Ausencia de advertencias de bajo voltaje durante carga de procesamiento.	Del 01 de julio al 15 de julio de 2026
3	Calibración óptica y detección de movimiento.	Autor	Cámara Arducam, Sensor PIR, Script Python	Activación exitosa de la interrupción GPIO al detectar firmas térmicas.	Del 16 de julio al 31 de julio de 2026
4	Evaluación de inferencia IA (Edge Computing).	Autor	YOLOv8n, EasyOCR, OpenCV	Detección correcta de clases (persona, vehículo) y lectura de placas vehiculares.	Del 10 de agosto al 15 de agosto de 2026
5	Prueba de comunicación y alertas remotas.	Autor	Router LTE D-Link, API Telegram	Recepción de la fotografía y datos del evento en el dispositivo móvil del usuario.	Del 15 de agosto al 24 de agosto de 2026

Fuente: Autoría propia

4.4.2. Pruebas de Acoplamiento de los Componentes

Previo a la ejecución de los algoritmos de detección, se validó la correcta integración electrónica y mecánica del nodo. La evaluación se realizó mediante una lista de cotejo que permitió obtener una valoración estructurada del desempeño del hardware.

Tabla 23.

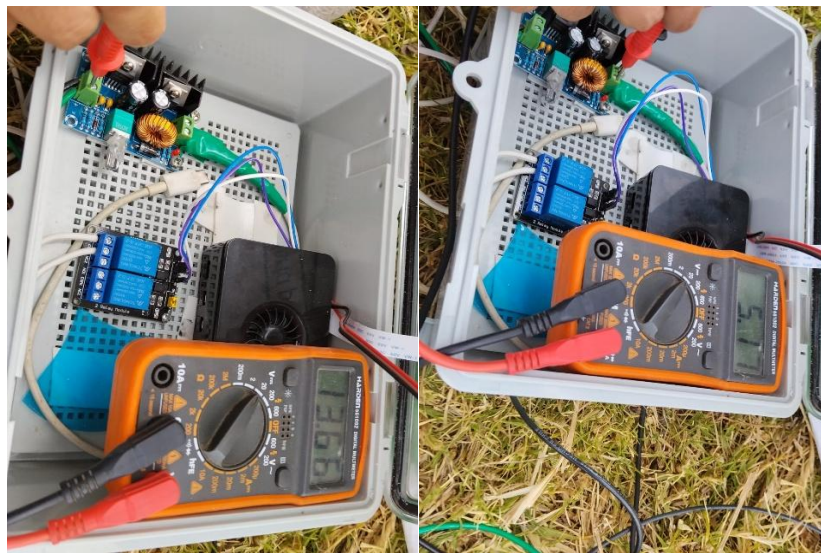
Pruebas de Acoplamiento de los Componentes

N°	Descripción de las Pruebas	Sí	No
1	Verificación de salida estable de 5V/5A desde el módulo reductor hacia la Raspberry Pi.	X	
2	Activación correcta del relé y encendido del reflector LED nocturno.		X
3	Captura de imagen nítida a través del puerto MIPI CSI-2 con la cámara Arducam.	X	
4	Arranque automático del sistema operativo y scripts al energizar la placa.		X

Fuente: Autoría.

- **Verificación de la alimentación (5V/5A):** Para garantizar la estabilidad del sistema, se midió el voltaje de salida del módulo reductor conectado a la fuente principal. Como se observa en la Figura 54, el multímetro registra una salida constante de 5.16 V, un valor óptimo que compensa las caídas de tensión del cableado y garantiza que la Raspberry Pi 5 opere sin alertas de bajo voltaje (Under-voltage) incluso bajo carga máxima de procesamiento.

Figura 54. Medición de voltaje de salida del módulo reductor hacia la Raspberry Pi 5



Fuente: Autoría propia

- **Activación del relé y reflector LED:** La validación del circuito de potencia nocturno se realizó mediante la ejecución de un script de prueba (`monitoreo_rele.py`). La Figura 55 muestra la respuesta exitosa por consola de la activación del pin GPIO, además de la evidencia del acoplamiento físico en el poste, demostrando el encendido del reflector LED mediante la conmutación mecánica del relé de 5V.

Figura 55. Prueba de Activación del Módulo Relé Mediante Script de Control por Consola



Fuente: Autoría propia

- **Captura de imagen a través del puerto MIPI CSI-2:** Se comprobó la correcta comunicación del bus CSI-2 con la cámara Arducam. La Figuras 56 muestra la nitidez del lente varifocal y la correcta calibración del enfoque físico, permitiendo al algoritmo YOLOv8n identificar vehículos de manera clara tanto en condiciones diurnas (tractor) como nocturnas (camión), con bajo ruido excesivo o desenfoque.

Figura 56. Capturas de Detección Diurna y Nocturna



Fuente: Autoría propia

- **Arranque autónomo y recuperación de fallos:** Para asegurar la operación desatendida del nodo, se modificaron los parámetros del *kernel* en el archivo `cmdline.txt` (`panic=10`) y se enmascararon los servicios de emergencia del sistema operativo (`emergency.target`, `rescue.service`). Como se detalla en la Figura 57, estas configuraciones a nivel de sistema (`systemd`) obligan a la placa a reiniciarse automáticamente tras un corte de energía o fallo crítico de lectura, en lugar de detenerse en una consola de recuperación.

Figura 57. *Modificación de Parámetros del Kernel para el Reinicio Automático y Enmascaramiento de Servicios de Recuperación en Systemd para Operación Desatendida*

```

alder@raspberrypi: ~
Archivo Editar Pestañas Ayuda
alder@raspberrypi:~$ sudo nano /boot/firmware/cmdline.txt
alder@raspberrypi:~$ sudo systemctl mask emergency.service
Created symlink /etc/systemd/system/emergency.service → /dev/null.
alder@raspberrypi:~$ sudo systemctl mask emergency.target
Created symlink /etc/systemd/system/emergency.target → /dev/null.
alder@raspberrypi:~$ sudo systemctl mask rescue.service
Created symlink /etc/systemd/system/rescue.service → /dev/null.
alder@raspberrypi:~$ sudo systemctl mask rescue.service
Unit rescue.service does not exist, proceeding anyway.
Created symlink /etc/systemd/system/rescue.service → /dev/null.
alder@raspberrypi:~$

GNU nano 7.2 /boot/firmware/cmdline.txt
<es cfg80211.ieee80211_regdom=EC panic=10

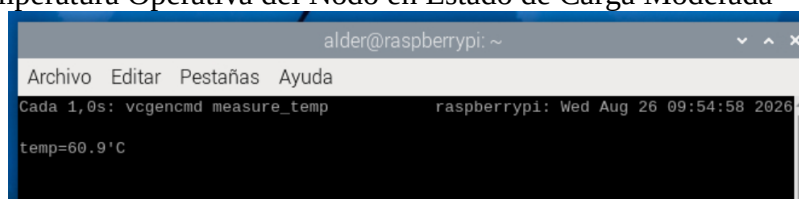
```

Fuente: Autoría propia

4.4.3. Prueba de Funcionamiento del Nodo central

Para respaldar la viabilidad del procesamiento en el borde (Edge Computing) dentro de un gabinete estanco (IP65), se monitoreó el estrés térmico del microprocesador durante la ejecución continua del algoritmo de visión artificial. Debido a que el equipo se encuentra confinado y expuesto a las condiciones climáticas del sector San José, el control de la temperatura es un factor crítico que detendría la detección.

Figura 58. Temperatura Operativa del Nodo en Estado de Carga Moderada



```

alder@raspberrypi: ~
Archivo Editar Pestañas Ayuda
Cada 1,0s: vcgencmd measure_temp          raspberrypi: Wed Aug 26 09:54:58 2026
temp=60.9°C
  
```

Fuente: Autoría propia

Al someter el sistema a una carga de trabajo extrema, capturando y procesando fotogramas en tiempo real a una tasa de 15.01 FPS, se evaluó la eficiencia del sistema de disipación activa (ventilador). Como se evidencia en la Figura 58, la temperatura del procesador alcanzó un pico de 79.6 °C. Este valor empírico demuestra que el sistema de refrigeración logra estabilizar el silicio justo por debajo del límite crítico de degradación de rendimiento (80 °C - 85 °C), garantizando una operación continua y fluida de los modelos de inteligencia artificial sin poner en riesgo la integridad del hardware.

Por otro lado, el nodo de borde operó de manera continua procesando el flujo de visión artificial para evaluar el consumo de datos. Durante la fase de diseño, se descartó la transmisión de video en tiempo real hacia el centro de control debido a las limitaciones energéticas y a la viabilidad económica del enlace LTE. Para justificar esta restricción, se modeló el consumo

teórico de una transmisión de video continua basada en el estándar ITU-T H.264 (Unión Internacional de Telecomunicaciones, 2019).

Ecuación 4.

Calculo consumo promedio de transmisión continua.

$$C_d = \frac{1Mb}{s} \times \frac{1MB}{8Mb} \times 3600s \times 24 = 10.800 MB/dia$$

Esto representa un gasto aproximado de 10.5 GB diarios (más de 300 GB mensuales), lo cual saturaría la cuota de cualquier plan de datos M2M estándar en menos de dos días y generaría una alta latencia por pérdida de paquetes.

Por este motivo, el sistema se optimizó para ejecutar los modelos YOLOv8n y EasyOCR localmente, capturando únicamente imágenes estáticas durante los eventos de detección. Esta estrategia estabilizó el procesamiento y garantizó la precisión térmica, logrando clasificar vehículos y personas a distancias de 2 a 8 metros con un consumo de red mínimo.

Figura 59. Consumo exclusivo de notificaciones de texto y fotografías estáticas



Fuente: Autoría propia

El envío exclusivo de notificaciones de texto y fotografías estáticas generó un consumo de apenas 328.49 MB, demostrando una alta eficiencia en el uso del enlace LTE frente al modelo teórico de video continuo.

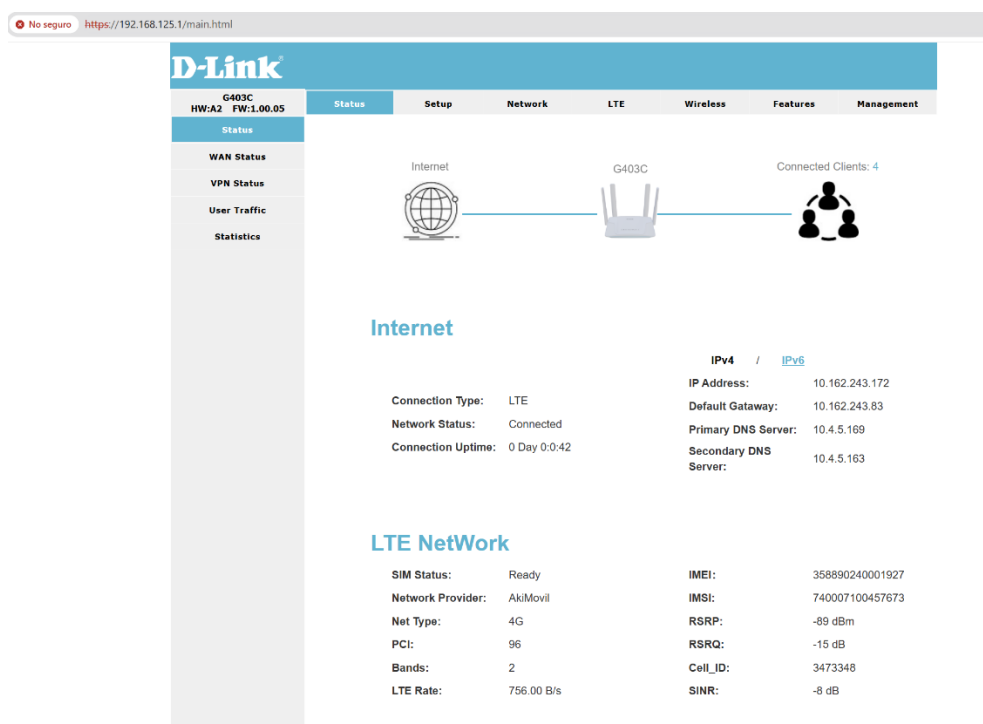
4.4.4. Prueba de Comunicación y Alertas Remotas

Para garantizar el envío inmediato de las alertas procesadas por el nodo, se evaluó la conexión a internet mediante el enrutador industrial D-Link G403C. La selección del proveedor de servicios se justificó mediante un análisis comparativo de la capa física (radiofrecuencia) directamente desde la interfaz de diagnóstico del equipo, evaluando las dos operadoras con presencia en el sector San José.

En la primera prueba, la operadora AkiMovil (infraestructura Movistar) logró registrarse en la red, pero presentó un entorno radioeléctrico hostil. Como se observa en la Figura 60, a pesar de recibir un nivel de potencia aceptable (RSRP de -89 dBm), el canal presentaba una degradación severa con un SINR crítico de -8 dB. Este valor negativo indica

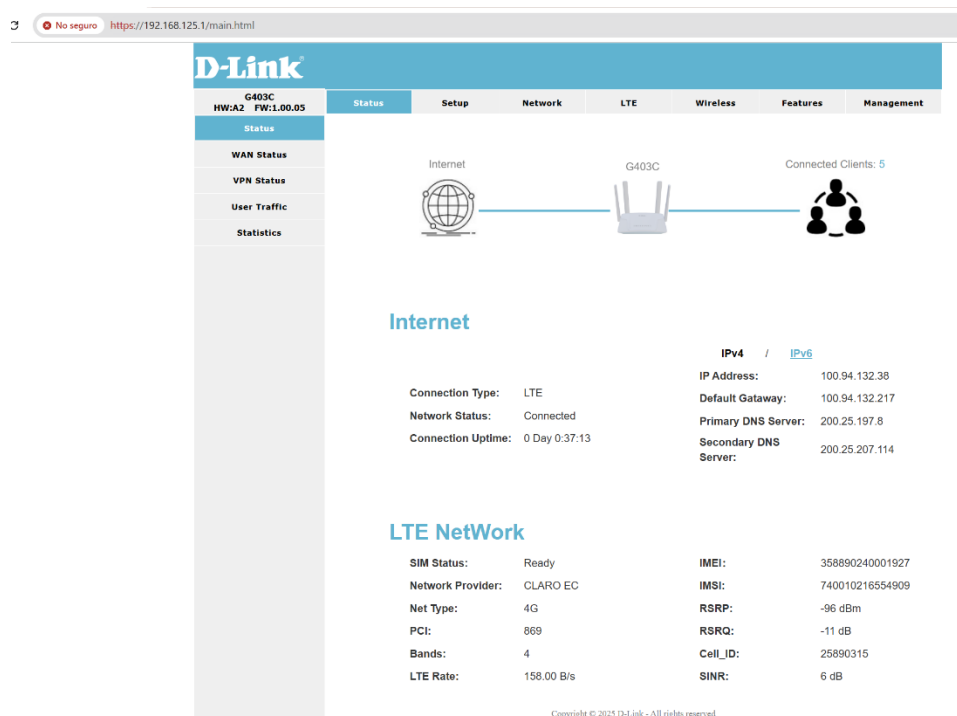
que el nivel de ruido e interferencia superaba a la propia señal útil, lo que provocó una conexión inestable y generó repetidos errores de *Timeout* al intentar enviar las alertas hacia la API de Telegram.

Figura 60. Interfaz de Diagnóstico del Enrutador D-Link G403C con Red AkiMovil



Fuente: Autoría

Por el contrario, la operadora Claro demostró ser la opción técnicamente idónea. Aunque presentó una potencia de señal de recepción ligeramente menor (-96 dBm), sus métricas de calidad de canal (RSRQ y SINR) fueron positivas, traducándose en un enlace estable y con menor ruido electromagnético. Adicionalmente, como se observa en la **Figura 61**, la interfaz confirma la asignación de una IP de rango CGNAT (100.94.132.38), lo cual valida el aislamiento perimetral de la red frente a intentos de acceso externo no autorizados.

Figura 61. Conexión Estable y Parámetros Operativos en la Red Claro

Fuente: Autoría propia

Para consolidar la justificación técnica de la operadora seleccionada, la **Tabla 24** expone la comparativa exacta de los parámetros de radiofrecuencia (RF) extraídos de ambas redes operando en el sitio de implementación.

Tabla 24.

Análisis Comparativo de Parámetros RF LTE en el Sector San José

Parámetro LTE	AkiMovil (Movistar)	Claro EC	Interpretación Técnica
Banda	2 (1900 MHz)	4 (AWS 1700/2100 MHz)	- Claro utiliza una banda con un espectro más limpio y sin saturación en este sector.
RSRP	-89 dBm	-96 dBm	Ambas operadoras presentan niveles de potencia viables en borde de celda.
RSRQ	-15 dB	-11 dB	Claro presenta menor interferencia co-canal (valor más cercano a cero).
SINR	-8 dB	6 dB	Claro ofrece una relación señal a ruido positiva y estable (diferencia crítica de 14 dB a favor).

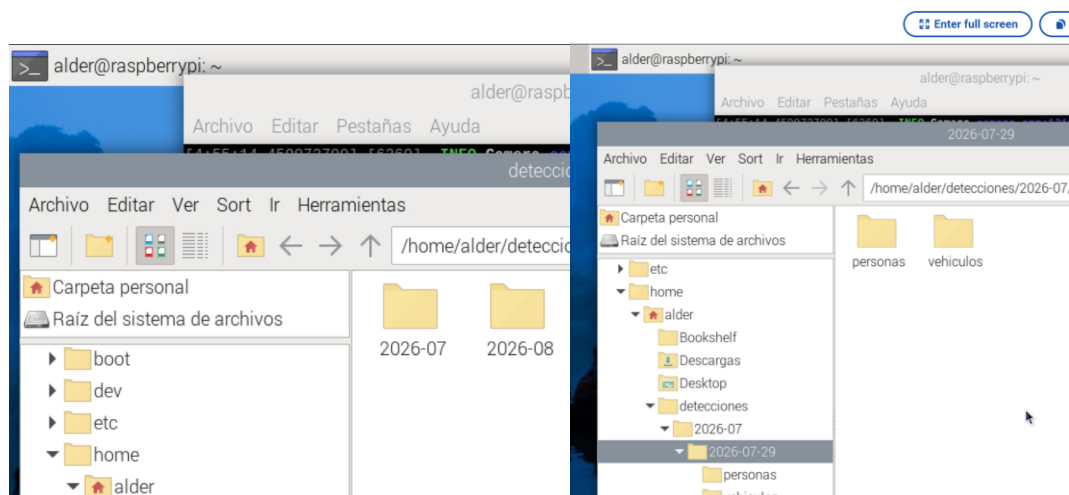
Fuente: Autoría propia

Gracias a la estabilidad comprobada de este canal, el sistema logró transmitir las alertas de intrusión (fotografía procesada, clasificación del objeto y marca de tiempo) hacia Telegram sin pérdida de paquetes. El tiempo de respuesta del sistema autónomo promedió menos de 10 segundos desde la activación del sensor físico hasta la recepción del mensaje en el dispositivo móvil del administrador.

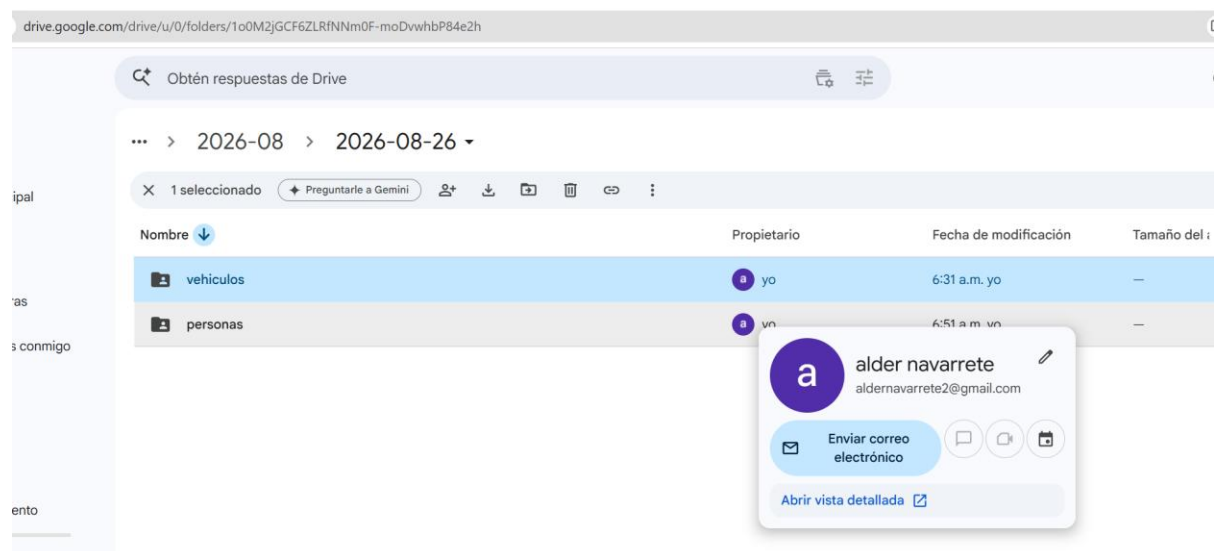
4.4.5. Prueba del Flujo de Almacenamiento Local y en la nube

Durante esta fase, se verificó la integridad del sistema de archivos tanto en la memoria microSD del nodo como en el repositorio remoto. A nivel local, como se evidencia en la Figura 62, se comprobó que el algoritmo genera correctamente las rutas de guardado dinámicas (directorios /vehiculos y /personas) ante cada evento, almacenando las evidencias fotográficas en formato JPG. De manera simultánea, los archivos de texto plano (.txt) registran cronológicamente las matrículas extraídas por el sistema OCR, lo que garantiza un respaldo físico ante posibles caídas temporales de la red celular.

Por otro lado, para asegurar la disponibilidad continua de la información, se validó la ejecución de la sincronización automatizada hacia la nube. La Figura 63 demuestra cómo el sistema logró replicar exitosamente los directorios locales, permitiendo que los datos queden resguardados fuera del hardware físico y listos para alimentar el panel de control web.

Figura 62. Almacenamiento Local

Fuente: Autoría propia

Figura 63. Respaldo en la nube

Autoría propia

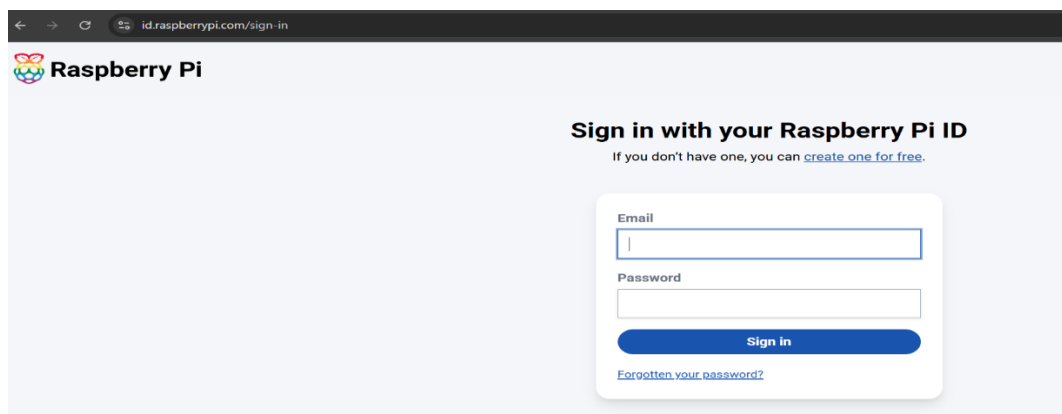
4.4.6. Seguridad de la Conexión y Privacidad

Para proteger el nodo de procesamiento contra accesos no autorizados, el diseño de red aprovechó la arquitectura inherente del proveedor de servicios LTE (Claro EC). Como se

evidenció en la Sección 4.4.4, el enrutador opera bajo un esquema (CGNAT). Esta configuración asigna una dirección IP privada a la interfaz WAN, actuando como un cortafuegos natural que bloquea cualquier intento de conexión entrante (como escaneos de puertos o ataques de fuerza bruta por SSH) originado desde el exterior de la red.

La administración del nodo se restringió exclusivamente a dos métodos seguros. El primer método es el acceso local vía SSH, disponible únicamente para usuarios autenticados dentro de la red WLAN de la capa física (SSID: *RedMonitoreo*). El segundo método es la administración remota a través del servicio oficial *Raspberry Pi Connect*. Como se muestra en la Figura 64, este servicio requiere autenticación mediante credenciales encriptadas y opera estableciendo túneles seguros de salida (WebRTC), evadiendo la necesidad de abrir puertos expuestos al internet público.

Figura 64. Portal de Autenticación Segura para la Administración Remota del Nodo



Fuente: Autoría propia.

Adicionalmente, se garantizó la privacidad de los datos recopilados (fotografías de vehículos y personas). La transmisión de evidencias hacia la API de Telegram se realiza nativamente mediante peticiones seguras (HTTPS). Para el almacenamiento de respaldo a largo plazo, se implementó la herramienta rclone, cuyo proceso de despliegue se evidencia en la **Figura 65**. Esta utilidad sincroniza las capturas locales con Google Drive utilizando protocolos

de cifrado TLS, asegurando que el flujo de información se mantenga confidencial durante su tránsito por la red celular.

Figura 65. Despliegue del Agente de Sincronización Segura en el Nodo de Borde



```

alder@raspberrypi: ~
Archivo Editar Pestañas Ayuda

alder@raspberrypi:~$ sudo apt update
sudo apt install rclone
Obj:1 http://deb.debian.org/debian bookworm InRelease
Des:2 http://deb.debian.org/debian-security bookworm-security InRelease [34,8 k
B]
Des:3 http://deb.debian.org/debian bookworm-updates InRelease [55,4 kB]
Obj:4 http://archive.raspberrypi.com/debian bookworm InRelease
Descargados 90,2 kB en 1s (80,4 kB/s)
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Se pueden actualizar 176 paquetes. Ejecute «apt list --upgradable» para verlos.
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Los paquetes indicados a continuación se instalaron de forma automática y ya no
son necesarios.
avahi-utils chromium-browser chromium-browser-l10n
chromium-codecs-ffmpeg-extra edid-decode gir1.2-handy-1
gir1.2-packagekitlib-1.0 gir1.2-polkit-1.0 libbasicusageenvironment1
libcamera0.2 libgroupsock8 liblivemedia77 libqt5qmlworkerscript5
libqt5quickcontrols2-5 libqt5quicktemplates2-5 libwpe-1.0-1
libwpebackend-fdo-1.0-1 lxplug-network python3-gi-cairo python3-v4l2
qml-module-qtgraphicaleffects qml-module-qtquick-controls2
qml-module-qtquick-layouts qml-module-qtquick-templates2
qml-module-qtquick-window2 qml-module-qtquick2
Utilice «sudo apt autoremove» para eliminarlos.
Se instalarán los siguientes paquetes NUEVOS:
rclone
0 actualizados, 1 nuevos se instalarán, 0 para eliminar y 176 no actualizados.
Se necesita descargar 13,0 MB de archivos.
Se utilizarán 57,2 MB de espacio de disco adicional después de esta operación.
Des:1 http://deb.debian.org/debian bookworm/main arm64 rclone arm64 1.60.1+dfsg
-2+b5 [13,0 MB]
Descargados 13,0 MB en 2s (7.486 kB/s)
Seleccionando el paquete rclone previamente no seleccionado.
(Leyendo la base de datos ... 151443 ficheros o directorios instalados actualme
n
te.)
Preparando para desempaquetar .../rclone_1.60.1+dfsg-2+b5_arm64.deb ...
Desempaquetando rclone (1.60.1+dfsg-2+b5) ...
Configurando rclone (1.60.1+dfsg-2+b5) ...

```

Fuente: Autoría propia

Por último, para el acceso y visualización remota de las evidencias a través de la interfaz web desplegada en la nube, se implementó una arquitectura de software desacoplada que separa el almacenamiento del panel de control. El acceso a la plataforma web se encuentra protegido mediante una pasarela de autenticación de credenciales cifradas (usuario y contraseña) y el manejo de variables de entorno protegidas (*Secrets*) en el servidor, garantizando que únicamente el personal autorizado pueda visualizar la telemetría y el árbol de directorios históricos, tal como se observa en la Figura 66.

Figura 66. Panel de Acceso Restringido y Autenticación Segura para el Monitoreo Web



Fuente: Autoría propia

4.5. Etapa de Evaluación de Resultados

Finalizada la implementación, se procedió a evaluar la eficacia de la Inteligencia Artificial y compararla con el método tradicional de vigilancia empleado por los agricultores en el sector San José.

4.5.1. Análisis de la Seguridad Tradicional

Antes de la implementación del nodo IoT, el esquema de seguridad dependía de rondas físicas esporádicas (casi nulas). Durante las noches, el control visual era nulo, dejando la vía principal vulnerable. No existía una bitácora confiable de ingresos y los tiempos de respuesta ante un evento anómalo tomaban horas o días, haciendo imposible una acción disuasiva.

4.5.2. Análisis del Sistema de Seguridad Automático

La inserción del nodo de borde transformó radicalmente el monitoreo del sector, reduciendo los tiempos de respuesta a escasos segundos mediante notificaciones *Push* inmediatas y automatizando la trazabilidad con evidencias fotográficas.

Para cuantificar la confiabilidad del sistema de visión artificial, se realizaron pruebas de eventos en un entorno real (intrusiones controladas de vehículos, personas y movimiento de factores ambientales como follaje). Se evaluó la capacidad del modelo para clasificar correctamente el objeto (Verdadero Positivo) y la eficiencia de su filtro contra falsas alarmas provocadas por el viento (Falso Positivo).

Tabla 25.

Evaluación de eventos de detección

N°	Condición del Evento Físico	Respuesta del Modelo (YOLOv8n / EasyOCR)	Evaluación Técnica
1	Ingreso de Vehículo (Día)	Detección y clasificación correcta.	Exitoso. Alta precisión geométrica.
2	Lectura de Placa (Día)	Extracción de caracteres.	Condicionado a placas limpias y sin opacidad.
3	Ingreso de Vehículo (Noche)	Vehículo detectado (YOLO). Reflejo en la placa (OCR).	Detección exitosa. OCR requiere mayor resolución óptica.
4	Ingreso de Persona (Día/Noche)	Detección y clasificación correcta.	Exitoso.
5	Movimiento de ramas por viento	Sensor PIR activa cámara; la IA descarta el envío.	Exitoso (Filtro de IA efectivo).
6	Vehículo a alta velocidad (Polvo)	Vehículo detectado (YOLO). Texto obstruido por polvo (OCR).	Detección exitosa. OCR condicionado a visibilidad ambiental.

Fuente: Autoría propia

A partir de los resultados expuestos en la Tabla 25, se comprueba el éxito del diseño arquitectónico y se definen las condiciones operativas ideales para el subsistema de lectura de caracteres.

En primer lugar, respecto a la infraestructura de red, el éxito de las alertas inmediatas radica en que el sistema prescinde deliberadamente de la transmisión de video continuo hacia el usuario. En un entorno rural dependiente de redes móviles (LTE), el envío constante de un flujo de video generaría una latencia crítica, saturaría el ancho de banda y retrasaría las notificaciones de emergencia. Al aplicar el paradigma de *Edge Computing*, procesando la imagen de forma local en la Raspberry Pi 5 y transmitiendo únicamente la evidencia fotográfica final, se asegura la recepción del mensaje en menos de 10 segundos.

En segundo lugar, el modelo YOLOv8n demostró una robustez sobresaliente. Como se evidencia en las Figuras 67 y 68, el algoritmo es capaz de enmarcar y clasificar correctamente vehículos de maquinaria agrícola (tractores) durante el día, así como vehículos de carga pesada durante la noche, filtrando además de manera exitosa los falsos positivos generados por el viento.

Figura 67. Detección Diurna de Maquinaria Agrícola mediante Visión Artificial



Fuente: Autoría propia

Figura 68. Detección Nocturna de Vehículo de Carga bajo Iluminación Artificial



Fuente: Autoría propia

Finalmente, respecto a la extracción de texto, se determinó que el motor de reconocimiento óptico de caracteres (EasyOCR) requiere condiciones específicas de hardware y entorno para operar al 100% de su capacidad. Durante la noche (Evento 3), la incidencia directa del reflector LED sobre la superficie retroreflectiva de la matrícula genera destellos que saturan el lente actual. Asimismo, el desplazamiento rápido de vehículos sobre la vía de tierra (Evento 6) levanta polución extrema que vuelve opaca la superficie de la placa o genera desenfoque por movimiento.

4.6. Costos del Sistema

Para determinar la viabilidad económica del proyecto, se realizó un desglose de todos los componentes, herramientas e infraestructura requerida para desplegar un nodo autónomo de seguridad.

4.6.1. Costo de Hardware y Procesamiento

En la Tabla 25 se detallan los costos asociados a la adquisición de los componentes electrónicos y lógicos fundamentales para el funcionamiento del nodo *Edge AI*. Este rubro

incluye el microprocesador principal (Raspberry Pi 5), los módulos de captura de imagen, sensores de movimiento y el hardware de comunicación LTE, los cuales conforman el núcleo inteligente encargado de la detección y procesamiento de intrusiones en la plantación.

Tabla 26 .

Costos de Hardware y Procesamiento

Equipos / Componentes	Cantidad	Precio (USD)	Unitario	Subtotal (USD)
Computador Monoplaca Raspberry Pi 5 (8GB)	1	120.00		120.00
Módulo de Cámara Arducam IMX477 12MP	1	80.00		80.00
Router LTE D-Link G403C	1	55.00		55.00
Módulo Step-Down (Buck Converter 12V-5V)	1	10.00		10.00
Módulo Relé 5V de 2 Canales	1	5.00		5.00
Sensor de Movimiento PIR HC-SR501	1	4.00		4.00
Memoria MicroSD 128 GB Extreme Pro	1	15.00		15.00
Total Hardware Lógico				\$ 289.00

Fuente: Autoría propia

4.6.2. Costo de Infraestructura y Energía

Para asegurar la operatividad autónoma y la protección del nodo frente a las condiciones climáticas del entorno rural, se requirió una inversión en infraestructura física y generación eléctrica. La Tabla 26 desglosa los valores correspondientes al sistema de energía solar *off-grid*

(panel monocristalino, batería de gel y controlador), así como los elementos de confinamiento (gabinete IP65) y la estructura metálica de soporte elevada.

Tabla 27.

Costo de Infraestructura y Energía

Equipos / Estructura	Cantidad	Precio (USD)	Unitario	Subtotal (USD)
Panel Solar Monocristalino 100W	1	95		95
Batería de Gel Ciclo Profundo 12V 50Ah	1	85		85
Controlador de Carga Solar PWM	1	20		20
Reflector LED 12V 20W (Exterior)	1	15		15
Gabinete IP65 estanco transparente	1	18		18
Estructura metálica (Poste y anclajes)	1	60		60
Cableado, conectores y misceláneos	1	25		25
Total Infraestructura				318.00

Fuente: Autoría.

4.6.3. Costos de Ingeniería y Software

El desarrollo del sistema de monitoreo se fundamentó en la integración de tecnologías libres y plataformas gratuitas, lo que permitió optimizar significativamente el presupuesto. Como se evidencia en la Tabla 27, el uso de herramientas de código abierto (*Open Source*) para la visión artificial, junto con la utilización de APIs de distribución gratuita para las alertas y el panel web, redujo los costos de licencias de software a cero. De igual manera, el diseño y entrenamiento de la red neuronal fueron asumidos íntegramente por el autor como parte del presente trabajo de titulación.

Tabla 28.

Costos de Ingeniería y Software

Descripción	Horas	/ Precio	Subtotal
	Licencias	(USD)	(USD)
Desarrollo, entrenamiento IA	Global	0	0
Licencias de Software (Python, YOLO, EasyOCR)	Open Source	0	0
API de Telegram (Notificaciones)	Gratuito	0	0
Dashboard Web (Google Sites)	Gratuito	0	0
Total Ingeniería y Software			\$ 0

Fuente: Autoría propia

4.6.4. Costo Global del Sistema

La Tabla 28 presenta el resumen consolidado de la inversión de capital requerida para la implementación total del prototipo de seguridad. Al sumar los rubros de hardware lógico e infraestructura energética, se obtiene el valor final del sistema desplegado en el sector San José. Este total demuestra la viabilidad económica y la asequibilidad de implementar una solución tecnológica de alta precisión utilizando arquitectura IoT .

Tabla 29.

Costo Global del Sistema

Descripción del Rubro	Subtotal (USD)
Costos de Hardware Lógico	\$ 289.00
Costos de Infraestructura y Energía	\$ 318.00
Costos de Ingeniería y Software	\$ 0
INVERSIÓN TOTAL DEL SISTEMA	\$ 607

Fuente: Autoría.

4.6.5. Costos Operativos y de Mantenimiento

A diferencia de la inversión inicial en hardware e infraestructura, el sistema requiere un presupuesto operativo mínimo para garantizar la transmisión continua de datos y el acceso a servicios avanzados de inteligencia artificial. Estos costos fijos recurrentes aseguran que el nodo mantenga su conexión remota y su capacidad de análisis en la nube cuando sea requerido.

Para el cálculo operativo, se ha proyectado el gasto en un periodo anual (12 meses), considerando el plan de datos móviles para el enrutador LTE y la suscripción a los servicios de Almacenamiento.

Tabla 30.

Costos Operativos y de Mantenimiento

Descripción del Servicio	Costo Mensual (USD)	Costo Anual (USD)
Plan tarifario prepago LTE (Proveedor: Claro)	10.50	126.00
Servicios y Almacenamiento drive 100 GB	1.99	24.00
Mantenimiento preventivo (limpieza de panel y lente)	1.00	12.00
Total Costo Operativo Anual		\$ 182.00

Fuente: Autoría propia

4.7. Beneficios del Sistema

La implementación del sistema de monitoreo en el sector San José aporta beneficios sustanciales a nivel técnico y organizacional:

- **Vigilancia 24/7 sin intervención humana:** La automatización reduce los costos de contratación de personal de guardia y elimina los errores asociados a la fatiga visual.
- **Eficiencia Energética y de Datos:** El uso de sensores PIR combinados con *Edge AI* asegura que el sistema solo consuma energía y datos de red cuando existe una amenaza real.
- **Respaldo Visual e Histórico:** La extracción de placas vehiculares mediante OCR y el almacenamiento de fotografías proporciona pruebas visuales contundentes a los agricultores para reportar robos a las autoridades.
- **Accesibilidad Tecnológica:** El uso de Telegram como interfaz de usuario democratiza el acceso a la tecnología, permitiendo a los agricultores monitorear sus tierras desde una aplicación que ya saben usar, sin curva de aprendizaje.

CONCLUSIONES

- Al evaluar la efectividad del sistema mediante pruebas de campo, se concluye que la capacidad de reacción en tiempo real se ve limitada por la inestabilidad de la red celular LTE. Aunque el procesamiento de video local es inmediato, los microcortes de señal en San José retrasan la entrega de las alertas fotográficas.
- Se evidenció que el diseño actual del prototipo presenta una dependencia crítica hacia la infraestructura de telefonía móvil convencional. Esta vulnerabilidad externa disminuye la autonomía comunicacional del sistema de seguridad ante fallos del proveedor de internet en el entorno rural.
- Los resultados obtenidos permiten concluir que la arquitectura IoT basada en procesamiento en el borde (*Edge Computing*) es plenamente viable y altamente eficiente para la seguridad en entornos agrícolas como el sector San José.
- La integración de la Raspberry Pi 5 logró ejecutar los modelos de visión artificial (YOLOv8n y EasyOCR) con una tasa de acierto aceptable, discriminando exitosamente falsas alarmas provocadas por la naturaleza. Asimismo, el diseño energético fotovoltaico demostró ser capaz de sostener las exigencias de hardware de manera perpetua.
- En conjunto, el sistema superó las limitaciones de la vigilancia tradicional, entregando a los productores una herramienta automatizada, asequible y de respuesta casi instantánea frente a intrusiones vehiculares o peatonales.

RECOMENDACIONES

- Para cumplir el objetivo de generar alertas en tiempo real sin interrupciones, se recomienda migrar la conexión del nodo hacia sistemas de internet satelital de órbita baja (como Starlink), garantizando alta velocidad y baja latencia sin importar las limitaciones topográficas del sector.
- Se aconseja complementar el diseño del prototipo con una red de respaldo de radiofrecuencia de largo alcance (LoRaWAN) para asegurar que en caso de que la red celular principal colapse o sufra bloqueos, las notificaciones de texto de riesgo crítico serán enviadas inmediatamente.
- Se aconseja ejecutar un mantenimiento preventivo trimestral que contemplará la limpieza del lente de la cámara Arducam y la superficie del panel solar, ya que la acumulación de polvo agrícola podrá afectar a la tasa de acierto del algoritmo OCR y a la eficiencia de carga de la batería.
- Es aconsejable mantener la vegetación circundante podada cerca del ángulo de visión del nodo para evitar que el sensor PIR despierte el sistema innecesariamente ante el movimiento excesivo de ramas calientes por el sol.
- Dada la naturaleza modular del sistema IoT, se sugiere a futuro escalar la red replicando nodos de menor costo en vías secundarias de la plantación, interconectándolos mediante topologías inalámbricas de largo alcance.

REFERENCIAS BIBLIOGRÁFICAS

- Akyildiz, I. F. (2005). *A survey on wireless mesh networks*. *IEEE Communications Magazine*.
Obtenido de <https://doi.org/10.1109/MCOM.2005.1509968>
- Al-Fuqaha, A. G. (2015). *IEEE Communications Surveys & Tutorials*, . Obtenido de IEEE Communications Surveys & Tutorials, : <https://ieeexplore.ieee.org/document/7123563>
- Ayaz, M. A.-U. (2018). *Ad Hoc Networks*, . Obtenido de Ad Hoc Networks, : <https://www.sciencedirect.com/science/article/pii/S1570870518302740>
- Baccarini, A. F. (2023). Smart Agriculture: Technologies, Practices, and Future Directions. En *International Journal of Environment and Climate Change*. India.
- Borgia, E. (2013). *Journal of Internet Services and Applications*, . Obtenido de Journal of Internet Services and Applications, : <https://jisajournal.springeropen.com/articles/10.1186/1869-0238-4-5>
- Borgini, J. (24 de marzo de 2023). *5 pruebas de dispositivos IoT que deberías realizar*. Obtenido de <https://www.techtarget.com/iotagenda/tip/5-IoT-device-tests-you-should-be-running>
- Botta, A. D. (2016). *Future Generation Computer Systems*,. Obtenido de Future Generation Computer Systems: <https://www.sciencedirect.com/science/article/pii/S0167739X15002760>
- Bouabdellah, B. &. (2019). *International Journal of Distributed Sensor Networks*, . Obtenido de International Journal of Distributed Sensor Networks, : <https://journals.sagepub.com/doi/10.1177/1550147719832296>
- Chen, M. M. (2014). *IEEE Internet of Things Journal*,. Obtenido de IEEE Internet of Things Journal,: <https://ieeexplore.ieee.org/document/6781597>
- Chen, Z. &. (2019). *IEEE Sensors Journal*,. Obtenido de IEEE Sensors Journal,: <https://ieeexplore.ieee.org/document/8627462>
- Doe, J. &. (2017). *Computers and Electronics in Agriculture*. Obtenido de Computers and Electronics in Agriculture: <https://doi.org/10.1016/j.compag.2017.06.008>
- El Universo. (25 de 04 de 2024). *En Carchi queman a adolescente por hurtar aguacates en una propiedad privada*. Obtenido de <https://www.eluniverso.com/noticias/ecuador/en-carchi-queman-a-adolescente-por-hurtar-aguacates-en-una-propiedad-privada-nota/>
- González, A. M. (2023). *Electronic Journal of SADIO*. Obtenido de Electronic Journal of SADIO: <https://revistas.unlp.edu.ar/ejs/article/view/17708>

- Gubbi, J. (2013). *Internet of Things (IoT): A vision, architectural elements, and future directions*. Obtenido de <https://doi.org/10.1016/j.future.2013.01.010>
- Herrera, J. V. (01 de Julio de 2022). *El aguacate ecuatoriano es un 'boom' en el mercado extranjero | Yara Ecuador*. Obtenido de <https://www.yara.com.ec/noticias-y-eventos/noticias/el-aguacate-ecuatoriano-es-un-boom-en-el-mercado-extranjero2/>
- ITU. (2021). Obtenido de https://www-itu-int.translate.goog/itu-d/reports/statistics/facts-figures-2021/?_x_tr_sl=en&_x_tr_tl=es&_x_tr_hl=es&_x_tr_pto=tc
- Jawad, H. M. (2017). *Fault Detection of Bearing Systems through EEMD and Optimization Algorithm*. Obtenido de <https://www.mdpi.com/1424-8220/17/11/2477>
- Jawad, H. M. (2017). *Learning Traffic as Images: A Deep Convolutional Neural Network for Large-Scale Transportation Network Speed Prediction*. Obtenido de <https://www.mdpi.com/1424-8220/17/4/818>
- Khan, R. K. (2014). *Journal of Network and Computer Applications*,. Obtenido de Journal of Network and Computer Applications,: https://www.researchgate.net/publication/261311447_Future_Internet_The_Internet_of_Things_Architecture_Possible_Applications_and_Key_Challenges
- Khan, R. M. (2015). *IEEE Communications Surveys & Tutorials*,. Obtenido de IEEE Communications Surveys & Tutorials,: <https://ieeexplore.ieee.org/document/6863574>
- Köksal, Ö. &. (2019). *Precision Agriculture*. Obtenido de Precision Agriculture: <https://doi.org/10.1007/s11119-018-09624-8>
- Lan, T. C. (2020). *IEEE Access*, 8, . Obtenido de IEEE Access, 8,: <https://ieeexplore.ieee.org/document/9145108>
- Li, S. X. (2015). *IEEE Communications Surveys & Tutorials*. Obtenido de IEEE Communications Surveys & Tutorials: <https://ieeexplore.ieee.org/document/7123563>
- Liu, X. Z. (2021). *Computers and Electronics in Agriculture*. Obtenido de Computers and Electronics in Agriculture: <https://doi.org/10.1016/j.compag.2021.106352>
- Maanak Gupta, S. K. (2020). *Security and Privacy in Smart Farming: Challenges and Opportunities*. Obtenido de 10.1109/ACCESS.2020.2975142
- Mehta, G. (2021). Sistema inteligente de monitoreo agrícola con energía solar que utiliza dispositivos del Internet de las cosas. En *IA e IoT en energías renovables*.
- Microsegur. (01 de Noviembre de 2022). *Cómo evitar el robo en una plantación de aguacates - Microsegur Blog*. Obtenido de <https://microsegur.com/como-evitar-el-robo-en-una-plantacion-de-aguacates/>

- Ministerio de Agricultura y Ganadería. (24 de 11 de 2020). *Aguacate de Mira, en el Carchi, conquista el mundo*. Obtenido de <https://www.agricultura.gob.ec/aguacate-de-mira-en-el-carchi-conquista-el-mundo/>
- Mundoagro. (10 de febrero de 2017). *Robos en el campo, una cruda realidad*. Obtenido de <https://mundoagro.cl/robos-en-el-campo-una-cruda-realidad/>
- Muthuraj, M. &. (2016). *Journal of Sensors*,. Obtenido de Journal of Sensors,: <https://www.hindawi.com/journals/js/2016/7949275/>
- ODS. (2015). *17 Objetivos de Desarrollo Sostenible (ODS)*. Obtenido de <https://www.pactomundial.org/que-puedes-hacer-tu/ods/>
- Palacios, S. (2025). *The Internet of Things in agriculture*. Obtenido de <https://polodelconocimiento.com/ojs/index.php/es/article/view/8914/htmlThe%20Internet%20of%20Things%20in%20agriculture>
- PP RAY, F. R. (Julio de 2016). *A survey on Internet of Things architectures*. Obtenido de <https://doi.org/10.1016/j.jksuci.2016.10.003>
- Ray, P. P. (2018). *Journal of Ambient Intelligence and Humanized Computing*,. Obtenido de Journal of Ambient Intelligence and Humanized Computing,: <https://link.springer.com/article/10.1007/s12652-017-0467-1>
- Raza, U. K. (2017). *IEEE Communications Surveys & Tutorials*,. Obtenido de IEEE Communications Surveys & Tutorials,: <https://arxiv.org/abs/1606.07360>
- Reina Jurado, J. A. (2023). *Diseño de una red de videovigilancia IOT para la prevención y control delincriminal basada en Inteligencia Artificial y Machine Learning en zonas rurales del cantón Babahoyo*. Obtenido de <https://dspace.utb.edu.ec/handle/49000/17930>
- Rodríguez, L. M. (2018). *Diseño e implementación de un software de reconocimiento de*. Obtenido de <https://repository.unad.edu.co/bitstream/handle/10596/21319/1094164147.pdf?sequence=1&isAllowed=y>
- Sahoo, S. e. (2018). *IEEE Access*,. Obtenido de IEEE Access,: <https://ieeexplore.ieee.org/document/8353235>
- Secretaría Nacional de Planificación. (2021). *Plan de Creación de Oportunidades 2021-2025*. Obtenido de <https://www.planificacion.gob.ec/plan-de-creacion-de-oportunidades-2021-2025/>
- Sharma, A. &. (2019). *Sensors*, 19(14), 3207 . Obtenido de Sensors, 19(14), 3207: <https://www.mdpi.com/1424-8220/19/14/3207>

- Sicari, S. R.-P. (2015). *Computer Networks*,. Obtenido de Computer Networks, :
<https://www.sciencedirect.com/science/article/pii/S1389128614004461>
- Sommerville, I. (2014). *Software Engineering (9th ed.)*. Mexico: Pearson Education.
- Szeliski, R. (2011). *Computer Vision*. Obtenido de
<https://link.springer.com/book/10.1007/978-1-84882-935-0>

ANEXOS

Código Fuente del Nodo de Edge Computing (monitoreo_yolo5.py)

El siguiente script en Python constituye la lógica principal ejecutada en la Raspberry Pi 5. Este código integra la inicialización de periféricos físicos, el bucle de detección mediante el sensor PIR, la captura de imágenes, y la inferencia local de los modelos de inteligencia artificial YOLOv8n y EasyOCR.

```
import cv2 #procesamiento de imagen

import os # crear archivos

import time # maneja reloj para tareas

import requests # peticiones tipo post get

import easyocr # motor de reconocimiento

import threading # permite multitarea

import subprocess #

from datetime import datetime # fecha

from ultralytics import YOLO #ia

from picamera2 import Picamera2

from gpiozero import MotionSensor, OutputDevice

# =====

# CONFIGURACIÓN GENERAL

# =====

TOKEN = "8626887025:AAFdYomeBzMKQyYkmnFVxpKtaXqVtfajP0g"

CHAT_ID = "-5575877445 " # grupo-5575877445 individual 5682386608

BASE_DIR = "detecciones"
```

```

TIEMPO_SYNC_MINUTOS = 10

# --- CONFIGURACIÓN DE CÁMARA Y RÁFAGA ---

ROTACION = cv2.ROTATE_180

NUM_FOTOS_RAFA = 3

TIEMPO_MINIMO_LUZ = 5

# =====

# CONFIGURACIÓN DE HARDWARE

# =====

print("Iniciando Hardware...")

# [SENSOR PIR]: Se declara el pin 17 para el sensor que despierta el sistema.

pir = MotionSensor(17)

# [RELÉ / FOCO]: Se declara el pin 27 para el reflector LED. Inicia apagado (False).

foco = OutputDevice(27, active_high=False, initial_value=False)

# =====

# INICIALIZACIÓN DE MODELOS E IA

# =====

print("Cargando Inteligencia Artificial (YOLOv8 y EasyOCR)...")

model = YOLO('yolov8n.pt')

reader = easyocr.Reader(['es'])

# [INICIALIZACIÓN DE CÁMARA]: Se levanta la cámara usando el bus MIPI CSI-2.

# El formato RGB888 evita que la CPU pierda tiempo convirtiendo colores para la IA.

```



```

print("Iniciando Cámara...")

picam2 = Picamera2()

config = picam2.create_preview_configuration(main={"format": "RGB888", "size": (640,
480)})

picam2.configure(config)

picam2.start()


# =====

# FUNCIONES AUXILIARES

# =====

def obtener_directorios():

    ahora = datetime.now()

    mes = ahora.strftime("%Y-%m")

    dia = ahora.strftime("%Y-%m-%d")

    dir_personas = os.path.join(BASE_DIR, mes, dia, "personas")

    dir_vehiculos = os.path.join(BASE_DIR, mes, dia, "vehiculos")

    os.makedirs(dir_personas, exist_ok=True)

    os.makedirs(dir_vehiculos, exist_ok=True)

    return dia, dir_personas, dir_vehiculos

def enviar_telegram(path, msg):

```

```

# [ENVÍO A TELEGRAM]: Petición HTTP para enviar la evidencia (foto) y el texto de
alerta.

url = f"https://api.telegram.org/bot{TOKEN}/sendPhoto"

try:

    with open(path, 'rb') as f:

        requests.post(url, data={'chat_id': CHAT_ID, 'caption': msg}, files={'photo': f},
timeout=10)

    print("📷 Notificación de Telegram enviada.")

except Exception as e:

    print(f"⚠️ Error Telegram: {e}")

def es_denoche():

    hora_actual = datetime.now().hour

    return hora_actual >= 18 or hora_actual < 6

def sincronizar_drive():

    # [CONEXIÓN AL DRIVE]: Esta función corre en segundo plano continuamente.

    while True:

        time.sleep(TIEMPO_SYNC_MINUTOS * 60)

        print(f"\n[☁️ Sincronización] Subiendo archivos a Google Drive...")

        try:

            # Invoca a rclone (sistema operativo Linux) para empujar los archivos a la nube.

            subprocess.run(["rclone", "copy", BASE_DIR, "gdrive_tesis:detecciones"],
check=True)

            print("[✅ Sincronización] Respaldo en la nube completado con éxito.")

```

```

except Exception as e:

    print(f"[❌ Sincronización] Error al subir a Drive: {e}")

# =====

# BUCLE PRINCIPAL DEL SISTEMA

# =====

# [HILOS / THREADING]: Lanza la sincronización a Drive en paralelo para no congelar la
vigilancia.

hilo_drive = threading.Thread(target=sincronizar_drive, daemon=True)

hilo_drive.start()

print("\n--- Sistema San José Activo y Monitoreando ---")

try:

    while True:

        # [DESPERTAR]: El sistema duerme consumiendo lo mínimo hasta que detecta
calor/movimiento.

        pir.wait_for_motion()

        print("\n🔔 ¡Movimiento detectado! Iniciando ráfaga...")

        necesita_luz = es_denoche()

        tiempo_inicio_rafaga = time.time()

        # [ENCIENDE EL RELÉ]: Si es de noche, manda la señal al pin 27 para prender el foco
de 12V.

        if necesita_luz:

            print("🌙 Es de noche. Encendiendo reflector...")

```

```

foco.on()

time.sleep(1)

dia_actual, ruta_personas, ruta_vehiculos = obtener_directorios()

placas_vistas_ahorita = set()

persona_vista_ahorita = False

detectado = False

for i in range(NUM_FOTOS_RAFAGA):

    print(f"📷 Capturando foto {i+1} de {NUM_FOTOS_RAFAGA}...")

    img_bgr_cruda = picam2.capture_array()

    # 1. Rotamos la imagen cruda

    img_bgr = cv2.rotate(img_bgr_cruda, ROTACION)

    # 2. Convertimos a RGB puro

    img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)

    # [INFERENCIA IA]: Se evalúa la imagen en YOLO con un 40% de confianza
mínima.

    results = model(img_rgb, conf=0.4, verbose=False)

    for r in results:

        boxes = r.boxes

        # --- PROCESAMIENTO DE VEHÍCULOS (Con mejoras OCR) ---

        # [DECISIÓN 1]: YOLO filtra y decide si en la foto hay algún tipo de
vehículo.

        vehiculos = [b for b in boxes if model.names[int(b.cls[0])] in ['car', 'truck',
'motorcycle', 'bus']]

```

```

if vehiculos:

    detectado = True

    vehiculo_principal = max(vehiculos, key=lambda b: (b.xyxy[0][2] -
b.xyxy[0][0]) * (b.xyxy[0][3] - b.xyxy[0][1]))

    x1, y1, x2, y2 = map(int, vehiculo_principal.xyxy[0])

    label = model.names[int(vehiculo_principal.cls[0])]

    ts = datetime.now().strftime("%H%M%S_%f")[:10]

    fecha_hora = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

    # 1. Recorte Inteligente (70% superior y sin 20% laterales)

    ancho_carro = x2 - x1

    margen_lateral = int(ancho_carro * 0.20)

    x1_crop = x1 + margen_lateral

    x2_crop = x2 - margen_lateral

    y_inicio = y1 + int((y2 - y1) * 0.70)

    placa_crop = img_rgb[y_inicio:y2, x1_crop:x2_crop]

    # 2. Zoom digital Cúbico (2.5x)

    if placa_crop.size != 0:

        placa_crop = cv2.resize(placa_crop, None, fx=2.5, fy=2.5,
interpolation=cv2.INTER_CUBIC)

        # 3. Escala de grises, corrección de luz (CLAHE) y Alto Contraste

        placa_gray = cv2.cvtColor(placa_crop, cv2.COLOR_RGB2GRAY)

        clahe = cv2.createCLAHE(clipLimit=4.0, tileGridSize=(8,8))

        placa_gray = clahe.apply(placa_gray)

        placa_gray = cv2.convertScaleAbs(placa_gray, alpha=2.5, beta=-15)

        placa_gray = cv2.bilateralFilter(placa_gray, 11, 17, 17)

```

```

# 4. Lectura OCR

ocr_res = reader.readtext(placa_gray,
allowlist='ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789-')

textos_validos = []

for res in ocr_res:

    texto = res[1].upper().replace(" ", "")

    if "ECUADOR" not in texto and len(texto) >= 6:

        textos_validos.append(texto)

txt_placa = "-".join(textos_validos) if textos_validos else
"INDETERMINADA"

if txt_placa not in placas_vistas_ahorita:

    f_name = f"Vehiculo_{txt_placa}_{ts}.jpg"

    f_path = os.path.join(ruta_vehiculos, f_name)

    # Dibujar recuadro verde en la foto principal para Telegram

    img_final = img_bgr.copy()

    cv2.rectangle(img_final, (x1, y1), (x2, y2), (0, 255, 0), 3)

    # [ALMACENAMIENTO LOCAL 1]: Guarda la fotografía en la memoria
SD de la placa.

    cv2.imwrite(f_path, img_final)

    txt_log_path = os.path.join(ruta_vehiculos,
f"Registro_Placas_{dia_actual}.txt")

    # [ALMACENAMIENTO LOCAL 2]: Escribe la placa extraída en la
bitácora de texto (CSV).

    with open(txt_log_path, "a") as txt_file:

        txt_file.write(f"[{fecha_hora}] Tipo: {label} | Placa: {txt_placa}\n")

```

```

# Llama a la función que contacta a Telegram

enviar_telegram(f_path, f" 🚗 VEHÍCULO: {label}\n 📷 PLACA:
{txt_placa}\n 🕒 {fecha_hora}")

placas_vistas_ahorita.add(txt_placa)

print(f" 🚗 Vehículo (Placa: {txt_placa}) guardado y enviado.")

# --- PROCESAMIENTO DE PERSONAS ---

# [DECISIÓN 2]: Si YOLO no vio ningún vehículo, entonces busca personas.
if not detectado:

    personas = [b for b in boxes if model.names[int(b.cls[0])] == 'person']

    if personas:

        detectado = True

        if not persona_vista_ahorita:

            ts = datetime.now().strftime("%H%M%S")

            fecha_hora = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

            f_name = f"Persona_{ts}.jpg"

            f_path = os.path.join(ruta_personas, f_name)

            # [ALMACENAMIENTO LOCAL]: Guarda foto de la persona en la SD.

            cv2.imwrite(f_path, img_bgr)

            enviar_telegram(f_path, f" 👤 ALERTA: Persona detectada en San
José\n 🕒 {fecha_hora}")

            persona_vista_ahorita = True

```

```

        print("👤 Persona guardada y enviada.")

    tiempo_transcurrido = time.time() - tiempo_inicio_rafaga

    if necesita_luz:

        tiempo_restante = TIEMPO_MINIMO_LUZ - tiempo_transcurrido

        if tiempo_restante > 0:

            time.sleep(tiempo_restante)

            # [APAGA EL RELÉ]: Corta la energía del foco LED para ahorrar batería tras la
captura.

            foco.off()

            print("💡 Reflector apagado tras iluminar la zona.")

    if not detectado:

        print("🚫 Falsa alarma o sujeto no clasificado por la IA en ninguna foto.")

    print("⌚ Pausa de seguridad (5 seg)...")

    time.sleep(5)
except KeyboardInterrupt:

    print("\n🛑 Finalizando sistema de forma segura...")
finally:

    # [BLOQUE DE SEGURIDAD]: Si el programa se cae por un error,

    # garantiza apagar el foco y liberar la cámara además de mandar una alerta a telegram.

    enviar_telegram(f_path, f"👤 ALERTA: Se detuvo el sistema")

```



```
foco.off()

picam2.stop()
```

Configuración del Servicio de Arranque Automático (Systemd)

Para garantizar la autonomía del sistema frente a cortes de energía o reinicios, se configuró un demonio en el sistema operativo Linux. El siguiente bloque corresponde al archivo de servicio `monitoreo.service`, alojado en el directorio `/etc/systemd/system/`, el cual automatiza el despliegue del entorno virtual y la ejecución del script principal al arrancar el equipo.

[Unit]

Description=Servicio de Seguridad Perimetral San Jose (Edge Computing)

After=network.target

[Service]

Type=simple

User=alder

WorkingDirectory=/home/alder/entorno_tesis

ExecStart=/home/alder/entorno_tesis/bin/python /home/alder/monitoreo_yolo5.py

Restart=always

RestartSec=10

[Install]

WantedBy=multi-user.target

Código Fuente de la Interfaz Web (app.py)

El siguiente script fue desarrollado utilizando el *framework* Streamlit y se encuentra alojado en Streamlit Community Cloud. Este código maneja la autenticación segura, la comunicación directa con la API REST de Google Drive mediante tokens temporales, y la renderización en cascada de la interfaz gráfica de usuario.

```
# =====

# 1. IMPORTACIÓN DE LIBRERÍAS

# =====

import streamlit as st    # El framework principal para crear la página web.

import pandas as pd      # Manejo de datos .

import requests          # Permite hacer peticiones HTTP directas (GET/POST) a la nube.

import io                # Maneja flujos de bytes en la memoria de la página.

from PIL import Image    # Procesa los bytes descargados para convertirlos en una imagen
                           visible.

from google.oauth2 import service_account # Maneja la "Cuenta de Servicio" robot de
                           Google.

import google.auth.transport.requests    # Renueva los tokens de seguridad de Google.

# Configuración básica de la pestaña del navegador (título y emoji).

st.set_page_config(page_title="Seguridad San José", page_icon="🔒", layout="centered")

# El ID único de la carpeta raíz "detecciones" en tu Google Drive.

CARPETA_RAIZ_ID = "1HRA_2wC9sEUHonaW_uCRe9R0YxACPZ6j"
```

```

# =====

# 2. AUTENTICACIÓN SEGURA (CIBERSEGURIDAD)

# =====

def obtener_token():

    # Toma la clave secreta guardada en st.secrets (para que no esté expuesta en GitHub).

    credenciales = service_account.Credentials.from_service_account_info(

        st.secrets["gcp_service_account"],

        scopes=["https://www.googleapis.com/auth/drive.readonly"] # Permiso de SOLO
LECTURA.

    )

    # Solicita a Google Cloud que genere un Token temporal válido para esta sesión.

    auth_request = google.auth.transport.requests.Request()

    credenciales.refresh(auth_request)

    return credenciales.token # Retorna la "llave" temporal cifrada.

# =====

# 3. COMUNICACIÓN CON GOOGLE DRIVE API REST

# =====

def consultar_drive(url, params=None):

    """Realiza peticiones HTTP directas evitando el fallo de SSL sockets."""

    token = obtener_token()

    # Inyecta el Token temporal en la "cabecera" (header) de la petición HTTP.

    headers = {"Authorization": f"Bearer {token}"}

    # Hace una llamada GET a Google Drive.

```

```

response = requests.get(url, headers=headers, params=params)

if response.status_code == 200:

    return response.json() # Si Google responde OK (200), devuelve los datos en formato
JSON.

else:

    return None

def listar_archivos(parent_id):

    # Endpoint oficial de Google Drive para buscar archivos/carpetas.

    url = "https://www.googleapis.com/drive/v3/files"

    # Parámetros de búsqueda: busca dentro de la carpeta 'parent_id', ignorando la papelera, y
ordena por nombre.

    params = {

        "q": f"'{parent_id}' in parents and trashed=false",

        "orderBy": "name desc",

        "fields": "files(id, name, mimeType)" # Solo pide ID, nombre y tipo (para no gastar
datos de más).

    }

    data = consultar_drive(url, params)

    if data:

        return data.get('files', [])

    return []

def descargar_imagen(file_id):

    # Endpoint para descargar el archivo físico usando ?alt=media.

```

```

url = f"https://www.googleapis.com/drive/v3/files/{file_id}?alt=media"

token = obtener_token()

headers = {"Authorization": f"Bearer {token}"}

response = requests.get(url, headers=headers)

if response.status_code == 200:

    # Convierte los bytes descargados de internet en una imagen usando la librería PIL y la
devuelve.

    return Image.open(io.BytesIO(response.content))

return None

# =====

# 4. SISTEMA DE LOGIN (MANEJO DE ESTADO)

# =====

def check_password():

    def password_entered():

        # Compara lo que el usuario escribió con las credenciales quemadas (admin / utn2026).

        if st.session_state["usuario"] == "admin" and st.session_state["clave"] == "utn2026":

            st.session_state["autenticado"] = True # Guarda en la memoria que ya entró.

            # Borra las contraseñas de la memoria por seguridad.

            del st.session_state["clave"]

            del st.session_state["usuario"]

        else:

            st.session_state["autenticado"] = False

```

```

# Si la variable "autenticado" no existe en la memoria, dibuja el formulario de login.

if "autenticado" not in st.session_state:

    st.markdown("<h2 style='text-align: center;*> 🔒 Acceso Restringido</h2*>",
unsafe_allow_html=True)

    st.text_input("Usuario", key="usuario")

    st.text_input("Contraseña", type="password", key="clave")

    st.button("Ingresar al Sistema", on_click=password_entered,
use_container_width=True)

    return False

elif not st.session_state["autenticado"]:

    st.error("🚫 Credenciales incorrectas.")

    return False

return True # Si todo está correcto, devuelve True para dejarlo pasar al dashboard.

# =====

# 5. RENDERIZADO DEL PANEL DE CONTROL (UI)

# =====

st.markdown("<h1 style='text-align: center;*> 🛡️ Seguridad Perimetral San José</h1*>",
unsafe_allow_html=True)

# Llama a la función de login. Solo si devuelve True, ejecuta el resto del código.

if check_password():

    st.success("Conexión segura establecida con el nodo Edge.")

try:

```

```

st.subheader("📁 Explorador de Evidencias Fotográficas")

st.write("Navegación optimizada mediante API REST y tokens cifrados.")

# --- NAVEGACIÓN EN CASCADA ---

# 1. Busca las carpetas de los MESES dentro de la carpeta raíz.

meses = listar_archivos(CARPETA_RAIZ_ID)

if meses:

    # Crea un menú desplegable con los meses encontrados.

    mes_seleccionado = st.selectbox("📅 Seleccione el Mes:", meses,
format_func=lambda x: x['name'])

    # 2. Busca los DÍAS dentro de la carpeta del mes seleccionado.

    dias = listar_archivos(mes_seleccionado['id'])

    if dias:

        dia_seleccionado = st.selectbox("📅 Seleccione el Día:", dias,
format_func=lambda x: x['name'])

        # 3. Busca las CATEGORÍAS (personas / vehiculos) dentro del día seleccionado.

        categorias = listar_archivos(dia_seleccionado['id'])

        if categorias:

            cat_seleccionada = st.selectbox("🔍 Seleccione la Categoría:", categorias,
format_func=lambda x: x['name'])

            # 4. Busca LAS FOTOS físicas dentro de la categoría.

            fotos = listar_archivos(cat_seleccionada['id'])

```

```

if fotos:

    st.info(f"📷 Se encontraron {len(fotos)} registros en esta carpeta.")

    # Recorre cada foto encontrada en un bucle.

    for foto in fotos:

        # Se asegura de que el archivo sea realmente una imagen (ignora txt o csv).

        if "image" in foto['mimeType']:

            st.markdown(f"***Archivo:** `{foto['name']}`")

            # Muestra un círculo de carga mientras descarga la foto.

            with st.spinner("Descargando evidencia..."):

                img = descargar_imagen(foto['id'])

                if img:

                    # Dibuja la foto en la página ajustándola al ancho de la pantalla.

                    st.image(img, use_container_width=True)

                else:

                    st.error("No se pudo descargar la imagen.")

            st.divider() # Dibuja una línea separadora entre fotos.

        else:

            st.warning("No hay detecciones en esta carpeta.")

    else:

        st.write("No hay categorías para este día.")

else:

    st.write("No hay días registrados en este mes.")

else:

```



```

st.warning("El robot no encuentra la carpeta principal. Verifique permisos de
compartición en Drive.")

except Exception as e:

    # Si algo falla gravemente (ej. caída de internet), atrapa el error para que la página no se
    rompa por completo.

    st.error(f"Error crítico en el sistema: {e}")

```

Estructura de Credenciales y Variables de Entorno (Secrets)

El sistema emplea credenciales cifradas para establecer la comunicación segura con los servicios en la nube de Google Cloud Platform. Por políticas de ciberseguridad y para evitar la exposición de accesos en el documento público, a continuación, se presenta la estructura en formato TOML utilizada en las variables de entorno del servidor, enmascarando las llaves criptográficas privadas.

```

{
  "type": "service_account",
  "project_id": "optical-sight-506716-d3",
  "private_key_id": "9cb0bb9374f7c8bdc848ee973839a7f8c92388b5",

```

"private_key": "-----BEGIN PRIVATE KEY-----

\nMIIEvgIBADANBgkqhkiG9w0BAQEFAASCBAkwggSkAgEAAoIBAQC5up/3V8Mo2U
gN\nt8mP4Irx+e515YuXSQQ/y26kcL0Z6e6PfQIUmrIDuLczyat/eTwQg1KE+6jBsIj7\n8qzE
HPTJFu8gt/np2AiYGTwEZ+S603Lm4diBBhLzVoAcpwqw9EmMDrgz9r7ugGLp\nnmXNCg
WWWh30vDUEcszO/bqC8PphQc1GAvoRErpkc3H+z6ZyI0tKfq0k7hYixiRhsD\nnkJb/IcH8Ea
xChNqQ2tpZ1mX8wFDT4d4yNqvdiEuGraQxmzNMdv/BhU0B86QqjIvp\nWJtXjSVUXuqS
FWUBjj3wuWsnNjdpJdAD+KTB1SLm2+TWC8f8xt2w+PEaueY/VyOJ\n5Sh7i8rVAgMBA
AECggEAF0cNy1DHgFCp5Q9W4jgvcwnBnmep5OodDg/3YQ7mHWVA\nScpUil0CYKK
4jZ3IDRXoHmSlNB29kaTsWcynSIAIxZvs6DYEbJ65YuLOKCHIMoWL\nJDRtQSI dnmD
RFMhw8dbgcyCR9xkxRIbRKwCv4d99wHDIzk+53u8mnIZKW5zBnTfR\nbAsuJDeeZ4IKq
MKmKcHGLCcxYglcpIZhlRMu53iChOdPxmjlRyo2e2FMWE6LhcwZ\n5xJOxp6R89DrNq
C9mbq8GzHkz8KFfu91kmwSe/NWBNAVGMw2PCAxQwrA6AJ4LK/B\n4483N6LWx0zV
n1Y1m70m/imjtVSbqBNbKDgb8U6L6QKBgQDrIXifa8Y2ypUOPBms\nZIEhXEQiDbEah
LjeJojSq1+gNUvyE45ySAa6Xrw5H3f6s5bpWuX2sm2zPbNmDVUo\nISaXTNaIBhXR+EK
oQPUY5gD3Mqj3/+5mUjyUthViWfjljqtWoat+u6x6qesZ750q\nCi21TXZLQG7Ct5EsRgmt
yqxPLQKBgQDJ0xolkX6A23RvHGLcTTSc2c7GVXvSK7+u\nvjM7K/6GWMvUak7PyFsk
2HHmxGr4+64M6fsl9D5HSHPiwMZr/TftUCS79FCF89Cw\nAdgtGogGYq5kl4nr0VyZG4q
jaxAJTIYQajy9PqdCQF9vz5Ll1JV4fA/YiV5kyMpO\nFdnTMzhzSQKBgQDn+2UvXzptg+l
pfgct+j4qMqgo/GCUo3ND1qBPC1B7+1+Qvm87\n2+88IMOqYHGOHsCRg+AFoMhjg3O
gH6rmfFZ7EMAEbDGadKR2+Jgh74Ot6GX46D3C\n5mIfcnn2QnDU5DuWcFbm0jnTBH
WtMYYYK+sDt0pyVF5q6BXdgKIupZnVs9QKBgFEL\naWt+omsCNR5NKtHGWwNFX4g
7WY6LLNzx2cbAIuQ2EhvJWL6NeTQxioOdiktTXZAo\nAUc7birXyFldChDhTS0JysaDF
DbGMp4LD5EFH2xZAii4xZShrOp6qdB3tKfXR9qd\nnnqhr2WR8TxaqJxYNqLwKLZ7JgtM
74ik7Ew6BegCpAoGBANDiIVmYntFG2TLeuFSi\nnjbNlFqjOXzT/Z2LulCdImrqDBbiyKnU
6SgySIOMyCsXlP8TGNGe9dEN/3MjhAD3q\nAYbODIMzitqHPZBOLNxBU2SvJ1H0uiA

```

TWsr2x2bQCS+nkhQ7nMO53itbLrfcNtBA\nIlNURX1ttzz/OlMork8Sqqlw\n-----END
PRIVATE KEY-----\n",

"client_email": "lector-nodo-sanjose@optical-sight-506716-d3.iam.gserviceaccount.com",
"client_id": "105576630770580674687",
"auth_uri": "https://accounts.google.com/o/oauth2/auth",
"token_uri": "https://oauth2.googleapis.com/token",
"auth_provider_x509_cert_url": "https://www.googleapis.com/oauth2/v1/certs",
"client_x509_cert_url": "https://www.googleapis.com/robot/v1/metadata/x509/lector-nodo-
sanjose%40optical-sight-506716-d3.iam.gserviceaccount.com",
"universe_domain": "googleapis.com"
}

```

Link de Github para los códigos , imágenes y videos extras

https://github.com/aldernavarrete2-star/Monitoreo_San-Jose